Universidade do Minho
Escola de Engenharia

Ivan Ribeiro Castro

Controlo da Navegação e Manobras de Acostagem de Manipuladores Móveis Autónomos em Ambientes de Logística Interna nas Linhas de Produção

outubro de 2024

Controlo da Navegação e Manobras de Acostagem de Manipuladores Móveis Autónomos em Ambientes de Logística Interna nas Linhas de Produção

Ivan Ribeiro Castro

UMinho | 2024

Universidade do Minho
Escola de Engenharia

Ivan Ribeiro Castro

Controlo da Navegação e Manobras de Acostagem de Manipuladores Móveis Autónomos em Ambientes de Logística Interna nas Linhas de Produção

Dissertação de Mestrado
Mestrado Integrado em
Engenharia Eletrónica Industrial
e Computadores

Trabalho efetuado sob a orientação do(a)
Doutora Estela Guerreiro Silva da Bicho Erlhagen e
Doutor Luís Filipe Castro Freitas Louro

outubro de 2024

DIREITOS DE AUTOR E CONDIÇÕES DE UTILIZAÇÃO DO TRABALHO POR TERCEIROS

Este é um trabalho académico que pode ser utilizado por terceiros desde que respeitadas as regras e boas práticas internacionalmente aceites, no que concerne aos direitos de autor e direitos conexos.
Assim, o presente trabalho pode ser utilizado nos termos previstos na licença abaixo indicada.
Caso o utilizador necessite de permissão para poder fazer um uso do trabalho em condições não previstas no licenciamento indicado, deverá contactar o autor, através do RepositóriUM da Universidade do Minho.

Licença concedida aos utilizadores deste trabalho

Atribuição
CC BY
https://creativecommons.org/licenses/by/4.0/

Agradecimentos

A conclusão deste projeto marca um momento importante na minha vida, e não poderia deixar de expressar o meu reconhecimento a todas as pessoas que contribuíram para esta jornada.

Em primeiro lugar, gostaria de agradecer aos meus orientadores, Professora Estela Bicho e Professor Luís Louro, pelas suas orientações, paciência, tempo despendido e por todo o apoio incondicional ao longo de todo o processo de desenvolvimento e revisão da dissertação.

A toda a equipa do MARLab, pelo ambiente acolhedor, motivador e ainda pelo constante apoio e disponibilidade prestados.

Estou também grato aos meus colegas do projeto MIAR, cujas discussões e companheirismo tornaram este percurso mais leve.

Gostaria de dedicar um agradecimento especial à minha família, em especial aos meus pais e às minhas irmãs, por serem um exemplo a seguir, por sempre acreditarem em mim e me incentivarem na minha formação.

Agradeço, ainda, o apoio e bons conselhos prestados pelos meus amigos que me acompanharam nesta viagem e, ainda pelos momentos de descontração e pela amizade, que me ajudaram a manter um equilíbrio entre a vida pessoal e académica.

Por fim, agradeço a todos que, de alguma forma, contribuíram para a realização deste trabalho. Cada um de vocês fez parte desta conquista. Muito obrigado a todos!

DECLARAÇÃO DE INTEGRIDADE

Declaro ter atuado com integridade na elaboração do presente trabalho académico e confirmo que não recorri à prática de plágio nem a qualquer forma de utilização indevida ou falsificação de informações ou resultados em nenhuma das etapas conducente à sua elaboração.

Mais declaro que conheço e que respeitei o Código de Conduta Ética da Universidade do Minho.

Resumo

O presente projeto de dissertação, realizado no âmbito do mestrado em Engenharia Eletrónica Industrial e Computadores da Universidade do Minho, está integrado na agenda *Connected Manufacturing* da parceria Bosch & UMinho, no âmbito do Projeto MIAR - "*Flexible system to transport products between shared processes with mobile intelligent autonomous robots (MIAR) synchronized with production lines*". O objetivo deste projeto é desenvolver módulos de navegação para a plataforma Kuka KMP200 OmniMove, promovendo uma solução autónoma para o transporte de materiais.

Com o desafio de manter a competitividade no mercado global, as indústrias precisam otimizar continuamente a sua eficiência operacional. A proposta atende às exigências específicas de ambientes industriais, garantindo maior agilidade e precisão nas operações.

Este projeto propõe uma solução flexível para automação de processos industriais. Desenvolveu-se um sistema autónomo capacitado para efetuar transporte de material, navegando desde do local de carga às rampas de acesso à linha de produção, e realizando manobras de acostagem, em ambientes industriais exíguos. A plataforma utilizada trata-se de um veículo omnidirecional, o que é necessário para permitir movimentos holonómicos devido ao espaço confinado do cenário. Não obstante, o sistema encontra-se preparado para contornar obstáculos ou parar caso não consiga passar por eles.

A arquitetura do sistema foi estruturada em módulos que suportam a execução de vários comportamentos, como a navegação *(Non-Holonomic Nav, Holonomic Nav, Docking)*. Para orquestrar estes comportamentos recorreu-se a um *Task Manager*, que coordena as tarefas solicitadas por um servidor externo *(Service Manager).* De forma a tornar a navegação segura e mais precisa, elaboram-se dois módulos para a perceção do ambiente, o *Obstacle Detection* e o *Positioning*, que são responsáveis pela deteção de obstáculos e por saber a *pose* do veículo, respetivamente.

Por forma a validar a solução, foram realizadas simulações de modelos dinâmicos, em ambientes industriais, com a finalidade de testar diversos cenários, respondendo aos requisitos do projeto.

Palavras-chave: ROS, Veículos Autónomos, Veículos Omnidirecionais, Manobras de Acostagem, Navegação de Veículos, Controlo de Movimentos, Simulação.

Abstract

This dissertation project, carried out as part of the Master's in Industrial Electronic Engineering and Computers at the University of Minho, is integrated into the Connected Manufacturing agenda of the Bosch & UMinho partnership under the MIAR Project – "Flexible system to transport products between shared processes with mobile intelligent autonomous robots (MIAR) synchronized with production lines." The objective of this project is to develop navigation modules for the Kuka KMP200 OmniMove platform, promoting an autonomous solution for material transportation.

Facing the challenge of maintaining competitiveness in the global market, industries must continually optimize their operational efficiency. This proposal addresses the specific requirements of industrial environments, ensuring enhanced agility and precision in operations.

This project proposes a flexible solution for the automation of industrial processes. An autonomous system has been developed, capable of transporting materials from the loading area to the access ramps of the production line while performing docking maneuvers in confined industrial environments. The platform used is an omnidirectional vehicle, which is essential to enable holonomic movements due to the confined space of the setting. Moreover, the system is designed to navigate around obstacles or stop if it is unable to pass through them.

The system's architecture was structured in modules that support the execution of various behaviours, such as navigation (Non-Holonomic Nav, Holonomic Nav, Docking). To orchestrate these behaviours, a Task Manager was used to coordinate the tasks requested by an external server (Service Manager). To make navigation safer and more accurate, two modules were developed for environmental perception, Obstacle Detection and Positioning, which are responsible for detecting obstacles and knowing the vehicle's position, respectively.

To validate the solution, dynamic model simulations were carried out in industrial environments to test various scenarios that met the project's requirements.

Keywords: Ros, Autonomous Vehicles, Omnidirectional Platform, Docking Manoeuvres, Autonomous Navigation, Navigation Control, Simulation

CONTEÚDO

1. Introdução 1
1.1. Enquadramento e Motivação 2
1.2. Objetivos 3
1.3. Estrutura da Dissertação 4
2. Estado da Arte 6
2.1. Revolução Industrial 6
2.2. O que é um Robô 7
2.2.1. Robôs Móveis Industriais 8
2.3. Veículos Autónomos para Tarefas de Transporte 8
2.3.1. AGVs usados para Transportar Paletes em Armazém 10
2.3.2. AGVs Empilhadores 11
2.3.3. AGVS de Reboque 11
2.3.4. AGVs de Superfície Lisa 12
2.3.5. Kiva 12
2.3.6. *Fetch and Freight* 13
2.3.7. Locus 14
2.3.8. Toru 14
2.3.9. Manipulador Móvel Annie 15
2.4. Controlo da Navegação 17
2.4.1. Controlo da Navegação Em Veículos Omnidirecionais 21
2.5. Manobras de Acostagem 23
2.5.1. Manobras de Acostagem Em Veículos Omnidirecionais 24
2.6. Navegação Com Social Aware 27
2.7. Discussão 31
3. Componentes de Hardware e Software 33
3.1. Plataforma Móvel 33
3.1.1. Dimensões da plataforma 33

3.1.2. Rodas Omnidirecionais 35
3.1.3. Sensores *SICK* S300 *Expert* 35
3.2. Softwares 38
3.2.1. ROS 2 – Robot Operating System 2 38
3.2.2. O que é um Node? 38
3.2.3. O que é a comunicação via ROS Topic? 39
3.2.4. O que é a comunicação via ROS Service? 40
3.2.5. O que é a comunicação via ROS Actions? 41
3.2.6. O que é um Package? 42
3.3. Simulador 43
4. Fundamentos Teóricos 49
4.1. *Pose* de um Veículo no Espaço 49
4.2. Sistemas Dinâmicos Não-Lineares 50
4.2.1. Controlo da Orientação de Navegação 52
4.2.2. Controlo da Velocidade Linear 55
4.3. Navegação Holonómica 57
4.3.1. Controlo da Direção do Vetor Velocidade Linear na Navegação Holonómica 58
4.3.2. Controlo da Orientação da Plataforma na Navegação Holonómica 62
4.3.3. Controlo da Velocidade Linear Holonómica 63
4.4. Navegar em Marcha-Atrás 63
4.5. Manobras de Acostagem 64
4.5.1. Controlo da Orientação do Vetor Velocidade nas Manobras de Acostagem 65
4.5.2. Controlo da Orientação nas Manobras de Acostagem 65
4.5.3. Controlo da Velocidade Linear das Manobras de Acostagem 66
4.6. Cinemática Inversa do Veículo 66
5. Arquitetura do Sistema 69
5.1. *Service Manager* 70
5.2. *Environment Perception* 71
5.2.1. *Positioning* 71

5.2.2. *Obstacle Detection* ... 71

5.3. *Movement Controller* ... 72

5.3.1. *Task Manager* ... 72

5.3.2. *Non-Holonomic Navigation* ... 72

5.3.3. Holonomic Navigation ... 73

5.3.4. Docking ... 73

6. Implementação ... 74

6.1. Simulação ... 74

6.2. Posicionamento ... 76

6.3. Deteção de Obstáculos ... 77

6.4. Task Manager ... 84

6.5. Navegação Não-Holonómica ... 86

6.6. Navegação Holonómica ... 88

6.7. Manobras de Acostagem ... 90

7. Ensaios e Resultados Experimentais ... 93

7.1. Ensaio Navegação com Obstáculos Espalhados pelo Caminho ... 93

7.2. Ensaio Navegação Sem Obstáculos pelo Caminho ... 99

7.3. Ensaio Navegação com Operadores Humanos ... 102

7.4. Ensaio Contorno de Obstáculos através da Navegação Holonómica. ... 104

7.4.1. Ensaio Contorno de Obstáculos através da Navegação Holonómica em Espaços Amplos 104

7.4.2. Ensaio Contorno de Obstáculos através da Navegação Holonómica em Espaços Exíguos 105

8. Conclusões e Trabalho Futuro ... 108

8.1. Conclusão ... 108

8.2. Trabalho futuro ... 110

Referências ... 112

Índice de Figuras

Figura 1: Manipulador Móvel Kuka iiwa (KUKA AG, 2020.) 3
Figura 2: Revolução industrial. 6
Figura 3: AGV Porta-Paletes. 11
Figura 4: AGV Empilhadora. 11
Figura 5: AGV de Reboque (Tugger da Toyota). 12
Figura 6: AGV de superfície lisa. 12
Figura 7: Robô Kiva da Amazon (Eugene, 2015). 13
Figura 8: Robôs *Fetch and Freight*. 13
Figura 9: Robôs da Locus Robotics. (a): Locus Origin; (b): Locus Vector; (c): Locus Max. 14
Figura 10: TORU Robot. 15
Figura 11: Manipulador Móvel ANNIE, Fraunhofer IFF (Fraunhofer Institute for Factory Operationand Automation IFF, n.d.). 15
Figura 12: Gerações dos robôs Care-O-bot. (a): Care-O-bot I; (b): Care-O-bot II; (c): Care-O-bot III; (d): Care-O-bot IV. 16
Figura 13: Manipulador Móvel *OmniMan*: Braço Robótico UR5 numa plataforma móvel com rodas *Mecanum*. 21
Figura 14: Robô Omnidirecional MKR (*Muratec Keio Robot*). 22
Figura 15: (a) - Robô desenvolvido; (b) - *AprilTag*. 24
Figura 16: Sistema da Cadeira Inteligente (X. Li et al., 2018). 25
Figura 17: Esquemático do *docking* da plataforma (Dong et al., 2023). 26
Figura 18: Aplicações dos recursos visuais no contexto da colaboração humano-robô (Mukherjee et al., 2022). 28
Figura 19: Fatores de controlo de segurança física para a colaboração humano-robô (Mukherjee et al., 2022). 30
Figura 20: KUKA *Mobile Plataform* 200 *OmniMove* (*KUKA AG*, 2020.). 33
Figura 21: Representação da plataforma móvel e das respetivas dimensões (Comprimento - Largura – Altura,). 34
Figura 22: Roda *Mecanum* utilizado pelo KMP 200 *OmniMove* (*KUKA AG*.). 35
Figura 23: Sensor laser *Sick* S300 *Expert* (*Sick / Sensor Intelligent*, n.d.). 36

Figura 24: Demonstração da gama de varredura, resolução angular e alcance máximo do Sensor *Sick* S300 *Expert*. 37
Figura 25: Funcionamento dos Nodes. 39
Figura 26: Funcionamento da comunicação via ROS *topic*. 39
Figura 27: Estrutura do *topic*. 40
Figura 28: Funcionamento da comunicação via ROS Service. 41
Figura 29: Formato utilizado pelos ficheiros .srv. 41
Figura 30: Funcionamento da comunicação via ROS Actions. 42
Figura 31: Formato utilizado para os ficheiros .action. 42
Figura 32: Exemplo da estrutura de um *workspace*. 43
Figura 33: Biblioteca de objetos de cenário suportados pelo *CoppeliaSim*. 45
Figura 34: Propriedades dos objetos cenários do *CoppeliaSim*. 45
Figura 35: Localização da plataforma em um plano bidimensional. 50
Figura 36: Sistema a controlar. 52
Figura 37: Dimensionamento do vetor velocidade linear bidimensional. 59
Figura 38: Representação da orientação de navegação, que é resultante da soma da orientação do robô com a orientação do vetor velocidade resultante. 60
Figura 39: Arquitetura do sistema, composta pelos módulos: Service Manager, Environment Perception e Movement Controller. 69
Figura 40: Conteúdo do tipo de mensagem que é estabelecida entre o Service Manager e o Movement Controller (task Manager). 70
Figura 41: Estrutura e descrição da mensagem *miar_msgs::msg::GraphNode*. 70
Figura 42: Cenário da linha do ambiente industrial. 74
Figura 43: Cenário elaborado no *CoppeliaSim* para demonstrar o funcionamento do projeto. 75
Figura 44: Fluxo da aquisição da *pose* do veículo. 76
Figura 45: Definição da mensagem */geometry_msgs::msg::Pose2D*. 77
Figura 46: Estrutura da mensagem */sensor_msgs::msg::LaserScan*. 77
Figura 47: Máquina de estados que sustenta o módulo *Obstacle Detection*. 78
Figura 48: Estrutura e descrição da mensagem / *miar_msgs::msg::DistanceSector*, que é a que comunica entre o *Obstacle Detection* e as componentes de navegação. 79

Figura 49: Estrutura e descrição da mensagem */miar_msgs::msg::ProtectDistance*, que comunica com os módulos do controlo de movimentos e, contém as distâncias lineares entre o veículo e os obstáculos. 79
Figura 50: Método de conversão dos dados do referencial do sensor para o referencial do veículo e de interesse à navegação. 80
Figura 51: Setores de interesse consoante a marcha. 81
Figura 52: Representação dos limites entre secções: Regiões Frontal, Traseira, Lateral Direita e Lateral Esquerda. 82
Figura 53: Fluxograma do algoritmo executado pelo *Task Manager*. 85
Figura 54: Estrutura e descrição do tipo de mensagem que estabelece a comunicação entre o *Task Manager* e o *Service Manager*. 86
Figura 55: Estrutura e descrição da *action Movement* que comunica entre o *Task Manager* e o *Non Holonomic Nav.* 86
Figura 56: Estrutura e descrição da mensagem que contém a rota desejada, bem como as caraterísticas a seguir para a navegação. 87
Figura 57: Máquina de Estados do comportamento de navegação não-holonómica. 88
Figura 58: Estrutura e descrição da *action* que comunica entre o *Task Manager* e o *Holonomic Nav.*. 89
Figura 59: Máquina de estados que define o comportamento da *Holonomic Nav.* 90
Figura 60: Estrutura e descrição da *action Docking* que comunica bidireccionalmente entre o *Docking* e o *Task Manager*. 91
Figura 61: Máquina de estados que define o comportamento das manobras de acostagem. 92
Figura 62: Cenário exemplo do ensaio com obstáculos. Apresenta a localização inicial do robô, das rampas de abastecimento, dos pontos de *docking*, dos obstáculos e do supermercado. 94
Figura 63: Navegação Não-Holonómica de marcha atrás a contornar um obstáculo. Analisar o Vídeo no Youtube. 94
Figura 64: *Docking* para replicar o ir buscar caixas de material. 95
Figura 65: Navegação Holonómica capacitada de evitar colisões com obstáculos. 96
Figura 66: Manobras de Acostagem na rampa de abastecimento. 97
Figura 67: Navegação Não-Holonómica no corredor principal evitando colisões com obstáculos. 97
Figura 68: Navegação Holonómica no corredor superior evitando colidir com obstáculos. 98
Figura 69: Manobras de Acostagem no corredor superior. 98
Figura 70: *Docking* na zona do *supermarket*. 99

Figura 71: Trajetória realizada no ensaio sem obstáculos. A ordem dos acontecimentos é de (a) até (s). Analisar Vídeo Ensaio sem Obstáculos. 100
Figura 72: Cenário exemplo do ensaio com operadores humanos em movimento. Apresenta a localização inicial do robô, das rampas de abastecimento, dos pontos de docking, dos operadores (em movimento e parado) e do supermercado.Analisar Vídeo Navegação Segura coexistindo com Operadores. 102
Figura 73: Trajetória realizada no ensaio com a presença de operadores humanos em movimento e estático. 103
Figura 74: Cenário exemplo da simulação de contorno de obstáculos através da navegação holonómica, com orientação fixa, em espaço amplo. Analisar Vídeo Desvio Obstáculo em Navegação Holonómica. 104
Figura 75: Ensaio de contorno de obstáculos através da navegação holonómica, em espaços amplos. 105
Figura 76: Cenário exemplo para o ensaio de contorno de obstáculos a navegar de forma holonómica, com a orientação fixa, em espaços exíguos. Analisar Vídeo da Colisão em Navegação Holonómica a desviar de obstáculos em espaços exíguos. 106
Figura 77: Ensaio de contorno de obstáculos através da navegação holonómica, em espaços exíguos. 107

Índice de Tabelas

Tabela 1: Comparação Campos Potenciais VS Atractores Não Lineares. 20

Tabela 2: Tipos de métodos de comunicação homem-robô mais utilizados em sistemas HRC (Zhang et al., 2023). 29

Tabela 3: Diferentes tipos de riscos potenciais de HRC(Zhang et al., 2023). 29

Tabela 4: Características da plataforma móvel KUKA *Mobile Plataform* 200 *OmniMove* (*KUKA AG*, n.d.).. 34

Tabela 5: Características do Sensor *laser Sick* S300 *Expert* (*Sick / Sensor Intelligent*, n.d.). 36

Tabela 6: Comparação dos tempos dos Ensaios com e Sem Obstáculos. 101

Lista de Abreviaturas, Siglas e Acrónimos

MIAR	Mobile Intelligent Autonomous Robots (Robôs Móveis Autónomos Inteligentes)
AGVs	Automated Guided Vehicles (Veículos Guiados Automatizados)
AMRs	Autonomous Mobile Robots (Robôs Móbeis Autónomos)
AUVs	Autonomous Underwater Vehicles (Veículos Subaquático Autónomos)
DOFs	Degrees of Freedom (Graus de Liberdade)
E-Commerce	Electronic Commerce (Comércio Eletrónico)
GPS	Global Positioning System (Sistema de Posicionamento Global)
HRC	Human Robot Colaboration (Colaboração Humano-Robô)
IMU	Inertial Measurement Unit (Unidade de Medição Inercial)
MIAR	Mobile Intelligent Autonomous Robots (Robôs Móveis Inteligentes e Autónomos)
NLAD	Nonlinear Attractor Dynamics (Dinâmica de Atractores Não Lineares)
SDNL	Sistema Dinâmico Não Linear
PFM	Potential Field Method (Método dos Campos Potenciais)
ROS	Robot Operating System (Sistema Operativo de Robôs)
SLAM	Simultaneous Localization and Mapping (Mapeamento e Localização Simultâneos)
KMP	Kuka Mobile Plataforma (Plataforma Móvel Kuka)
KMR	Kuka Mobile Robot (Robô Móvel Kuka)
iiwa	Intelligent industrial work assistant (Assistente Inteligente de Trabalho Industrial)

1. INTRODUÇÃO

Com os avanços a nível tecnológico e industrial, tem-se presenciado uma constante transformação nas organizações de modo a manterem a competitividade no mercado. As mesmas têm reformulado a sua estratégia, para alcançar uma maior eficácia e eficiência nos seus processos.

A mais recente evolução industrial visa transformar o paradigma das organizações ao criar fábricas inteligentes. Assim, a engenharia aliada à tecnologia permite melhorar vários processos das organizações, nomeadamente a criação de fluxos de informação automatizados, aumentando assim o seu conhecimento e, consequentemente, apoiando a tomada de decisão (Mokyr & Strotz, 1998a).

Existem diversas operações nas organizações que não acrescentam valor para o cliente, que são denominados "desperdícios". De entre os oito desperdícios (esperas, transporte, movimentações, stock, defeitos, excesso de processamento, sobreprodução, conhecimento das pessoas), o foco incide sobre esperas dos materiais, as movimentações das pessoas (a forma atual de transportar os materiais é feita manualmente por uma pessoa denominada de *POUP*), o transporte desnecessário do material (sendo que o *POUP* leva um *trolley* de material e vai distribuindo para responder à necessidade que observa) (Ohno, 1998).

O transporte de materiais sofreu avanços significativos ao longo do tempo, acompanhando as revoluções industriais. Inicialmente, a produção era, predominantemente manual, sendo que todas as tarefas eram realizadas de forma artesanal. Com a revolução industrial é agora possível uma integração dos sistemas, operando com análise de dados em tempo real, inteligência artificial, possibilitando assim uma automação avançada e até a tomada de decisões de forma autónoma (Davies, 2015).

A tecnologia continua a evoluir, impulsionando a eficiência, precisão e a adaptabilidade dos processos de fabricação. Perante isso, de modo a responder aos desafios, as estratégias apontam para a introdução de mecanismos automatizados, por exemplo o fluxo eficiente de materiais, em ambiente fabril. Por outras palavras, as matérias-primas devem ser entregues no momento certo, sempre que determinado pelo plano de produção, e os produtos acabados devem ser recolhidos em determinado tempo. Não obstante, estes sistemas permitem a rastreabilidade dos materiais. A rastreabilidade refere-se à capacidade de monitorizar e registar em tempo real o percurso, localização e estado de cada caixa de matéria-prima transportada pelo robô móvel. Com a implementação de um sistema autónomo, cada movimento, desde

a recolha até à entrega das caixas nas rampas de abastecimento, é automaticamente registado, garantindo um acompanhamento preciso de todas as operações. Esta tarefa tornando-se autónoma, permite aumentar a capacidade de fabricação e a possibilidade de realocar os operadores em outros sectores ou tarefas menos entediantes e repetitivas, isto é, tarefas de maior valor.

1.1. ENQUADRAMENTO E MOTIVAÇÃO

O presente projeto de dissertação está integrado na agenda *Connected Manufacturing* da parceria Bosch&UMinho - Projeto MIAR- "Flexible system to transport products between shared processes with mobile intelligent autonomous robots (MIAR) synchronized with production lines".

Portanto, no projeto MIAR pretende-se desenvolver uma frota flexível de manipuladores móveis autónomos, transportadores de caixas com matéria-prima para abastecimento de linhas de montagem em ambientes de chão de fábrica. De modo a tornar possível o controlo da frota é necessário, primeiramente, garantir a capacidade de navegação autónoma dos veículos em ambientes dinâmicos, ou seja, partilhando o espaço com operadores humanos e outros veículos.

É importante descrever o processo atual de transporte de caixas de matéria-prima para contextualizar as melhorias que o projeto MIAR procura implementar. Atualmente, existe um sistema parcialmente automatizado que realiza o transporte de matéria-prima e produtos desde a zona de armazém até aos *buffers* de armazenamento intermédios. No entanto, o processo subsequente, que vai desde esses *buffers* até às rampas de abastecimento das linhas de montagem, é feito manualmente pelos *POUPs* (operadores), resultando em ineficiências e maior dependência da intervenção humana.

Neste cenário, a escolha de robôs móveis omnidirecionais não se deve apenas à flexibilidade necessária para a replicação e adaptação a novos *layouts*, mas também à necessidade de lidar com o *layout* exíguo da fábrica. Os ambientes de produção com espaços limitados, onde ocorrem deslocamentos simultâneos de operadores humanos e veículos, exigem uma solução que possibilite manobras precisas e movimentação eficiente em todas as direções. Os robôs omnidirecionais, Figura 1, são ideais neste contexto, pois oferecem uma manobrabilidade superior em comparação com veículos diferenciais, sendo capazes de se movimentar lateralmente e realizar acostagens precisas em espaços congestionados.

Figura 1: Manipulador Móvel Kuka iiwa (KUKA AG, 2020.)

Adicionalmente, cada robô móvel será equipado com um manipulador (como é o caso do braço apresentado na Figura 1), dotando-o da capacidade de carregar e descarregar caixas, aumentando a sua autonomia e, por isso, diminuindo ainda mais a necessidade de intervenção humana. Os veículos deslocam-se até às ilhas (*buffers* de armazenamento intermédio) onde se encontram as caixas previamente transportadas pelos veículos da logística interna da zona de armazém, recolhe e transporta um conjunto de caixas para as diversas rampas de carregamento da linha de montagem. Ao colocar uma caixa na zona de carregamento o robô deve verificar se existem caixas vazias para devolução à ilha.

O presente documento apresenta uma proposta de solução, que se concentra na utilização de uma plataforma móvel omnidirecional, parte integrante de um manipulador móvel, que além de executar tarefas de manipulação e de transporte, executa-as em certos períodos, com flexibilidade e eficiência. Em mais detalhe, foi abordado o transporte, a acostagem e o controlo em ambientes dinâmicos e não estruturados, considerando a presença de operadores humanos, e a capacidade de operar em espaços congestionados.

Posto isto, foi estimulada a procura de soluções inovadoras que possam melhorar o comportamento dos veículos omnidirecionais, permitindo uma navegação autónoma, com uma dependência mínima de infraestrutura, permitindo fluxos eficiente de transporte autónomos, possibilitando até o mapeamento e a localização do veículo.

1.2. OBJETIVOS

No âmbito deste projeto, o principal objetivo desta dissertação incidiu sobre o controlo do movimento da plataforma móvel omnidirecional que faz parte do manipulador móvel usado no projeto, concretamente na navegação do veículo entre a zona de carregamento de caixas até às rampas de abastecimento das linhas de montagem e na tarefa de acostagem à zona de carregamento ou descarregamento de caixas.

O presente trabalho segue e expande o trabalho realizado por Correia (2023), que surgiu numa fase inicial do projeto e serviu de base para o desenvolvimento de várias tarefas. Algumas soluções e abordagens propostas por Correia foram mantidas, especialmente no que diz respeito ao controlo básico do movimento da plataforma e à perceção do ambiente. No entanto, a dissertação atual introduziu melhorias em alguns aspetos, ajustando a lógica de navegação para uma maior precisão e eficiência nas manobras de acostagem, além de otimizações nos algoritmos de controlo para se movimentar em espaços exíguos. Ademais, fez-se uma migração da *framework* usada no projeto de *ROS* para *ROS2.*

Estas melhorias foram implementadas com o intuito de atender às necessidades atuais do projeto, numa fase em que a robustez e a eficiência do sistema são cruciais para o sucesso da operação nas linhas de montagem. Assim, as principais atividades realizadas incluem:

- Familiarização com o Robot Operating System (ROS), aprendendo os conceitos básicos do ROS2, migrando os módulos já desenvolvidos em ROS para ROS2;
- Desenvolvimento dos métodos e algoritmos que permitam a navegação segura em ambientes complexos e exíguos, isto é, garantindo que os veículos sejam capazes de navegar em ambientes dinâmicos com partilha do espaço com operadores humanos e outros veículos;
- Desenvolvimento dos módulos de controlo e execução das manobras de acostagem, necessárias para a aproximação à linha de produção e às ilhas de armazenamento intermédio;

1.3. Estrutura da Dissertação

O presente documento apresenta-se estruturado em oito capítulos, apresentados seguidamente.

A introdução e os objetivos estão descritos neste primeiro capítulo, assim como o enquadramento do projeto.

No segundo capítulo, realizou-se um estado de arte sobre veículos que são usados para tarefas de transporte em ambiente fabril e as várias formas de controlo utilizando movimentos holonómicos e controlo de movimentos não-holonómicos em ambientes dinâmicos e congestionados. Foram ainda apresentadas diferentes estratégias para as manobras de acostagem.

O terceiro capítulo apresenta uma descrição da plataforma móvel escolhida para o projeto, sendo descritas as dimensões da mesma, quais as rodas que possui e, que elementos constituem para sensorização. Ainda, no que diz respeito ao terceiro capítulo, fez-se um estudo acerca dos *softwares* que

foram utilizados, dando realce à utilização da *framework ROS2* e a uma breve explicação do seu funcionamento e das suas funcionalidades e do simulador *CoppeliaSim*.

Relativamente ao quarto capítulo, foi realizado um estudo alargado dos algoritmos implementados para o controlo de movimentos, marcha à frente e marcha atrás, desvio de obstáculos e operadores, manobras de acostagem e técnica de localização.

Relativamente ao quinto capítulo, foi apresentada a arquitetura do sistema desenvolvido, bem como uma descrição detalhada da arquitetura e de todos os módulos que fazem parte da mesma.

O sexto capítulo diz respeito ao capítulo da implementação, o qual aborda qual cenário foi desenvolvido e como foram implementados os módulos do posicionamento, deteção de obstáculos e do controlo de movimentos (Navegação Não-Holonómica, Navegação Holonómica e Manobras de Acostagem).

No decorrer do sétimo capítulo, serão apresentados os resultados experimentais das simulações, bem como uma análise aos resultados obtidos nas diferentes condições e ambientes de simulação.

No oitavo e último capítulo, está exposto um resumo sobre a solução proposta, foram discutidos os resultados obtidos, de forma a apresentar as principais conclusões sobre o trabalho realizado e, terminando com as considerações finais relativas ao trabalho realizado, assim como as oportunidades de trabalho futuro.

2. Estado da Arte

Neste capítulo, são apresentados os principais tópicos abordados no estado da arte, fundamentais para o desenvolvimento deste trabalho, com uma análise detalhada das soluções existentes e das abordagens mais relevantes na área.

2.1. Revolução Industrial

A indústria tem sido aprimorada ao longo da história da humanidade, e a introdução de novas tecnologias associadas a ela teve um impacto significativo na inovação e na transformação. Estas mudanças são frequentemente designadas como "Revolução Industrial". Na Figura 2, é possível verificar que a Revolução Industrial se encontra dividida em cinco períodos da história.

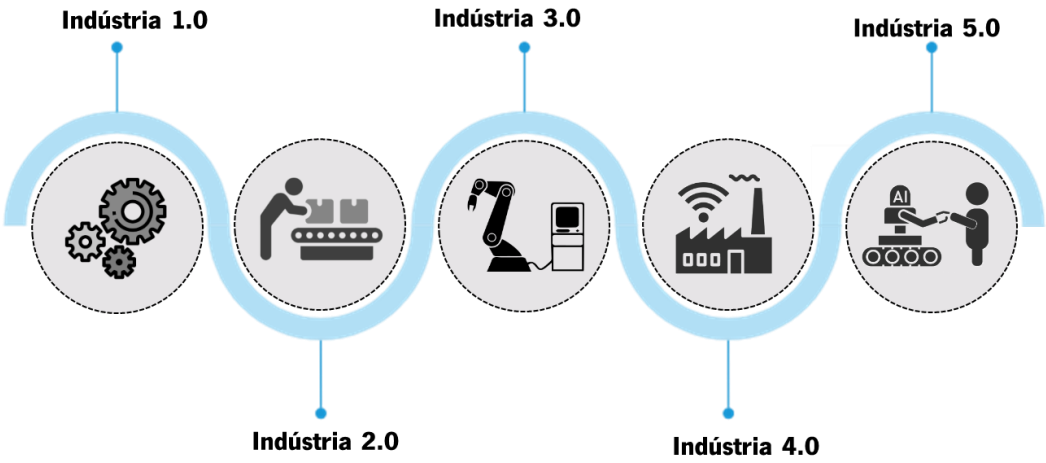


Figura 2: Revolução industrial.

A 1ª revolução (Indústria 1.0) surge no final do século XVIII, representando a transição da produção manual para a produção mecânica baseada em máquinas a vapor, usando a água e o vapor (Iqbal & Yadav, 2021).

No início do século XX surge a 2ª revolução, denominada Indústria 2.0. Esta revolução foi marcada pela utilização de eletricidade como principal fonte de energia (Mokyr & Strotz, 1998b). De salientar, outro grande marco nesta revolução, foi a criação de linhas de montagem e, consequentemente, a produção em massa, que havia sido projetada por Henry Ford, conseguindo uma produção mais rápida e eficiente, através da divisão de tarefas em etapas específicas (Crowther & Ford, 1924; Frederick Winslow, 2004).

A terceira revolução (Indústria 3.0) surge na década de 70 do século XX, consequente de desenvolvimentos tecnológicos que permitiram a produção automatizada e informatizada, tal impulsionou a utilização de robots via *Programmable Logic Controllers* (Mpofu & Nicolaides, 2019).

A Indústria 4.0 é impulsionada pela descoberta e introdução de tecnologias emergentes, como robôs autónomos, manufatura aditiva, computação em nuvem, simulação e integração de sistemas. O principal objetivo desta revolução consiste em conectar a informação de todas as atividades produtivas, permitindo uma interação em tempo real. Esta nova era da produção é uma continuidade das revoluções anteriores, que já automatizaram partes repetitivas do processo produtivo (Viswanadham, 2002). Assim, a combinação de maquinaria com avanços tecnológicos possibilitou o desenvolvimento da robótica e da automação, permitindo a realização de tarefas complexas e precisas com eficiência e alta exatidão (Wallén, 2008).

Apesar de, globalmente, as organizações ainda não terem implementado por completo a Indústria 4.0, está a surgir uma nova revolução (Alves et al., 2023).

A mais recente revolução, apelidada Indústria 5.0, surge em meados do ano 2020, e assenta na colaboração entre humano-robô, promovendo a partilha das melhores habilidades de cada um, de forma atingir uma maior rapidez na execução das tarefas. Ou seja, é uma revolução focada no ser humano, em que combina as tecnologias, as organizações e os humanos, de forma a criar sistemas mais sustentáveis e resilientes, que por sua vez aumentem a produtividade e a eficácia nas operações (Ivanov, 2023).

2.2. O QUE É UM ROBÔ

Há muitos séculos, que as pessoas têm vindo a desenvolver mecanismos para imitar partes do corpo humano. Estes mecanismos têm sido melhorados com o decorrer dos tempos com o desenvolvimento da tecnologia. A estes mecanismos denominamos robôs. A palavra robô deriva do checo *robota*, que significa trabalho penoso.

Com a evolução dos robôs na sociedade, Isaac Asimov apresentou três leis fictícias que servem como ponto de partida para discutir a ética e a responsabilidade no desenvolvimento da robótica. Porém, estas não são aplicadas no fabrico de robôs atualmente (Salge & Polani, 2017).

Além das três leis originais, Asimov posteriormente introduziu uma Lei Zero, que estabelece um princípio mais abrangente. As leis são as seguintes:

1. *Um robô não pode ferir um ser humano ou, por inação, permitir que um ser humano sofra algum mal.*

2. *Um robô deve obedecer às ordens dadas por seres humanos, exceto nos casos em que tais ordens entrem em conflito com a Primeira Lei.*

3. *Um robô deve proteger sua própria existência, desde que tal proteção não entre em conflito com a Primeira ou Segunda Lei.*

0. *Um robô não pode causar dano à humanidade ou, por inação, permitir que a humanidade sofra algum mal.*

2.2.1. Robôs Móveis Industriais

Um robô móvel, tal como o nome indica, é um dispositivo que consegue mover-se autonomamente de um lugar para o outro, de forma a alcançar um objetivo (S. G. Tzafestas, 2014). Estes, têm vindo a revelar-se uma peça essencial nas soluções de logística interna das indústrias.

A introdução desses veículos nos meios industriais, possuiu alguns requisitos, nomeadamente no que concerne aos critérios para a adoção, a performance, a interação Humano-Robô e a segurança. De entre os diferentes critérios, salienta-se o fator segurança, que possuiu um valor significativo para a inclusão destes veículos, dada a partilha do ambiente com humanos. De modo a aumentar a segurança, os veículos devem possuir medidas adicionais, como sensores, de modo a poder evitar obstáculos e colisões com os operadores, permitindo uma navegação segura (Zhang et al., 2023)

2.3. Veículos Autónomos para Tarefas de Transporte

Dentro das organizações, quando existem vários pontos de recolha e entrega distantes, é comum recorrer-se ao uso de veículos de transporte de materiais, de modo a facilitar a deslocação dos mesmos.

Nos primórdios da indústria, nomeadamente em ambientes de armazéns e centros de distribuição, eram utilizados veículos industriais manuais, como empilhadoras e outros veículos de transporte operados por humanos, possuindo um papel fundamental para o transporte de cargas e materiais. No que diz respeito aos veículos impulsionados pela força do operador, estes apresentam versatilidade e são bastante utilizados para a movimentação de cargas em espaços confinados, ou áreas onde outros veículos de maior dimensão não conseguem ter acesso.

A par das revoluções, surgem os veículos de transporte de carga motorizados. Aqui, em vez da força física do operador, surgem os motores, que facilitam o transporte de cargas mais pesadas ou a movimentação em maiores distâncias.

No que concerne a tarefas de logística, transporte e manuseio, é possível identificar diversas soluções industriais deste tipo de equipamentos, sendo que a maior parte delas recorre a veículos que são operados manualmente (Porunov & Trifonov, 2020). Os mais utilizados são os capazes de transportar grandes quantidades, como é o caso dos empilhadores.

A evolução digital projetou o aumento do *E-Commerce* (Laudon & Traver, 2021), o que obriga as organizações a evoluírem e adaptarem-se mais rapidamente. Este aumento requer uma maior exigência e eficiência nas operações de armazenamento e distribuição de mercadorias. Isto inclui o possível redesenho do *layout* dos armazéns, uso de sistemas de gestão de *stock* em tempo real, métodos de recolha mais eficientes, otimização da logística e transporte das mercadorias.

Perante estes cenários, os robôs móveis têm vindo a ser uma aposta em diversas tarefas industriais, poupando assim, o esforço físico dos operadores. A nível industrial existem dois grandes grupos de robôs móveis, os AGVs (*Automated Guided Vehicles*) e os AMRs (*Autonomous Mobile Robots*), cada um com características específicas, mas onde se encaixam a maioria dos robôs do mercado (Shneier & Bostelman, 2015).

1. AGVs: operam seguindo caminhos pré-definidos, não são totalmente autónomos, podendo ser controlados por fitas magnéticas no chão, lasers ou sistemas de visão.

2. AMRs: operam de forma autónoma, sem necessidade de infraestrutura, recorrendo ao uso de sensores, câmaras e algoritmos para uma operação segura.

Os AGVs são, por definição, veículos autónomos que seguem trajetos pré-definidos, utilizando tecnologias como fitas magnéticas, lasers ou sistemas de navegação baseados em marcas no chão (Ganesharajah et al., 1998). Ao contrário de outros tipos de robôs móveis, como os AMRs, os AGVs têm uma capacidade limitada de adaptação a mudanças no ambiente em tempo real, sendo mais adequados para ambientes estruturados e previsíveis. Estes cumprem rigorosas normas de segurança europeias, como a deteção de colisões, a utilização de sinais sonoros e luminosos, e a existência de um controlo manual para paragem de emergência, assegurando um funcionamento seguro em ambientes industriais (António da Silva Santos, 2013).

No entanto, importa salientar que os AGVs podem não ser tão eficientes quanto os veículos autónomos (AMRs), principalmente em termos de integração perfeita com o chão de fábrica e na libertação dos operadores para tarefas de maior valor. Os veículos autónomos, AMRs, oferecem uma abordagem diferente face aos AGVs, sendo esta baseada em algoritmos e sistemas de inteligência artificial para a navegação e para a tomada de decisão. Assim, estes são projetados para operar, dando uso a ferramentas como sensores, câmaras, algoritmos de mapeamento e aprendizagem (redes neuronais) para decidir em tempo real e navegar de forma segura em ambientes dinâmicos.

A nível de integração de chão de fábrica, os AMRs oferecem mais eficiência, dado que não dependem de supervisão constante do operador para o seu funcionamento, permitindo assim uma operação contínua e autónoma. Outro fator importante, passa pela libertação do operador para tarefas de maior valor, isto é, tarefas que se concentrem em funções que exijam habilitações específicas, aumentando assim a eficiência operacional.

Não obstante, os veículos autónomos possuem muitos desafios, nomeadamente em situações inesperadas ou em ambientes altamente dinâmicos. Isto requer mais exigência e preparação dos veículos para as situações que vão enfrentar. Por conseguinte, as questões de segurança e confiabilidade assumem um papel fundamental para a inserção destes veículos em organizações, pois as operações dos mesmos requerem sistemas robustos que lidem com diferentes cenários, de modo a garantir a segurança tanto do equipamento quanto das pessoas que o rodeiam.

2.3.1. AGVs usados para Transportar Paletes em Armazém

Os AGVs porta-paletes, Figura 3, são utilizados para movimentar paletes ao nível do solo, transferindo materiais dos armazéns para as áreas de produção ou outras localizações desejadas. Estes veículos podem operar com carregamento manual ou automático. No método manual, um operador conduz o veículo até a palete, acopla e, em seguida, liberta o AGV para continuar a sua trajetória. Em contrapartida, no modo automático, o veículo desvincula-se automaticamente da guia e acopla-se à palete sem intervenção direta do operador (Tomé de Andrade Silva, 2015).

Figura 3: AGV Porta-Paletes.

2.3.2. AGVs Empilhadores

Os AGVs empilhadores, exemplificado na Figura 4, são semelhantes fisicamente a uma empilhadora normal. No entanto, estes movimentam paletes tanto a nível do solo como em alturas, permitindo o acesso a estantes (Amaral, 2022). Alguns possuem ainda, mais que um garfo para poder transportar mais que uma palete de cada vez.

Figura 4: AGV Empilhadora.

2.3.3. AGVS de Reboque

Estes veículos possuem uma unidade central motorizada ligada a vários reboques. São utilizados quando há necessidade de múltiplos reboques com elevada carga, tornando a sua movimentação mais complexa, visto que precisa de um grande espaço para se manobrar (Tomé de Andrade Silva, 2015). A Toyota desenvolveu um veículo deste tipo, *tugger*, como é demonstrado na Figura 5.

Figura 5: AGV de Reboque (Tugger da Toyota).

2.3.4. AGVs de Superfície Lisa

Este tipo de AGVs são os mais utilizados para o transporte de grandes volumes de material, tendo uma elevada capacidade de manobra em espaços pequenos, face aos AGVs de reboque (Amaral, 2022). Um exemplo desse veículo está presente na Figura 6: AGV de superfície lisa.

Figura 6: AGV de superfície lisa.

2.3.5. Kiva

A integração em chão de fábrica de robôs móveis, assume um papel importante. Desta forma, torna-se possível criar um ambiente em que os armazéns sejam totalmente operados por diversos veículos autónomos, minimizando ou até eliminando a necessidade de intervenção humana. Um exemplo deste conceito é o sistema de gestão de armazéns Kiva, descrito por Wurman et al. (2008). A Figura 7 ilustra um robô Kiva.

Figura 7: Robô Kiva da Amazon (Eugene, 2015).

Os robôs Kiva são veículos autónomos omnidirecionais, equipados com sensores e elevadores, que se movem debaixo de estantes com prateleiras de armazenamento. A sua introdução permitiu organizar as prateleiras de forma estratégica para alocar os produtos, no entanto, necessita ainda de colaboradores para colocar as mercadorias nas prateleiras. O *software* que utilizam, permite gerir o movimento dos robôs, através da localização dos mesmos, e dos produtos a recolher.

2.3.6. *Fetch and Freight*

De modo a evitar a intervenção humana, têm vindo a surgir uma série de soluções no mercado. Exemplos disso são os robôs heterogéneos *Fetch and Freight* (Wise et al), presentes na Figura 8.

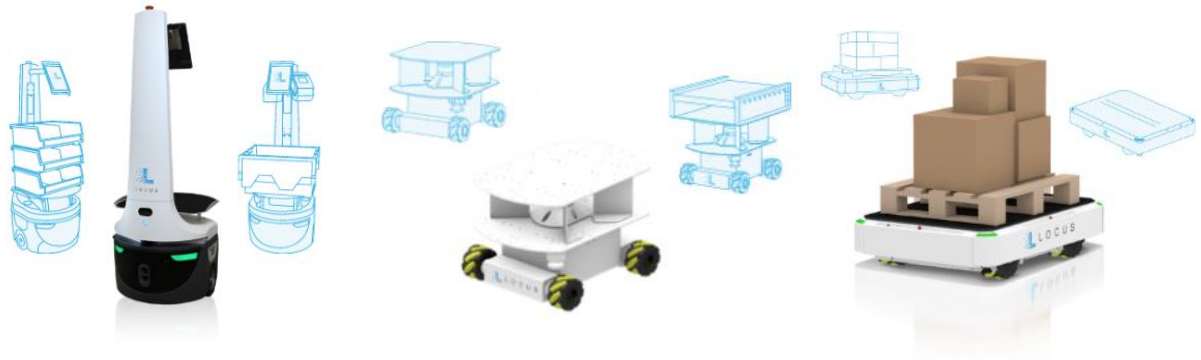
Figura 8: Robôs *Fetch and Freight*.

Estes robôs possibilitam a execução de tarefas de recolha e transporte, de forma coordenada, com uma navegação segura, evitando obstáculos ao detetá-los. O manipulador *Fetch* tem 7 DOFs, suporta cargas até 6 Kg, e possui uma movimentação autónoma livre de colisões. Para ser um trabalho colaborativo

entre robôs e humanos, o robô *Freight* está encarregue do transporte da mercadoria para o local pretendido.

2.3.7. Locus

Soluções para trabalho colaborativo com humanos, são apresentados pela *Locus Robotics*. Esta desenvolve sistemas para a recolha ou entrega de pedidos em armazém, recorrendo à utilização de diferentes robôs autónomos, como mostra a Figura 9, trabalhando de forma síncrona com os funcionários, de modo a acelerar o processo de preparação dos pedidos (Robotics, 2020).



(a) (b) (c)

Figura 9: Robôs da Locus Robotics. (a): Locus Origin; (b): Locus Vector; (c): Locus Max.

Estes oferecem uma vasta gama de funções, como o transporte em compartimentos, a entrega em depósitos, a possibilidade de transportar cargas mais pesadas e o transporte de variados tipo de materiais, como embalagens ou paletes.

2.3.8. Toru

Em Bogue (2016) são apresentados vários tipos de robôs móveis, além dos já mencionados anteriormente. Um exemplo europeu apresentado é o TORU robot, apresentado na Figura 10, cujo objetivo é retirar artigos das prateleiras de armazenamento e entregá-los em locais específicos, como estações de expedição.

Figura 10: TORU Robot.

2.3.9. MANIPULADOR MÓVEL ANNIE

O robô de assistência ANNIE, criado no Fraunhofer IFF, *(Fraunhofer Institute for Factory Operation and Automation IFF)*, ilustrado na Figura 11, é um manipulador móvel projetado para ser usado em tarefas industriais e comerciais. O principal foco está nas atividades logísticas, como a recolha e o transporte, assim como nas operações e na interconexão entre máquinas. Além dessas tarefas, são também possíveis tarefas de assistência, inspeção e monitorização.

Figura 11: Manipulador Móvel ANNIE, Fraunhofer IFF (Fraunhofer Institute for Factory Operationand Automation IFF, n.d.).

De modo a desenvolver um robô móvel para assistir pessoas em casa, Fraunhofer IPA, desenvolveu as quatro gerações do robô Care-O-bot. Tanto *ANNIE* quanto o *Care-O-bot* têm como objetivo melhorar a eficiência e a segurança nas suas respetivas áreas de atuação – industrial e assistência em casa, nascendo ambos os robôs na mesma instituição. Embora a natureza das tarefas seja diferente, ambos

os robôs são exemplos de como a robótica pode ser aplicada para apoiar o trabalho humano, seja em um ambiente de fábrica ou no quotidiano das pessoas.

A primeira geração consiste numa plataforma móvel capaz de navegar de forma segura e autónoma, comunicando ou guiando pessoas, em museus, como é o caso apresentado em Graf (2005). Na seguinte geração, foi adicionado um braço manipulador, apoios para caminhar e um sensor com duas câmara e scanner laser. A introdução desta segunda geração permitiu à plataforma robótica pegar em objetos de forma autónoma, além de oferecer suporte inteligente para a locomoção. Posteriormente, de modo a obter maior segurança na passagem de objetos entre humanos e robôs desenvolveu-se o Care-O-bot 3 (B. Graf et al., 2009). Este, além das características supramencionadas contém ainda, sensores de visão que permitem a aprendizagem e deteção de objetos de forma autónoma, bem como a supervisão do ambiente em que se inserem em tempo real. A quarta geração encarna um design, mais atrativo. Além das características dos modelos anteriores, este conta ainda com expressões faciais, reconhecimento de voz e a capacidade de detetar emoções com base no ambiente. Estes robôs encontram-se visíveis na Figura 12.

(a) (b) (c) (d)

Figura 12: Gerações dos robôs Care-O-bot. (a): Care-O-bot I; (b): Care-O-bot II; (c): Care-O-bot III; (d): Care-O-bot IV.

2.4. CONTROLO DA NAVEGAÇÃO

O desenvolvimento de um robô móvel autónomo apresenta vários desafios, destacando-se o controlo da navegação. De modo a enfrentar esse desafio, vários grupos desenvolveram e testaram vários métodos para controlar os movimentos dos veículos, em ambientes estático/dinâmicos.

Inicialmente, o método dos campos potenciais era uma técnica bastante utilizada no planeamento de trajetórias para o movimento de robôs. Warren (1989), desenvolveu um método através dos campos potenciais, com o objetivo de gerar uma trajetória segura do ponto inicial até ao final, evitando obstáculos do ambiente. Para este planeamento de trajetórias, são necessários campos potenciais atrativos e repulsivos. O campo potencial atrativo é criado em redor do ponto final (ponto com menor potencial), exercendo uma força atrativa no robô, de modo a direcioná-lo para o ponto desejado. Essa força atrativa diminui à medida que o robô se aproxima do ponto. Por outro lado, são necessários colocar campos potenciais repulsivos em torno dos obstáculos (pontos com maiores potenciais), que exerçam uma força repulsiva que afaste o robô dos mesmos (Vadakkepat et al., 2000). A intensidade da força aumenta à medida que o robô se aproxima dos obstáculos, de modo a evitar uma colisão. Após isso, procede-se à combinação dos campos para criar um campo de forças total, de modo a calcular a direção e magnitude da velocidade do robô, recalculando continuamente o campo de forças à medida que se desloca. No método proposto, inicialmente é escolhida uma trajetória experimental que é posteriormente modificada sob a influência do campo potencial até se encontrar uma trajetória adequada, minimizando os problemas de "mínimo local" onde o robô pode ficar preso.

Embora o método dos campos potencial seja uma abordagem simples e que apresente resultados satisfatórios para o planeamento de trajetórias, apresenta algumas limitações, (Vadakkepat et al., 2000) e (Warren, 1989):

1. Problema dos mínimos locais: em ambiente complexos, o robô pode ficar preso em mínimos locais (isto é, locais onde as forças repulsivas e atrativas equilibram-se), impedindo o robô de prosseguir;

2. Rota pouco eficiente: o caminho gerado pode não ser o mais eficiente a nível de distância ou de tempo;

3. Desconsidera a dinâmica do sistema: não tem em consideração a capacidade de aceleração e desaceleração do robô, podendo provocar movimentos bruscos;

4. Ambientes Dinâmicos: na presença de obstáculos em movimento, o método dos campos potenciais pode ter dificuldade em lidar com mudanças imprevisíveis, uma vez que os campos são calculados baseados na predisposição estática do ambiente.

Outro método de controlo da navegação utiliza a abordagem Dynamic Window (Seder & Petrovitch, 2007), permitindo o controlo de robôs em ambientes dinâmicos com obstáculos em movimento. O movimento dos obstáculos é previsível, porque se aplica um processo idêntico ao *dynamic window* para o movimento dos mesmos. Assim, é possível criar uma interseção, isto é, um ponto fictício de colisão, que é obtido pela colisão da previsão da trajetória do robô e da previsão da trajetória dos obstáculos dinâmicos. Esse ponto fictício provoca a existência de um obstáculo nessa localização, incitando assim o robô a seguir uma outra rota livre de colisão.

O clássico *Elastic Band*, (Quinlan & Khatib, 1993), utiliza um gerador de rotas que planeia um caminho livre de colisões. Após a definição do caminho otimizado, este é submetido a forças de atração e repulsão, fazendo com que a 'banda elástica' se deforme em tempo real. Esse ajuste contínuo permite gerar um percurso mais curto e suave, mantendo, ao mesmo tempo, uma distância segura em relação aos obstáculo. Caso sejam detetadas mudanças no ambiente, através de informação sensorial, o elástico deforma, o que permite que o robô se adapte e reaja a obstáculos em movimento.

Para garantir um comportamento seguro, mesmo quando o gerador de rotas não tem acesso ao mapa completo, foi desenvolvido o método *Timed Elastic Band* (Rösmann et al., 2023), baseado na abordagem anterior. Este método complementa o anterior com informações temporais, permitindo assim, restringir as dinâmicas do robô, como a velocidade e a aceleração do robô, e modificar as trajetórias, em vez de seguir o percurso gerado.

Outra estratégia é o *Fuzzy Control Method*, (Cherni et al., 2016), que inicialmente define uma rota até ao local desejado. Durante a navegação, o robô adquire informação do ambiente e dos obstáculos através de sensores. Consoante a deteção de obstáculos, o robô muda a sua direção e velocidade angular para contornar o obstáculo e, desta forma, alcançar o local pretendido (Almayyahi et al., 2017).

Em Louro et al (2019) e Louro et al (2020) são propostos dois veículos autónomos distintos, *tugger* e *stacker* respetivamente. Ambos utilizam sistemas dinâmicos não lineares uma ferramenta matemática que permite ao robô seguir para um alvo, desviando-se de obstáculo, proporcionando resultados que garantem uma navegação segura e eficiente, permitindo a coexistência entre humanos e veículos autónomos. Para isso, são desenhados sistemas baseados em *Dynamic Approach to Behavior-Based*

Robotics, (Bicho, 1999). Assim, uma arquitetura de controlo formalizada com sistemas dinâmicos não-lineares é implementada e desenvolvida nesses veículos. O comportamento, ou seja, o movimento, é gerado ao fornecer continuamente valores para a direção de navegação e velocidade linear ao longo do tempo. Através de sensores, o robô identifica os obstáculos, representados por repulsores e tem ainda conhecimento do alvo desejado, representado por um atractor. Posto isto, isoladamente cada força é representada por um atractor ou um repulsor para a dinâmica da direção de navegação. O princípio destas dinâmicas é sintonizar o sistema de modo a que a informação sensorial que entra no sistema dinâmico estabeleça forças atrativas ou repulsivas, determinando a intensidade e alcance dessas interações. Embora a influência de cada força isolada possa variar com as mudanças nos valores das variáveis comportamentais, e não necessariamente resultar nos atractores ou repulsores desejados, a sobreposição das contribuições de todos os sensores demonstra uma consistência na invariância que gera à dinâmica desejada. Como neste método o robô pode detetar mais que um atractor simultaneamente, a tomada de decisão é feita através de bifurcações criadas pelo próprio veículo. A direção de navegação ''cai'' para dentro da bacia de atração, ou seja, o atractor mais próximo. Estas assimetrias aparecem devido à assimetria de dados sensoriais ou induzidos pela força estocástica que foi adicionada ao campo vetorial da dinâmica da direção de navegação. Estas soluções incluem ainda soluções *human-aware*, para aumentar a segurança da navegação, ou seja, além de navegar de forma segura entre pontos, o sistema ao detetar a presença de operadores interage com eles, permitindo ajudar na perceção do seu comportamento.

Em Costa Silva et al (2006) é realizado um estudo comparativo entre a aplicação dos sistemas dinâmicos não lineares e a aplicação dos campos potenciais. A Tabela 1 apresenta as diferenças mais relevantes.

Tabela 1: Comparação Campos Potenciais VS Atractores Não Lineares.

	Campos Potenciais (PFM)	Sistemas Dinâmicos Não Lineares (SDNL)
Obstáculos	Representados por Repulsores	Representados por Repulsores
Alvo	Representado por 1 atractor	Representado por vários atractores com diferentes possibilidades (virar à esquerda, direita ou seguir em frente)
Determinação do Movimento	Segue numa única direção da força resultante (gradiente do campo)	Escolhe entre os atractores existentes baseados em bacias de atração
Geração da Trajetória	Soluções transientes de um sistema baseado no campo potencial escalar	Sequência de Soluções de Atractoras de um sistema dinâmico não linear
Flexibilidade e Opções de Movimento	Limitado em opções de movimento e é propenso a mínimos locais	Mais flexível, oferece múltiplas opções para a movimentação
Comportamento em Ambientes Dinâmicos	Menos adaptável a mudanças no ambiente	Maior adaptabilidade, comporta-se melhor em ambientes dinâmicos

Desta forma, através da Tabela 1 e do mencionado por Costa Silva et al (2006), verifica-se que as abordagens apesar de apresentarem semelhanças na representação de alvos e obstáculos, são bastante diferentes nos restantes tópicos, estando salientes as vantagens do método de sistemas dinâmicos não lineares.

Em (Chen et al., 2022) é apresentado um estudo que demonstra que os robôs frequentemente colidem com obstáculos dinâmicos, caso a velocidade da informação não seja considerada dentro da framework de navegação. Este estudo, desenvolve um método baseado em ROS, de modo a melhorar a viabilidade da dinâmica de evitar obstáculos para "altas" velocidades. Atualmente, muitos sistemas de controlo de navegação baseiam-se em geradores de trajetórias, complementados por técnicas de desvio de obstáculos e controlo de velocidade. Neste artigo, propõe-se a adição de uma nova camada de velocidade dos obstáculos, estimada através de um filtro de Kalman. Esta abordagem cria um mapa em tempo real

com as velocidades dos obstáculos dinâmicos, permitindo um planeamento mais eficaz de trajetórias seguras.

2.4.1. Controlo da Navegação Em Veículos Omnidirecionais

Os veículos omnidirecionais são caracterizados pela sua versatilidade e mobilidade, permitindo que se desloquem de um ponto ao outro em linha reta, sem necessidade de rodarem primeiro. Além disso, durante os movimentos de translação, os veículos deslocam-se ao longo de qualquer caminho desejado, podendo ser combinados com um movimento de rotação quando o robô chega ao destino e não está na orientação correta.

Os manipuladores móveis na indústria requerem um elevado grau de eficiência nas operações, seja na velocidade que se deslocam, quer na sua precisão ou robustez. Desta forma, as plataformas omnidirecionais têm um papel importante, dado que permitem a movimentação em torno de -x, -y e a rotação sobre -z, simultaneamente. Além do manipulador móvel OmniMan, conforme ilustrado na Figura 13, foi desenvolvida uma solução para a colaboração humano-robô (Roehrig et al., 2019) que adota uma estrutura em cascata. Este sistema consiste em controladores individuais para cada funcionalidade, incluindo três componentes principais: um controlador de posição, que define a orientação da plataforma em relação ao referencial do 'mundo'; um controlador de rodas, responsável por regular a velocidade das rodas; e um controlador de acoplamento, cuja função é ajustar as velocidades em resposta a restrições cinemáticas do movimento.

Figura 13: Manipulador Móvel *OmniMan*: Braço Robótico UR5 numa plataforma móvel com rodas *Mecanum*.

No que concerne ao controlador de posição, o mesmo obtém a orientação atual da plataforma através de odometria e calcula as velocidades para a plataforma, considerando o sistema de referência global. Essas velocidades são transformadas para o sistema de referência do robô e, em seguida, convertidas em velocidades de referência para o controlador das rodas. As velocidades de referência são calculadas usando cinemática inversa da plataforma, garantindo assim que atendam às restrições cinemáticas o tempo todo. No entanto, devido às diferenças nos parâmetros de controlo das rodas, estas podem ficar desequilibradas, levando a que as suas velocidades reais violem as restrições cinemáticas do movimento, o que resulta num deslizamento adicional.

A nível hospitalar, foi concebida uma solução para um robô móvel omnidirecional em Takahashi et al (2010), denominada de MKR (*Muratec Keio Robot*), como apresentado na Figura 14. O robô possui uma espécie de carruagem para transportar bagagens, amostras ou outros materiais, garantindo uma entrega segura num ambiente partilhado com humanos. O veículo é composto por quatro rodas configuradas diagonalmente, com ângulos de 45°, 135°, 225° e 315°, o que permitiu um desenvolvimento de um método para o controlo da navegação, que se baseia em forças potenciais atractoras ou repulsoras virtuais. Neste método há vários módulos, em que cada um utiliza tempos de previsão diferentes, sendo processados paralelamente, de modo a alterar o comportamento do robô, alterando assim a sua velocidade. A fim de não interferir com o ambiente, garantindo um transporte seguro, é determinado um ângulo para o qual o robô se deve deslocar, chamado ângulo desejado, sendo este o ângulo entre a direção do vetor tangente ao ponto mais próximo do caminho idealizado, e a direção atual do robô.

Figura 14: Robô Omnidirecional MKR (*Muratec Keio Robot*).

2.5. MANOBRAS DE ACOSTAGEM

O termo acostagem (em inglês *docking*) refere-se à aproximação e paragem numa posição em relação a um objeto, como por exemplo uma palete, uma estação ou um carril. Este termo, segundo Bostelman & Hong (2016), tornou-se motivo de pesquisa em diversas áreas, desde 1950, quando ocorreu a integração de veículos autónomos.

Foi proposta em Lucas et al (2005), uma técnica baseada em *multi-objective, evolutionary algorithm using multiple fuzzy logic controllers*, cujo objetivo é melhorar algumas características dos robôs, tais como a duração, precisão e a estabilidade nas tarefas de elevada precisão de encaixe, para uma empilhadora. A trajetória do veículo segue uma curva *B-Spline,* pré-gerada até ao ponto de acostagem, onde a técnica *Pure Pursuit* é aplicada para determinar o ponto desejado de deslocamento. Quanto maior o grau da curva *B-Spline*, maior a influência desses pontos na trajetória, permitindo assim um ajuste gradual da direção. Desta forma, são apresentados resultados, nos quais os erros de orientação e posição são reduzidos, provocando, desta forma, um menor esforço de controlo, uma menor variação no ângulo de direção, ao mesmo tempo que evita mudanças repentinas de velocidade para atingir manobras estáveis.

Em W. Geng & F. Guangrui (2017), é explorada uma solução para as operações de *docking*. A área de *docking* encontra-se divida em duas zonas, para ter um resultado mais preciso. Na primeira área, utilizando a técnica de localização e mapeamento ORB-SLAM para identificar pontos de interesse, o robô, ao receber a ordem para iniciar o carregamento, começa a procurar uma AprilTag, que o orienta até ao ponto de aproximação (imagem ilustrativa na Figura 15). Assim que encontre a *tag*, o robô muda o seu comportamento para o modo ajuste de orientação, alinhando de forma consistente e contínua a sua posição e o ângulo relativo à informação proveniente da *tag*, até à zona de *docking* do carregamento.

(a) (b)

Figura 15: (a) - Robô desenvolvido; (b) - *AprilTag*.

O trabalho realizado por Louro et al (2020) propõe um método que permite um veículo móvel realizar manobras de acostagem de forma autónoma. Este mecanismo recorre à utilização de conceitos de sistemas dinâmicos não-lineares. O sistema rege-se por um gestor de tarefas, que opera como uma máquina de estados que contém as operações a serem realizadas, considerando tanto a tarefa solicitada pelo servidor quanto a última operação concluída, indicando o estado atual do sistema. Para realizar a manobra, são adicionados três pontos auxiliares (dois alvos intermédios e um ponto de serviço). Inicialmente, o veículo move-se longitudinalmente, seguindo a dinâmica de "seguir uma parede", até convergir para o primeiro alvo auxiliar. Uma vez verificada a convergência, o sistema aproxima-se do segundo alvo, aplicando a estratégia de "seguir o alvo". Em seguida, o veículo ajusta sua direção de navegação para alinhar-se com o local de acostagem. Na implementação descrita, também foram introduzidas operações que garantem a segurança (ou seja, o funcionamento adequado dos sensores do veículo). Por último, a *pose* do veículo é ajustada com base nas informações obtidas por uma câmara acoplada, orientando-se pela localização do ponto de *docking*, até que seja alcançado o ponto, que é o último alvo auxiliar.

2.5.1. Manobras de Acostagem Em Veículos Omnidirecionais

No contexto de *docking*, especialmente em situações onde é importante ter uma elevada precisão e agilidade em espaços reduzidos, os veículos omnidirecionais oferecem vantagens ao realizarem manobras de aproximação e atracagem com estações de carga, movimentando-se em qualquer direção.

A área de cuidados de saúde, seja de envelhecimento ou de incapacidades, tem sido uma das áreas alvo, com maior atenção por parte dos progressos científicos, provenientes da indústria ou da comunidade científica. Assim, em X. Li et al (2018) é desenvolvida uma solução para o problema da acostagem numa cadeira de rodas omnidirecional, através do método de controlo servo visual baseado no *feedback* de posição, Figura 16.

Figura 16: Sistema da Cadeira Inteligente (X. Li et al., 2018).

Para o processo de acostagem, a cadeira inteligente, deve ficar numa posição paralela e adjacente relativamente à cama auxiliar, como ilustrado na figura acima. Este sistema possui uma câmara, que permite obter a localização visual da orientação relativa e posição entre a cadeira e a cama, com a finalidade de alcançar a orientação da cadeira e o controlo dos motores servo das rodas, guiando assim a cadeira de rodas da posição inicial para a posição desejada, através do feedback visual.

Este sistema divide-se em quatro sistemas de coordenadas: cama, câmara, cadeira e o "mundo". O sistema de coordenadas da cama, usa como ponto referência todos os restantes sistemas de coordenadas (câmara, «cadeira e "mundo"). O sistema de coordenadas da câmara relaciona-se com o sistema de coordenadas da cadeira de rodas através de uma matriz de translação. Por sua vez, o sistema de coordenadas do "mundo" é formado pela rotação do sistema de coordenadas da cama π radianos no sentido anti-horário em torno do eixo XL. O objetivo da localização visual é determinar a posição da cadeira de rodas no sistema de coordenadas do "mundo". Portanto, a cinemática utilizada para o controlo de movimentação, foi projetada com base na relação relativa entra a posição atual da cadeira e a desejada.

Por outro lado, os estudos dos oceanos revelam um aspeto crucial para uma melhor compreensão dos fenómenos extremos que causam destruição em várias partes do mundo. Como resultado, os AUVs (Veículos Autónomos Subaquáticos) têm despertado um crescente interesse no seu desenvolvimento (Smith et al., 2011). De modo a combater as limitações de armazenamento de energia para os AUVs, em Dong et al (2023) é proposta uma abordagem autónoma orientada para a recarga de energia através de manobras de *docking*, podendo ser adaptada para outros robôs em outras áreas. O diagrama esquemático está representado na Figura 17.

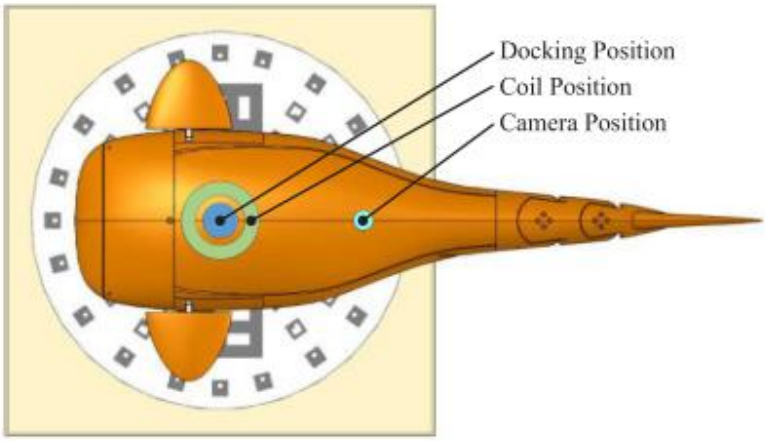


Figura 17: Esquemático do *docking* da plataforma (Dong et al., 2023).

Primeiramente, é apresentado um peixe robótico omnidirecional, capaz de reconhecer visualmente a plataforma de *docking* e realizar carregamento sem fio na plataforma. Assim, a plataforma apresenta no centro um ponto denominado de *destination point*, e o centro da superfície do robô é denominado de *docking point*. De forma a tornar o processo automático, o *docking point* deve estar o mais próximo possível do *destination point*. Dessa forma, as características de movimento são definidas com base no erro de posição entre esses dois pontos, que é subdividido nas coordenadas de profundidade, transversal e lateral.

O procedimento do robô face ao esquema acima referido, divide-se em quatro passos:

1. Procura a plataforma, para ficar próximo da região, e após a sua deteção desacelera de modo a conseguir parar perto da mesma;
2. Ajusta o seu posicionamento da lateral face à plataforma, através do ângulo de *yaw*;
3. Reduz o erro da profundidade, flutuando e aterrando;

4. Reduz e elimina os erros transversais e longitudinais até atingir um *docking* realizado com sucesso.

2.6. Navegação Com Social Aware

Navegar com robôs em espaços partilhados com pessoas pode ser um desafio. Neste cenário, surgem preocupações adicionais, pois não se trata apenas de mover o robô de forma eficiente pelo espaço, mas também de respeitar as normas sociais, de modo a garantir interações seguras e naturais. Nesse contexto, a investigação sobre "Social Aware Robot Navigation" tem vindo a ganhar destaque, com o objetivo de desenvolver técnicas que permitam aos robôs navegar tendo em conta essas convenções sociais (Zhang et al., 2023).

A colaboração eficaz humano-robô é fundamental em ambientes industriais para melhorar a produtividade, a eficiência e a segurança. Para conseguir isso, os robôs devem ser equipados com estratégias de aprendizagem, que lhes permitam operar ao lado de colaboradores humanos. Em Zhang et al. (2023), são exploradas as principais estratégias de aprendizagem, com o intuito de melhorar as interações em ambientes industriais. Entre os diferentes tipos de aprendizagem, destacam-se a aprendizagem por imitação, que se desenvolve com base em padrões, e a que é adquirida através de dados previamente inseridos, que geram uma resposta.

Mukherjee et al (2022) apresentou um estudo sobre os níveis de interação humano-robô, abordando diversos aspetos, como segurança, previsão de intenção de movimento humano e aprendizagem por demonstração e/ou manipulação, no contexto da colaboração humano-robô (HRC). O estudo utilizou modelos baseados em *Machine Learning* para suportar essas interações. As aplicações que fazem uso de indicações visuais são diversas e abrangem vários domínios, aproveitando a forma como as indicações visuais sugerem possíveis ações ou interações com objetos e ambientes. Estes recursos visuais referem-se às propriedades percetivas dos objetos, que indicam os seus potenciais usos ou funcionalidades, orientando os utilizadores nas suas interações.

No domínio da interação humano-robô, as indicações visuais desempenham um papel crucial, permitindo que os robôs compreendam e interpretem as intenções humanas. Por exemplo, robôs equipados com sistemas de visão computacional podem reconhecer gestos com as mãos, expressões faciais e movimentos corporais para inferir as necessidades ou comandos por parte do utilizador/operário. Estes reconhecimentos permitem interações mais intuitivas e naturais entre humanos e robôs, facilitando tarefas como trabalho colaborativo ou assistência pessoal. O estudo que Mukherjee apresentou, mostra

aplicações de funcionalidades visual no contexto do HRC, conforme apresentado na Figura 18 (Mukherjee et al., 2022).

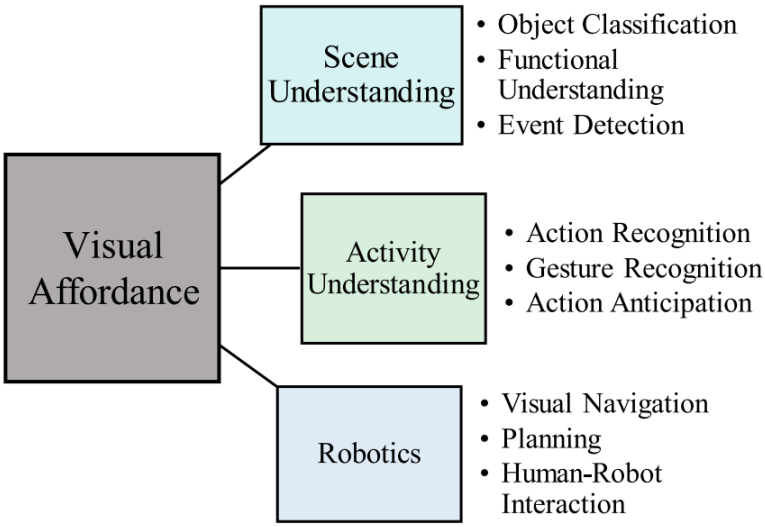


Figura 18: Aplicações dos recursos visuais no contexto da colaboração humano-robô (Mukherjee et al., 2022).

De acordo com o exposto no estudo, os recursos visuais são utilizados em veículos autónomos, os quais ajudam os sensores e algoritmos incluídos no veículo, a interpretar informações externas e obstáculos. Ao compreender estes dados visuais, os veículos autónomos podem tomar decisões informadas sobre navegação, velocidade e segurança, contribuindo para a sua eficácia e fiabilidade.

Os seres humanos podem ainda emitir instruções ou comandos para robôs através de comandos de voz. A compreensão da linguagem natural e o processamento da fala são utilizados para interpretar e executar as instruções dadas. Além disso, os robôs podem responder a comandos ou fornecer informações através de respostas verbais, auxiliando a comunicação e gestão de tarefas (Zhang et al., 2023). Assim sendo, as interações mais comuns encontram-se resumidas na Tabela 2.

Tabela 2: Tipos de métodos de comunicação humano-robô mais utilizados em sistemas HRC (Zhang et al., 2023).

Method	Description	Pros	Cons
Gesture command	Interact with robots naturally based on user gestures.	Has a user-friendly interface and is easy to use.	Gesture disagreement between users.
Voice command	Interact with robots based on user voice.	Enables complex instructions and parameterizations.	Less practicality in busy and noisy work environments.
Gaze command	Interact with robots based on user gaze.	Incorporates the user's visual focus of attention (VOA).	Expensive eye-tracking devices and difficulty in the interpretation of user eye-tracking data.
Bio-signal command	Interact with robots based on user bio-signal (e.g., EEG, sEMG)	Is adaptive and can reflect the user's physiological state.	Difficulties in stable data acquisition and data preprocessing, which are susceptible to environmental influences.
Vision-based proactive command	Interact with robots based on contextual visual information.	Proactively plans and acts accordingly in real time.	Harsh on-site conditions affecting algorithm accuracy.

De acordo com a tabela acima, a segurança do colaborador humano é um aspeto relevante do HRC, o que conduziu a posteriores estudos para compreender os potenciais riscos associados ao uso do mesmo. Alguns riscos potenciais desse método estão mostrados em (Zhang et al., 2023), apresentados naTabela 3, englobando riscos físicos, tecnológicos, condições do meio onde se inserem, entre outros.

Tabela 3: Diferentes tipos de riscos potenciais de HRC(Zhang et al., 2023).

Risk subject	Risk item	Description
Human factor	Physical injury	Risks related to injury caused by direct physical contact.
	Physical ergonomics	Risks related to working postures and MSD in the HRC system.
	Psychosocial	Risks related to workers' mental states, trust levels, and psychological factors.
Construction robot	Technology	Risks related to construction robot hardware, software, and program failure.
	Operating system	Risks due to human error or operating system failure in teleoperation.
	Material or equipment	Risks due to construction robots handling materials and construction equipment, grippers, and end effectors.
	Maintenance	Risks arising from the maintenance of construction robots.
External environment	Supervision	Risks due to the on-site management failure of the safety manager, wrong task assignment, and site layout problems.
	Environmental conditions	Risks related to extreme weather, extreme temperatures, and other harsh environments.
	Task hazards	Risks of specific construction tasks and materials on the environment and workspace, such as welding and chemical materials.

Como complemento aos riscos acima apresentados, o mesmo estudo apresenta ainda um diagrama sobre os fatores de controlo de segurança associados ao uso do HRC, apresentado de seguida(Mukherjee et al., 2022).

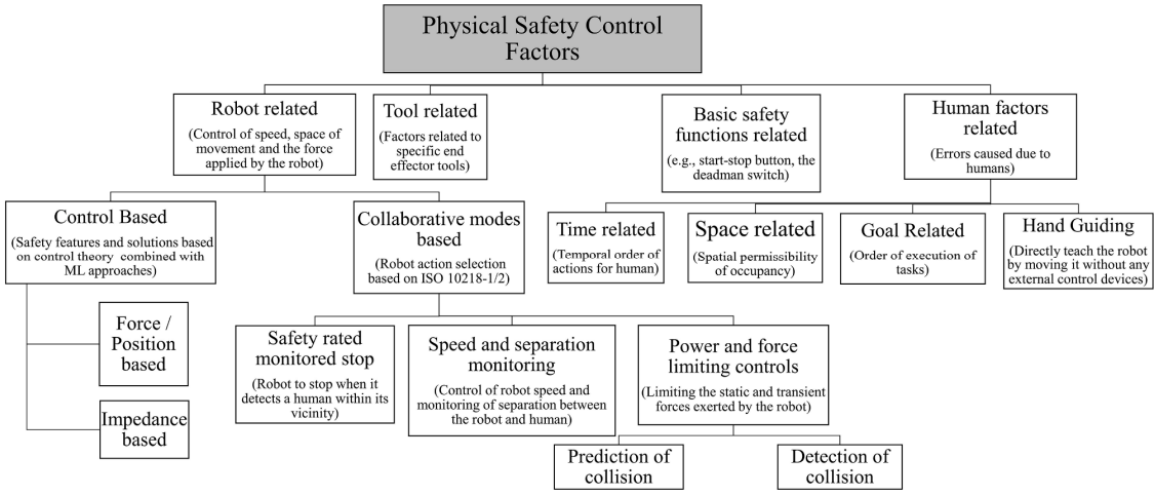


Figura 19: Fatores de controlo de segurança física para a colaboração humano-robô (Mukherjee et al., 2022).

Vários aspetos, como ilustrado na Figura 19, foram definidos para identificar os fatores que influenciam a segurança. Entre estes, destacam-se a descrição de modos de operação seguros e a especificação de medidas preventivas e de deteção essenciais para o design eficaz de espaços de trabalho colaborativos. Estas normas oferecem diretrizes abrangentes para assegurar que os ambientes colaborativos sejam projetados com a segurança como prioridade. Dentro do tópico da segurança, são abrangidos vários aspetos, incluindo a identificação de potenciais perigos, a implementação de protocolos de segurança e a adoção de estratégias para prevenir acidentes e detetar potenciais problemas. Ao aderir a estas diretrizes de controlo e segurança, as organizações podem criar espaços de trabalho colaborativos mais seguros e eficientes, promovendo o bem-estar dos trabalhadores e a eficácia operacional (Mukherjee et al., 2022).

No contexto colaboração entre robôs e humanos em ambientes industriais, Louro et al. (2019) explora a utilização de comandos verbais para melhorar a interação com robôs móveis. O veículo utilizado é o *tugger*, um veículo autónomo que é utilizado para transportar materiais. Este comunica com os operadores humanos verbalmente, indicando o sentido do seu movimento ou solicitando que sejam retirados obstáculos, ou que as pessoas se desviem do seu caminho. Esta capacidade de comunicação verbal visa não só aumentar a eficiência da navegação em ambientes complexos, mas também garantir a segurança dos operadores e a fluidez das operações. A abordagem de Louro et al. (2019) destaca a

importância de dotar os robôs com capacidade de perceção social e comunicação eficaz, permitindo-lhes reagir de forma adequada ao contexto e aos comportamentos humanos. Ao informar verbalmente os operadores sobre as suas intenções, o *tugger* evita paragens desnecessárias e lida com humanos de forma mais eficiente, ao contrário de veículos que possuem apenas sensores para deteção de obstáculos. O estudo realizado por Louro et al (2019), demonstra que a metodologia *social-aware* tem um impacto direto na redução de tempos de navegação em ambientes dinâmicos. Ao interagir com os operadores humanos de forma ativa, o *tugger* minimiza o tempo necessário para desimpedir o caminho e continuar o seu percurso, mitigando os atrasos causados pelos operadores que interrompem a sua trajetória. Assim, Louro et al. (2019), retirou que capacidade de interação social melhora a eficiência dos sistemas robóticos, particularmente em ambientes industriais onde o número de operadores humanos e de obstáculos móveis é elevado.

Em resumo, a colaboração humano-robô representa uma inovação promissora para a automação de uma organização, nomeadamente na secção de armazéns, com potencial para transformar significativamente as operações logísticas, (Mukherjee et al., 2022). À medida que as tecnologias HRC continuam a evoluir, espera-se que se tornem cada vez mais integradas e essenciais nas operações de armazém, contribuindo para a criação de ambientes de trabalho mais eficientes, seguros e adaptáveis. A investigação e o desenvolvimento em curso nesta área são cruciais para enfrentar os desafios e maximizar os benefícios da colaboração humano-robô (Mukherjee et al., 2022).

2.7. Discussão

A revolução industrial evoluiu até à quinta fase, conhecida como Indústria 5.0, que se concentra no ser humano e na integração e combinação entre tecnologias, organizações e indivíduos. O objetivo desta revolução passa pela criação de sistemas mais sustentáveis e resilientes que, por sua vez, aumentam a produtividade e a eficiência nas operações das organizações.

Perante estas revoluções, optou-se pela introdução de veículos móveis nos meios industriais, tendo em consideração os critérios para a sua adoção, como a performance, a interação humano-robô e a segurança. Os robôs móveis têm sido uma aposta em diversas tarefas industriais, reduzindo o esforço físico dos operadores.

Dentro dos dois tipos de robôs móveis, atualmente, os mais eficientes são os AMRs, pois oferecem a flexibilidade das capacidades deliberativas do operador humano, o que significa que, em vez de simplesmente seguir um caminho pré-definido, pode tomar decisões em tempo real e ajustar o trajeto

dele conforme necessário. Foram ainda mostrados alguns exemplos dos robôs existentes, tanto AGVs como AMRs.

No que diz respeito ao desenvolvimento de um robô móvel autónomo é necessário desenvolver o controlo da navegação. Foram então apresentados algoritmos para esse controlo, baseado em campos potenciais, e/ou baseados em dinâmicas de atractores não lineares. Os métodos de controlo baseados em dinâmicas de atractores não lineares mostraram-se ser a melhor solução para ambientes mais complexos e dinâmicos,

No que concerne às manobras de acostagem, o estudo revela-se, essencialmente, através de sistemas de visão e de coordenadas, podendo até recorrer ao uso de *tags*. Esta manobra passa, fundamentalmente, por ser uma manobra de correção de posição com a finalidade de o robô orientar-se na direção pretendida para a tarefa.

3. COMPONENTES DE HARDWARE E SOFTWARE

Este capítulo tem como objetivo detalhar a estrutura da plataforma móvel, utilizada na presente dissertação. São discutidas as características principais da plataforma, destacando o *design* das rodas que possibilitam movimentos holonómicos, sendo também explorados os métodos de monitorização do ambiente envolvente. Não obstante, são apresentados os softwares utilizados neste projeto.

3.1. PLATAFORMA MÓVEL

O KUKA *Mobile Plataform* (KMP) 200 *OmniMove*, apresentado na Figura 20, foi a plataforma escolhida para efetuar as tarefas de transporte de caixas, sendo esta uma plataforma omnidirecional.

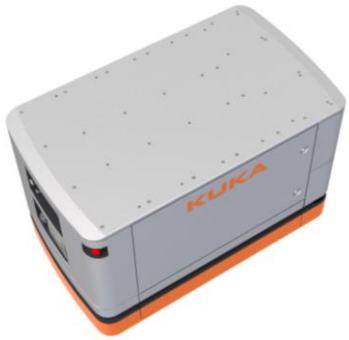


Figura 20: KUKA *Mobile Plataform* 200 *OmniMove* (*KUKA AG*, 2020.).

O KMP 200 OmniMove é uma plataforma móvel desenvolvida pela KUKA AG, e foi projetada para oferecer maior mobilidade, isto é, integra a possibilidade recorrer a movimentos omnidirecionais, podendo mover-se em todas as direções e, ainda permite o transporte de cargas pesadas.

3.1.1. DIMENSÕES DA PLATAFORMA

As principais características da plataforma móvel KUKA KMP 200 OmniMove, são apresentadas na Figura 21, destacando as suas dimensões: 1,13 m de comprimento, 0,63 m de largura e 0,70 m de altura.

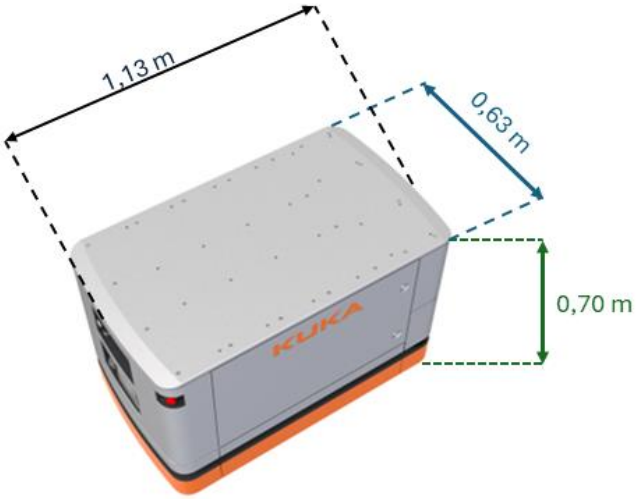


Figura 21: Representação da plataforma móvel e das respetivas dimensões (Comprimento - Largura – Altura,).

Além das dimensões acima expostas, na Tabela 4 são apresentadas algumas das características adicionais da plataforma.

Tabela 4: Características da plataforma móvel KUKA *Mobile Plataform* 200 *OmniMove* (*KUKA AG*, n.d.)..

	Unidade
Cumprimento	1,13 m
Largura	0,63 m
Altura	0,70 m
Carga Máxima (sem braço)	200 kg
Velocidade Longitudinal	1,00 m/s
Velocidade Lateral	0,56 m/s
Velocidade de Rotação	0,56 rad/s
Aceleração Máxima	0,50 m/s^2
Tempo de Operação (Máxima Carga)	8 h

De acordo com a Tabela 4, é fornecido um tempo de operação estimado padrão, mas é de ressaltar que esse tempo pode variar conforme o ambiente e as condições em que opera. Em condições padrão, a plataforma móvel pode operar continuamente até oito horas, equivalente a um turno de trabalho. Para

otimizar a eficiência nos vários postos, o algoritmo de gestão de tarefas deve considerar o tempo de operação contínua da plataforma móvel como parte integrante do planeamento operacional.

Não obstante, a plataforma é equipada por quatro rodas omnidirecionais independentes, cada uma delas composta por um conjunto de rolamentos livre. Além disso, está equipada com dois sensores lasers *SICK* 300 *Expert*, posicionados diagonalmente e em lados opostos.

3.1.2. Rodas Omnidirecionais

O KMP 200 OmniMove utiliza tecnologia de rodas omnidirecionais, como ilustradas na Figura 22, o que significa que pode mover-se em todas as direções, sem a necessidade de mudanças de direção. Isto é, as rodas podem girar independentemente umas das outras, proporcionando uma agilidade significativa em espaços confinados.

Figura 22: Roda *Mecanum* utilizado pelo KMP 200 *OmniMove* (*KUKA AG.*).

Cada roda é composta por dois aros e nove rolos de livre rotação, montados num ângulo de 45 graus face aos aros. As rodas são acionadas por um motor elétrico, incorporado.

3.1.3. Sensores *SICK* S300 *Expert*

O KMP 200 OmniMove é composto por dois sensores laser *SICK* S300 *Expert*, sendo um exemplar apresentado na Figura 23.

Figura 23: Sensor laser *Sick* S300 *Expert* (*Sick* / *Sensor Intelligent*, n.d.).

Trata-se de um sensor lidar, que usa como sistema de medição o *laser*, o que permite a recolha de informação do ambiente em redor, em duas dimensões. De modo a facilitar a visualização das caraterísticas do sensor, estas encontram-se descritas na Tabela 5.

Tabela 5: Características do Sensor *laser Sick* S300 *Expert* (*Sick* / *Sensor Intelligent*, n.d.).

	Unidade
Alcance Máximo	30 m
Ângulo de Varredura	270 °
Resolução Angular	0,5 °
Alcance do Campo de Proteção até	2 m

Como está evidenciado na Tabela 5, o sensor possui um alcance máximo de 30 metros, isto num ângulo de varrimento de 270°, com uma resolução de 0,5 °, sendo também ilustradas essas características na Figura 24.

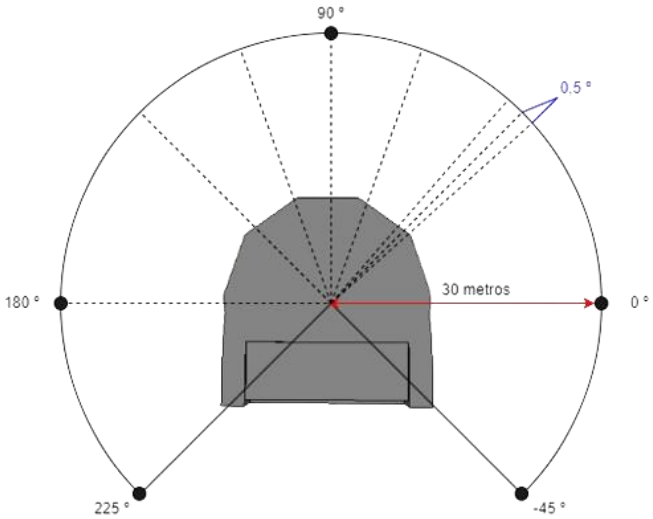


Figura 24: Demonstração da gama de varredura, resolução angular e alcance máximo do Sensor *Sick* S300 *Expert*.

Como a plataforma é composta por dois destes sensores, inversamente colocados, é possível ter um varrimento de 360 °.

Para determinar quantos feixes de *laser* são emitidos, divide-se o ângulo de varrimento pela resolução angular, como mostra a equação abaixo. Desta forma, retira-se que o sensor emite 540 feixes *laser*. Os cálculos estão demonstrados na equação (1).

$$\text{Número de Feixes} = \frac{\text{Ângulo de Varrimento}}{\text{Resolução Angular}} = \frac{270\ º}{0{,}5\ º} = 540 \qquad (1)$$

3.2. SOFTWARES

Este subcapítulo tem como objetivo detalhar os recursos de software utilizados para implementar a estratégia e analisar o comportamento do sistema. Inicialmente, será discutida a escolha da *framework* central para o desenvolvimento da solução, destacando as funcionalidades e recursos que justificam a seleção. Posteriormente, serão explorados simuladores que complementam o desenvolvimento e análise do sistema.

3.2.1. ROS 2 – ROBOT OPERATING SYSTEM 2

ROS, ou *Robot Operating System* (Sistema Operacional para Robôs), é um *framework open-source* que fornece ferramentas e bibliotecas para o controlo de sistemas robóticos, facilitando a criação e integração de sistemas complexos.

O ROS 2, é a evolução do *framework* ROS original, projetada para atender às necessidades crescentes da robótica moderna (Heggem & Wahl, 2020). É classificado como *middleware*, uma vez que reside entre a aplicação e o sistema de software, disponibilizando recursos, como mecanismos de comunicação entre processos.

A estrutura do ecossistema ROS 2 pode ser dividida em duas partes principais:

1. **Núcleo de Comunicação (core)**: responsável por garantir que os diferentes componentes do sistema possam comunicar de maneira eficiente e robusta.

2. **Bibliotecas e Ferramentas de Desenvolvimento**: focado no desenvolvimento, gestão, organização e a implementação de funcionalidades específicas em sistemas robóticos.

Os seguintes subcapítulos, têm como objetivo explicar, mais detalhadamente, como funcionam as bibliotecas e ferramentas de desenvolvimento.

3.2.2. O QUE É UM NODE?

Um *node* (nó), no contexto do ROS 2, é uma unidade de execução que desempenha uma função específica num sistema robótico. Os *nodes* são componentes modulares que operam de forma independente, mas interagem entre si para criar o comportamento completo do sistema robótico. Não obstante, facilitam a divisão de um sistema complexo em componentes simples de gerir, cada um responsável por uma parte específica do comportamento do sistema robótico.

Um exemplo ilustrativo de um sistema composto por *nodes* é o que está presente na Figura 25.

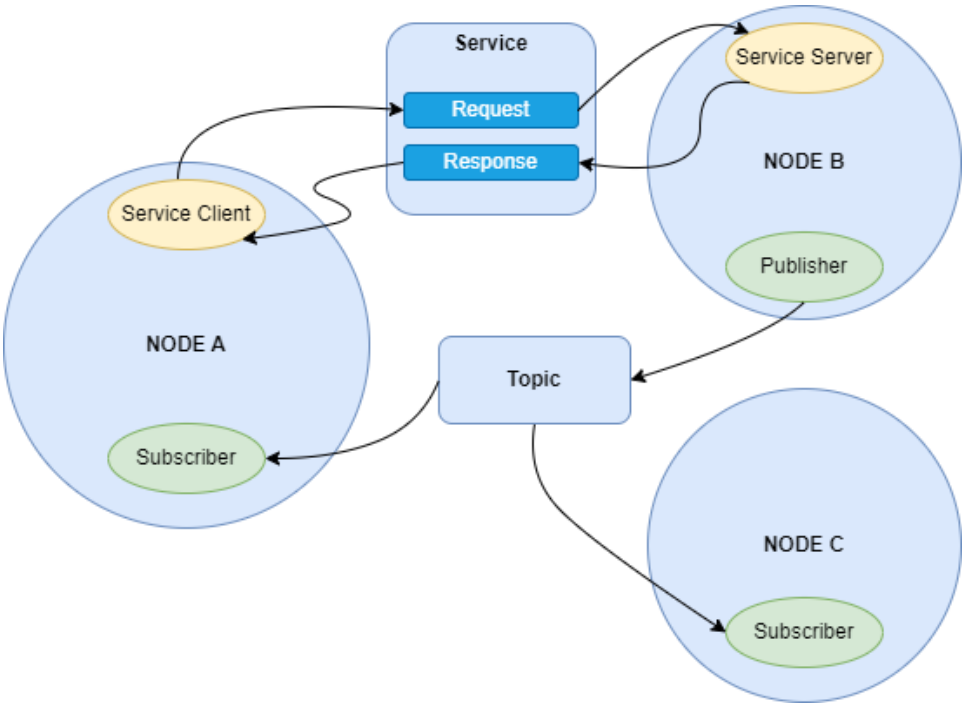


Figura 25: Funcionamento dos Nodes.

Cada nó presente no exemplo anterior, é responsável por um propósito modular, e pode enviar e/ou receber dados de outros nós, através de *topics*, *services* ou *actions*.

3.2.3. O QUE É A COMUNICAÇÃO VIA ROS TOPIC?

A comunicação via ROS *topic*, é uma comunicação do tipo assíncrona e unidirecional e é baseada na estrutura *publisher/subscriber* a um determinado tópico. Para que a comunicação seja efetiva, todos os *publishers* e *subscribers* associados a um determinado tópico devem utilizar o mesmo tipo de dados específico do tópico correspondente. Essa padronização garante que os *nodes* sejam capazes de interpretar corretamente as mensagens trocadas. A comunicação unidirecional significa que o emissor não recebe confirmação imediata da receção da informação pelo recetor, conforme ilustrado na Figura 26.

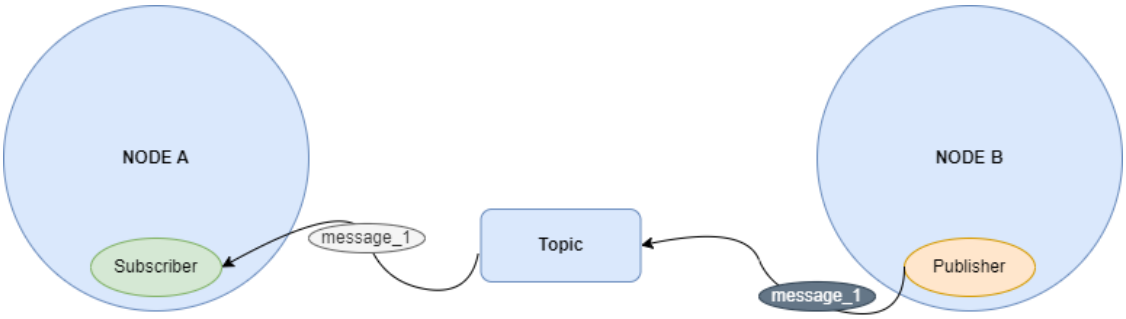


Figura 26: Funcionamento da comunicação via ROS *topic*.

Não obstante, vários *nodes* podem publicar no mesmo *topic*, assim como vários *nodes* podem subscrever o tópico, tendo assim uma relação de N:N (isto é, muitos para muitos). Por sua vez, o *topic*, é composto por um *<topic_name>*, e ainda uma mensagem *<type/msg/type_msg>*, como mostra a Figura 27.

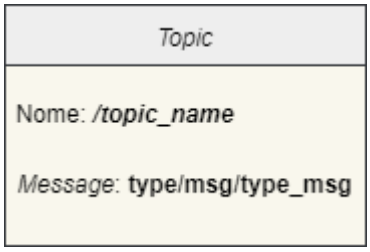


Figura 27: Estrutura do *topic*.

3.2.4. O QUE É A COMUNICAÇÃO VIA ROS SERVICE?

No que diz respeito á comunicação por *ROS services*, esta é baseada no modelo *client/server* e, é estabelecida de forma bidirecional síncrona/assíncrona. Por outras palavras, é um modelo de pedido e de resposta, em conjunto com um publicador/subscritor.

Os *services* seguem o padrão cliente/servidor, onde um *node* atua como servidor, oferecendo um serviço específico, sendo que os outros *nodes* podem agir como clientes, de modo a solicitar esse serviço conforme necessário. A comunicação por *services* é bidirecional, o que significa que envolve tanto o envio de pedidos do cliente para o servidor, quanto o envio de respostas do servidor de volta ao cliente. Na Figura 28, pode-se observar o esquemático do funcionamento da comunicação via ROS *Service*.

Enquanto a comunicação via *topic*s permite que os *nodes* tenham continuamente atualizações dos dados, os serviços apenas fornecem os dados quando são especificamente pedidos pelo cliente. Por outro lado, o sistema via *services* apenas permite, para o mesmo serviço, apenas um *node server.*

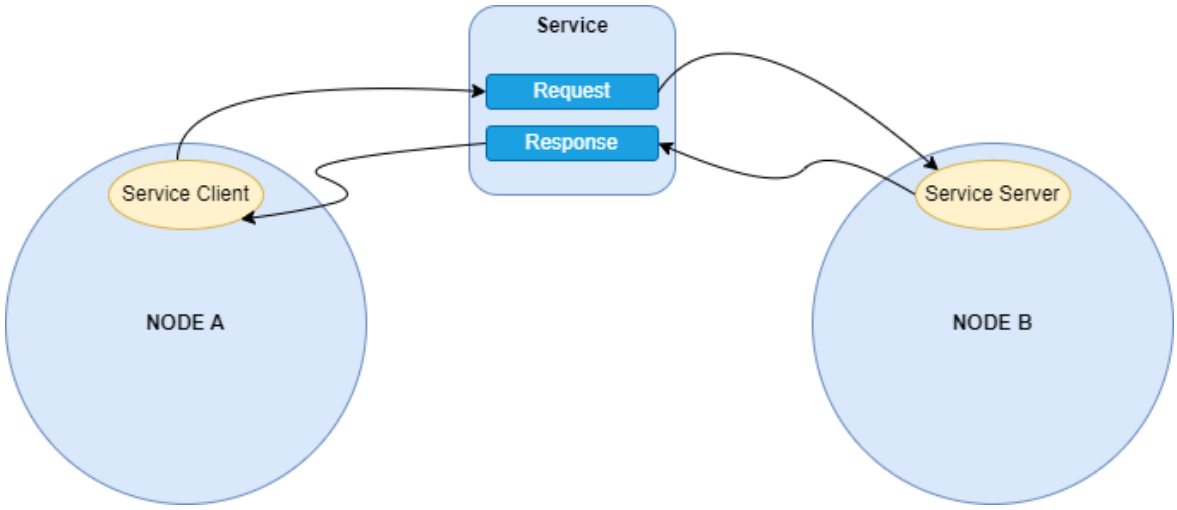


Figura 28: Funcionamento da comunicação via ROS Service.

Para o funcionamento de um serviço, é necessário recorrer a arquivos ". srv" para definir a estrutura do serviço. Dentro do arquivo, os ficheiros têm uma estrutura a cumprir, igual à presente na Figura 29.

```
<srv_name>

# tipo da mensagem pedida
---
# tipo da mensagem de resposta
```

Figura 29: Formato utilizado pelos ficheiros .srv.

Esta descrição inclui detalhes sobre os tipos de dados que o serviço espera receber como entrada (*request*), que se encontram antes dos traços "- - -", e o que ele retornará como saída (*response*), situa-se nas linhas depois dos traços "- - -".

3.2.5. O QUE É A COMUNICAÇÃO VIA ROS ACTIONS?

As *actions* são outro tipo de comunicação no ROS 2, e destinam-se a tarefas de longa duração. São compostas por três partes: um *goal*, o *feedback* e o *result*. Igualmente à comunicação via ROS Service, este mecanismo também é construído com base em *client/server* e bidirecional, no entanto apenas de forma assíncrona. Possui ainda uma funcionalidade extra, que é a possibilidade de cancelamento de pedido. Além disso, fornecem o *feedback* constante, ao contrário dos serviços que retomam apenas uma resposta.

De modo a ilustrar a comunicação de forma simples, apresenta-se na Figura 30 um exemplo simplificado desta comunicação.

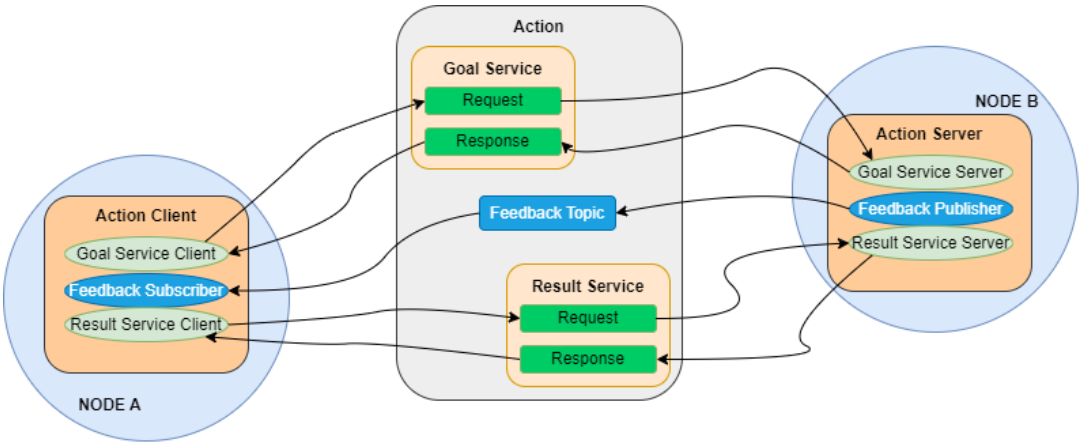


Figura 30: Funcionamento da comunicação via ROS Actions.

O esquema acima retrata um exemplo simplificado de uma comunicação entre dois *nodes.* O processo inicia-se quando o nó *"Action Client"* (NODE A) envia um *goal* para o nó *"Goal Service"*; este, por sua vez, reconhece o *goal* e comunica o pedido ao nó *"Action Server"* (NODE B), e, por fim, este último devolve o *feedback* e o resultado do pedido.

De igual forma ao funcionamento de um serviço, para o funcionamento das *actions* é necessário recorrer a arquivos ". *action*" para definir a estrutura da ação. Dentro do arquivo, os ficheiros têm de cumprir a estrutura igual à ilustrada na Figura 31, que contém as mesmas informações descritas no capítulo acima, acrescendo a este a mensagem de *feedback*.

```
<action_name>
# tipo da mensagem pedida
---
# tipo da mensagem de resposta
---
# tipo da mensagem de feedback
```

Figura 31: Formato utilizado para os ficheiros .action.

3.2.6. O QUE É UM PACKAGE?

Um *package* (ou pacote) é uma unidade estrutural para o código ROS 2. Para instalar o código ou compartilhá-lo com outras pessoas, é necessário organizá-lo em pacotes. Com *packages*, é possível partilhar o trabalho no ROS e permitir que outros o construam e utilizem facilmente. A criação de pacotes no ROS 2 utiliza o *ament* como sistema de construção e o *colcon* como ferramenta de construção. Pode-se criar um pacote usando *CMake* ou *Python*, que são oficialmente suportados.

Um *workspace* pode conter vários *packages*, cada um numa pasta própria. Também é possível ter pacotes de diferentes tipos de construção num único *workspace* (*CMake*, *Python*). Contudo, não é permitido ter pacotes dentro de pacotes.

A melhor prática é ter um diretório *src* dentro do seu *workspace*, onde os pacotes são criados. Isso mantém o nível superior do *workspace* organizado. Um exemplo de estrutura de um *workspace* está presente na Figura 32. Neste exemplo, é demonstrada a coexistência de pacotes *CMake* e *Python*, bem como a estrutura a seguir.

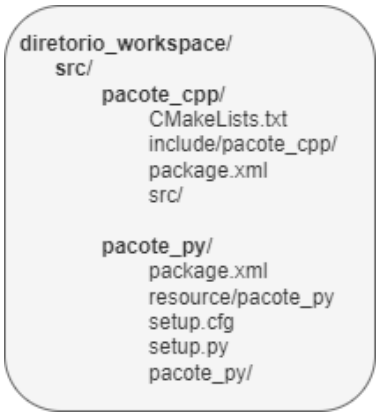


Figura 32: Exemplo da estrutura de um *workspace*.

Com esta organização, a manutenção e a colaboração no desenvolvimento de software utilizando ROS 2 tornam-se mais estruturadas, sendo mais facilmente compreendida por utilizadores novos.

3.3. SIMULADOR

A compreensão dos fenómenos que ocorrem no mundo ao nosso redor impulsionou a criação de ferramentas de software dedicadas à simulação de sistemas e modelos físicos. A simulação é realizada para projetar e executar um sistema físico, seja ele real ou teórico, permitindo a análise e a compreensão de seu comportamento. Este processo tem um papel essencial para a compreensão do comportamento de sistemas complexos e para a identificação de possíveis falhas antes da implementação real. Com o avanço das tecnologias, os *softwares* de simulação tornaram-se mais sofisticados, possibilitando a criação de cenários tridimensionais detalhados e interativos.

A simulação teve um papel importante neste trabalho, resultando num modelo de cenário criado no *CoppeliaSim*. Entre as várias ferramentas de simulação disponíveis, o *CoppeliaSim* destaca-se como uma escolha popular no campo da robótica. Este software oferece uma plataforma versátil e escalável para o desenvolvimento e a validação de algoritmos e sistemas robóticos. Uma das suas principais vantagens é

a capacidade de importar modelos CAD e monitorar variáveis em tempo real através, da sua interface gráfica intuitiva. O *CoppeliaSim* inclui a possibilidade de utilização de motores de física, tornando a simulação mais realista.

O *CoppeliaSim* suporta múltiplos sistemas operacionais, o que o torna acessível a uma ampla gama de utilizadores. Permite ainda a comunicação com outros *softwares* e sistemas de controlo via ROS ou bibliotecas *RemoteApi*, facilitando a integração em projetos robóticos complexos.

Um aspeto particular do *CoppeliaSim* é a possibilidade de replicar virtualmente um objeto físico, permitindo que o controlo e a análise sejam realizados em ambiente simulado antes da aplicação prática.

A arquitetura do *CoppeliaSim* é composta por três elementos principais:

1. **Objetos de cenário**: os objetos de cenário são os componentes básicos que compõem o ambiente de simulação no *CoppeliaSim*, podendo incluir:

    a. Modelos Robóticos: representações virtuais de robôs que podem variar desde manipuladores até robôs móveis.

    b. Elementos Estruturais: partes do ambiente como paredes, pisos, obstáculos e plataformas.

    c. Objetos Interativos: itens que podem ser manipulados pelos robôs, como caixas, ferramentas e peças de montagem.

    d. Sensores e Atuadores: dispositivos que permitem aos robôs perceber e interagir com o ambiente, incluindo câmaras, lasers, detetores de toque, motores e servos.

Exemplos das possibilidades dos objetos de cenário existentes na biblioteca, encontram-se na Figura 33.

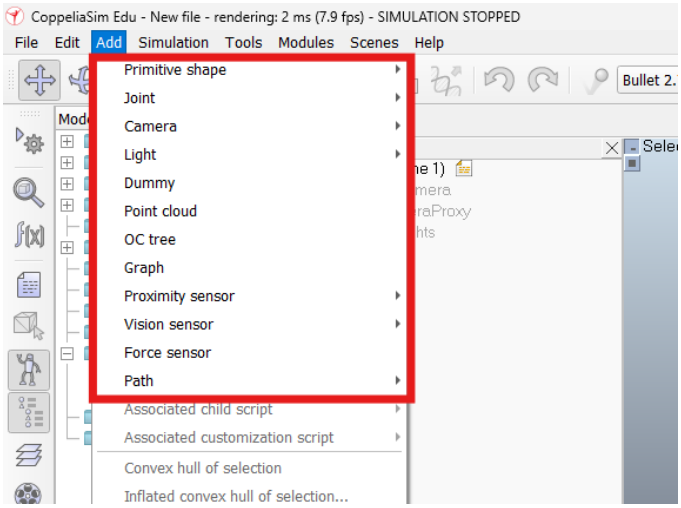


Figura 33: Biblioteca de objetos de cenário suportados pelo *CoppeliaSim*.

Cada objeto de cenário pode ser configurado com propriedades específicas, como massa, fricção e dinâmica, que afetam o seu comportamento durante a simulação, como mostra a Figura 34. A capacidade de importar modelos CAD permite que objetos complexos e detalhados sejam facilmente integrados ao cenário, proporcionando uma simulação mais realista.

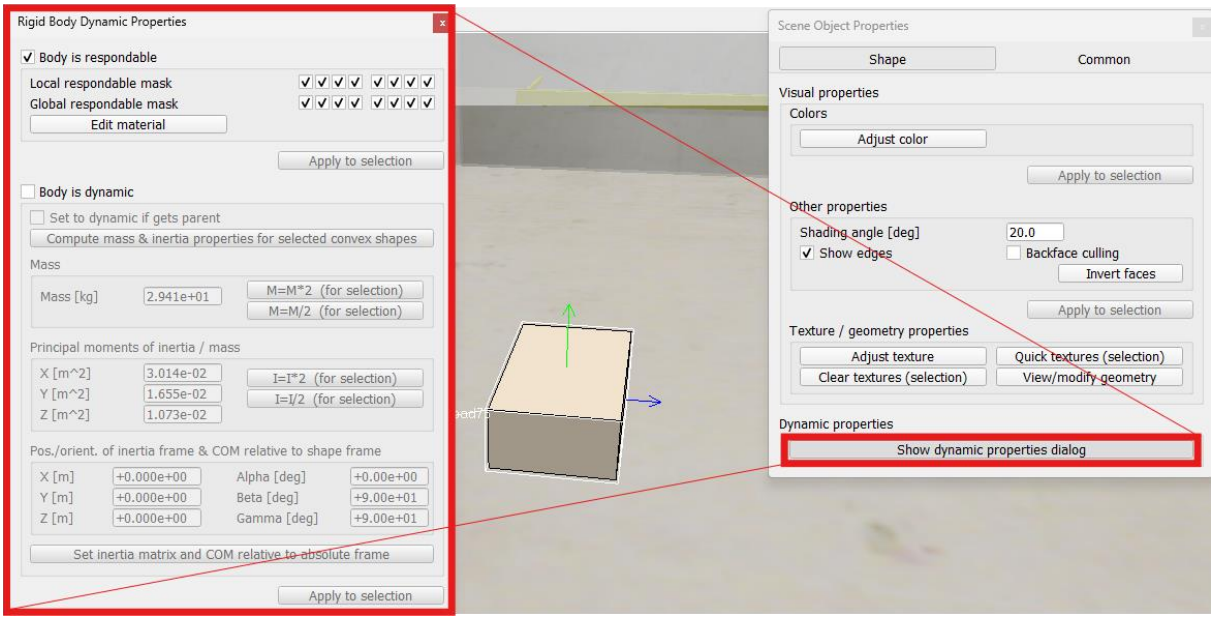


Figura 34: Propriedades dos objetos cenários do *CoppeliaSim*.

2. **Módulos de cálculo**: os módulos de cálculo são responsáveis por processar as simulações físicas e cinemáticas dos objetos de cenário, e incluem:

    a. Cálculo das interações físicas entre os objetos, como colisões, gravidade, forças e binários.

b. Cálculo da Cinética Direta/Inversa;

c. Cálculo das distâncias mínimas entre as formas físicas (*shapes*);

d. Simulação dos dados recebidos dos sensores.

3. **Mecanismos de controlo**: os mecanismos de controlo permitem a implementação e a execução de algoritmos de controlo que administram o comportamento dos robôs e outros objetos no cenário. As principais interfaces são:

   a. *Embedded Script*: algoritmos de customização que podem ser diretamente programados e executados dentro do CoppeliaSim utilizando *scripts* em *Lua* ou em *Python*. Por defeito, um cenário precisa de uma *script* principal, pode ainda ter outras *scripts* "filhos".

      - *Script* Principal: este *script* é utilizado para o funcionamento do simulador. Ainda, é responsável por iniciar e gerir a simulação, bem como invocar outros *scripts* associados aos objetos do cenário. O script principal gere a lógica global do cenário e é utilizado para configurar parâmetros iniciais, controlar o fluxo da simulação e monitorizar eventos de alto nível;

      - *Scripts* Filhos: ao contrário do *script* principal, os *scripts* filhos são associados diretamente a objetos ou modelos específicos no cenário. Cada objeto pode ter seu próprio *script* filho, que controla o seu comportamento individual. Esses scripts podem responder a eventos específicos, realizar cálculos em tempo real e ajustar parâmetros do objeto durante a simulação. A capacidade de associar *scripts* a objetos específicos permite uma elevada precisão de controlo e personalização no comportamento dos modelos.

   b. Interface com Sistemas Externos: o *CoppeliaSim* comunica com sistemas externos via *ROS*, *RemoteApi*, ou *Plug-ins*.

      - ROS: o *CoppeliaSim* pode ser integrado ao ROS, que oferece ferramentas e bibliotecas para o desenvolvimento de aplicações robóticas complexas. A integração com ROS permite que o *CoppeliaSim* seja utilizado como uma plataforma de desenvolvimento e teste para algoritmos robóticos que são

implementados posteriormente em robôs reais. Através do ROS, o *CoppeliaSim* pode enviar e receber dados de sensores, estado dos robôs e comandos de controlo.

- *RemoteApi*: a *RemoteApi* permite que *scripts* e programas externos controlem a simulação no *CoppeliaSim*, possibilitando a execução de algoritmos de controlo em diferentes linguagens de programação e em diferentes plataformas computacionais. A *RemoteApi* facilita a integração com outros softwares e sistemas, possibilitando a criação de simulações distribuídas e colaborativas.

- *Plug-ins*: o *CoppeliaSim* suporta a utilização de *plug-ins* de modo a estender as funcionalidades e integrar-se com outros softwares e ferramentas. Os *plug-ins* podem ser desenvolvidos para adicionar novos recursos ao simulador, melhorar o desempenho ou permitir a comunicação com sistemas específicos. Esta flexibilidade ajuda a adaptar o simulador às necessidades dos projetos e para incorporar novas tecnologias à medida que se tornam disponíveis.

A integração destes três elementos – objetos de cenário, módulos de cálculo e mecanismos de controlo – permite que o *CoppeliaSim* se torne numa plataforma robusta e flexível para simulação robótica. A arquitetura modular do *software* facilita a personalização e a adaptação para diferentes tipos de projetos, desde contexto académico até aplicações industriais.

Ademais, o *CoppeliaSim* dispõe de um motor de física que desempenha um papel fundamental para uma simulação precisa em ambientes virtuais, sendo essencial para o desenvolvimento e validação de sistemas robóticos. O simulador suporta múltiplos motores de física, incluindo o *Bullet*, *ODE* (*Open Dynamics Engine*), *Vortex* e *Newton*, permitindo assim a seleção do motor de física mais apropriado, de acordo com os requisitos específicos da simulação, seja a precisão ou o desempenho.

Uma das principais vantagens do motor de física do *CoppeliaSim* é a capacidade de fornecer simulações realistas, o que é crítico para projetos de automação e robótica colaborativa. As interações físicas entre objetos são calculadas em tempo real, permitindo que os robôs e outros sistemas dinâmicos se comportem de maneira semelhante ao que ocorreria em condições reais. Isso inclui a simulação de forças, binários, velocidades e acelerações em sistemas móveis, manipuladores robóticos, entre outros.

Além disso, o motor de física possibilita informação detalhada de colisões e do atrito, permitindo simulações robustas que consideram a interação entre o robô e seu ambiente. Sendo estes aspetos relevantes para a verificação dos cenários de navegação e manipulação, onde a precisão no comportamento físico pode determinar o sucesso ou falha de um sistema robótico.

O *CoppeliaSim* também permite ajustes nos parâmetros físicos, como gravidade, resistência do ar e atrito, proporcionando flexibilidade na configuração de ambientes de simulação específicos. Esses recursos tornam a plataforma ideal para aplicações que exigem alta-fidelidade física, como o desenvolvimento de sistemas autónomos e investigação avançada de interações entre robôs e ambientes em contextos industriais dinâmicos.

Dessa forma, o motor de física no *CoppeliaSim* é uma ferramenta crucial para validar e otimizar o desempenho de sistemas robóticos antes de sua implementação em cenários do mundo real, garantindo segurança e eficiência no processo de automação.

4. FUNDAMENTOS TEÓRICOS

Neste capítulo são explorados os fundamentos teóricos essenciais para o desenvolvimento da solução proposta. Primeiramente, são descritos os elementos necessários para a identificação de objetos específicos no espaço. De seguida, discute-se a estratégia subjacente à navegação autónoma em ambientes dinâmicos, isto é, os sistemas dinâmicos não-lineares, salientando os métodos de controlo e conceitos inerentes à abordagem.

4.1. *POSE* DE UM VEÍCULO NO ESPAÇO

A identificação da *pose* (posição e orientação) de um veículo envolve técnicas e tecnologias - como o uso do GPS, IMU, odometria entre outras – dependendo do contexto. Por exemplo, a localização de um veículo, como um drone, pode ser determinada através das coordenadas GPS (Sistema de Posicionamento Global), que utilizam sinais de satélites para estabelecer a sua posição geográfica.

Num plano tridimensional, a *pose* do veículo é descrita por duas componentes:

1. Posição cartesiana, $p = (x, y, z)$
2. Orientação espacial, $\gamma = (\Psi_{roll}, \Theta_{pitch}, \phi_{yaw})$

No âmbito deste projeto, a localização da plataforma a ser utilizada é descrita da mesma forma que a componente apresentada anteriormente. Como se trata de uma plataforma omnidirecional, apresenta três graus de liberdade – movimento longitudinal, movimento lateral e movimento angular (segundo eixo oz) – e, por conseguinte, é descrito pelo vetor posição $p = (x, y, z)$ e pela orientação da plataforma relativamente ao mundo que se encontra inserido, ϕ_{yaw}. Assume-se que a posição z é constante, dado que a plataforma não se desloca sobre esse eixo e, portanto, o seu valor é igual à altura do centro de massa relativamente ao nível do chão.

De forma a ser mais percetível a interpretar e visualizar a *pose* da plataforma, a mesma está representada na Figura 35.

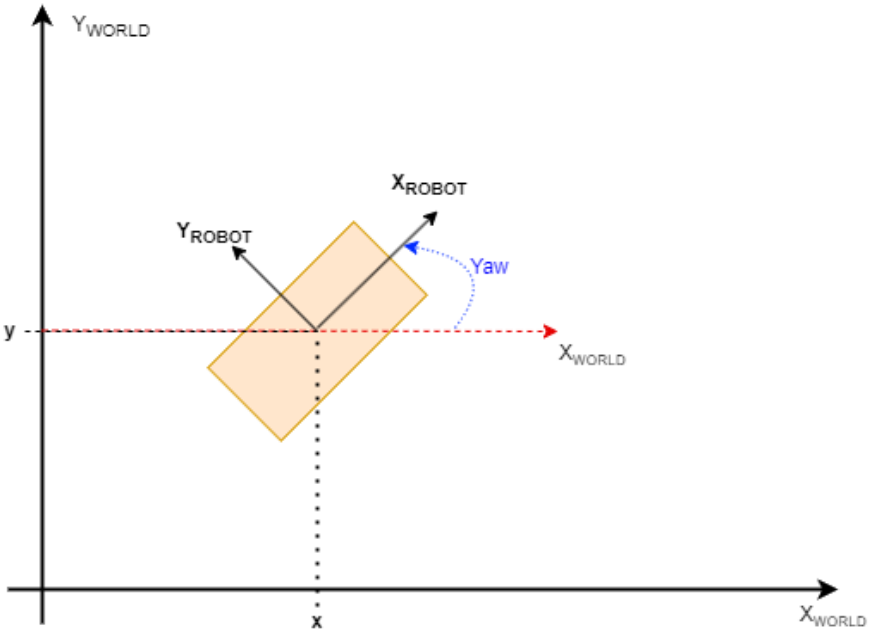


Figura 35: Localização da plataforma em um plano bidimensional.

Assim, a *pose* da plataforma é apresenta pelas coordenadas da posição $p = (x, y)$ e pela sua orientação ϕ_{yaw}, tal como descrito acima.

4.2. Sistemas Dinâmicos Não-Lineares

A navegação autónoma deve incluir métodos de controlo que capacitem a movimentação do veículo em ambientes dinâmicos, neste caso fabril. Devem então ser métodos robustos e capazes de se adaptarem às alterações dos cenários, sem reduzir a segurança.

Tendo em conta o objetivo deste projeto, a ferramenta matemática utilizada para garantir o controlo da navegação dos veículos são os sistemas dinâmicos não lineares, baseada em Bicho (1999).

Para controlar o movimento da plataforma usam-se aqui como variáveis de estado (ou variáveis de controlo) – também chamadas por vezes variáveis comportamentais – a direção de navegação, $\boldsymbol{\phi_{veíc}}$, e a velocidade linear, $\boldsymbol{v_x}$, do robô. A evolução temporal destas variáveis é gerida por sistemas dinâmicos não lineares, na forma de equações diferenciais ordinárias, equações (2) e (3), enunciadas em Bicho, 1999:

$$\frac{d\phi_{veic}}{dt} = \mathrm{F}(\phi, \text{parâmetros}) \ = f_{alvo}(\phi_{veíc}) + \ f_{obs}(\phi_{veíc}) \tag{2}$$

$$\frac{dv_x}{dt} = \mathrm{G}(v, \text{parâmetros}) = c_{alvo} g_{alvo}(v_x) + c_{obs} g_{obs}(v_x) \tag{3}$$

Nas equações apresentadas os parâmetros dependem, em geral, da informação sensorial, na medida em que os respetivos campos vetoriais variam à medida que o robô se desloca ou o ambiente se altera (isto é, ambientes dinâmicos). Ainda, o número de pontos fixos, a sua posição e a respetiva estabilidade variam à medida que o robô desloca-se ou o ambiente altera-se. Portanto, a dinâmica da direção de navegação, $\mathrm{F}(\phi, \text{parâmetros})$, depende simultaneamente da contribuição do comportamento de convergência para o alvo, $f_{alvo}(\phi_{veíc})$, bem como do contributo do comportamento de evitar colisões, $f_{obs}(\phi_{veíc})$. Não obstante, a dinâmica de $\mathrm{G}(v, \text{parâmetros})$, depende dos comportamentos da direção de navegação e do desvio de obstáculos. A dinâmica da velocidade possui duas variáveis c_{alvo} e c_{obs}, que são denominadas de variáveis de ativação e, em função do estado do sistema, o peso de cada uma varia com o comportamento do sistema.

Mudanças qualitativas no comportamento do robô como, por exemplo, passar entre obstáculos ou contorná-los. São geradas induzindo bifurcações adequadas nos campos vetoriais, em função dos parâmetros que dependem da informação sensorial. Os valores desejados para as variáveis de estado (como a direção em que o alvo é visto) são atribuídos como atratores do sistema dinâmico e valores indesejados são dados como repulsores, ou seja, as direções em que são detetados obstáculos.

Assim, esta abordagem permite que o planeador opere sem conhecimento prévio do ambiente onde vai navegar, dependendo apenas da informação sensorial. Isto possibilita que o sistema se ajuste às alterações do ambiente, como as que ocorrem em um ambiente fabril, que é o caso de estudo deste projeto.

Um robô móvel autónomo, tem de ser capaz de navegar para uma posição alvo. A informação sobre o alvo pode ser definida pelas coordenadas do alvo (x_{alvo}, y_{alvo}), ou o sistema de sensores pode indicar em que direção está a detetar o alvo, como é possível ver na Figura 36.

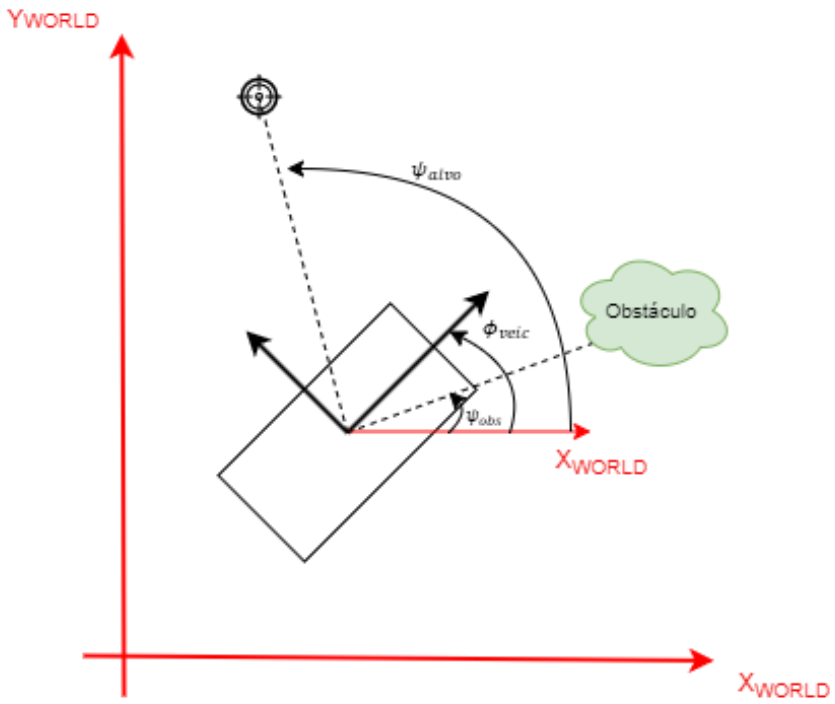


Figura 36: Sistema a controlar.

A dinâmica depende de dois comportamentos, o de convergência em direção à posição de um alvo e o de evitar colisões com obstáculos, como retratado na figura acima citada.

4.2.1. Controlo da Orientação de Navegação

O controlo da orientação de navegação é definido pela dinâmica $\mathrm{F}(\phi_{veic}, \text{parâmetros})$, sendo dividida em dois comportamentos:

1. **Navegação na direção de uma posição alvo**, $f_{alvo}(\phi_{veic})$;

Para a implementação da dinâmica $f_{alvo}(\phi_{veic})$, consideram-se alvos representados como coordenadas cartesianas, definidas no referencial do mundo (cenário). De modo a isto ser exequível, o planeador global (ou seja, a quem fornece os alvos a seguir) tem de conhecer previamente as posições para as quais deseja que o robô se dirija. Face ao exposto, o modelo matemático, desenvolvido por Bicho (1999), é descrito pela equação (4):

$$\frac{d\phi_{veic}}{dt} = f_{alvo}(\phi_{veic}) = -\lambda_{alvo}\sin(\phi_{veic} - \psi_{alvo}) \tag{4}$$

ψ_{alvo}, representa a orientação de navegação desejada para a posição alvo. Para calcular a direção do alvo face ao veículo, segue-se a equação (5):

$$\psi_{alvo} = tan^{-1}\left(\frac{y_{alvo} - y_{veic}}{x_{alvo} - x_{veic}}\right) \tag{5}$$

Como apresenta a equação, para o cálculo da direção do alvo, é necessário ter o conhecimento da posição do veículo no espaço, (x_{veic}, y_{veic})., e da coordenada do alvo, (x_{alvo}, y_{alvo}). Como ψ_{alvo} representa a orientação de navegação desejada, a dinâmica $f_{alvo}(\phi_{veíc})$ deve atribuir o ponto atractor em $\phi_{veíc} = \psi_{alvo}$.

No que diz respeito a λ_{alvo}, este é o parâmetro que define a magnitude da "força de atração" que o alvo exerce sobre a plataforma móvel. Perante isto, de modo que os requisitos sejam cumpridos e a plataforma consiga navegar até uma posição-alvo, o valor de $\lambda_{alvo} > 0$.

2. **Evitar colisões com obstáculos**, $f_{obs}(\phi_{veíc})$.

Para gerar um comportamento que garanta que a plataforma evite obstáculos, é necessário controlar o seu movimento de modo a detetar as direções onde se encontram os obstáculos e, consequentemente, evitá-los. Este comportamento é definido pela dinâmica $f_{obs}(\phi_{veíc})$. Neste projeto, como foram utilizados dois *lidars* em vez dos sensores de infravermelhos usados em Bicho (1999), foi necessário converter os feixes laser para os setores de interesse na dinâmica. Como são utilizados dois sensores *Sick* S300 *Expert* e, cada um emite 540 feixes, o método utilizado em Correia (2023) permite reduzir o número de sectores para a quantidade de interesse para a dinâmica de desvio de obstáculos.

A dinâmica do comportamento para evitar obstáculos é governada por:

$$f_{obs_{total}}(\phi_{veíc}) = \sum_{i=1}^{n} f_{obs,i}(\phi_{veíc}) + f_{stock} \tag{6}$$

Onde:

$$f_{obs,i}(\phi_{veíc}) = \lambda_{obs,i}(\phi_{veíc} - \psi_{obs,i})e^{-\frac{(\phi-\psi_{obs,i})^2}{2\sigma_i^2}} \tag{7}$$

Que é usada para erigir um repulsor na direção em que a obstrução é sentida,

- $\boldsymbol{\psi_{obs,i}}$ é a direção em que o setor *i* aponta,

- -$\boldsymbol{\sigma_i}$ é a gama angular sobre a qual a força criada pelo obstáculo/obstrução exerce seu efeito repulsivo e é dada por:

$$\sigma_i = \arctan\left(\tan\left(\frac{\Delta\theta}{2}\right) + \frac{\left(\frac{Largura}{2}\right)}{d_i + \left(\frac{Comprimento}{2}\right)}\right) \tag{8}$$

Sendo:

- $\Delta\boldsymbol{\theta}$ a largura angular de um sector

- $\boldsymbol{Largura}$ a largura da plataforma móvel e $\boldsymbol{Comprimento}$ o respetivo comprimento

- $\boldsymbol{d_i}$ a distância estimada pelo setor *i*.

Não só, mas também, tem-se que:

- $\boldsymbol{\lambda_{obs,i}}$ é o parâmetro que define a magnitude da força de repulsão criada por uma obstrução detetada pelo setor *i*. Este parâmetro deve apresentar valores grandes para pequenas distâncias e valores pequenos para grandes distâncias, e zero para $d_i \geq d_{max}$ (limite de alcance dos sensores). Por outras palavras, o seu valor deve ser maior que zero, e deve diminuir em função do aumento da distância ao obstáculo.

O valor da magnitude da força de repulsão é modelado pela seguinte função exponencial:

$$\lambda_{obs,i} = \beta_1 * e^{-d_i/\beta 2} \tag{9}$$

Sendo que:

- $\boldsymbol{\beta_1}$ representa a força máxima de repulsão

- $\boldsymbol{\beta_2}$ define a taxa de decaimento em função da distância ao obstáculo.

Por vezes, pode acontecer que o ponto de bifurcação seja coincidente com a orientação da navegação, estabilizando nesse ponto, podendo o mesmo ser do tipo instável. Em Bicho (1999) foi apresentada uma solução, que incide em introduzir um ruído branco gaussiano, de variância unitária, com uma força estocástica de variância efetiva.

A força estocástica é representada por:

$$f_{stock} = \sqrt{Q}\,\zeta_n(t) \tag{10}$$

Onde:

- $\boldsymbol{\zeta_n}$ é ruído branco gaussiano
- Q é a variância efetiva (magnitude) do ruído.

Após explicação detalhada dos dois comportamentos (convergir para o alvo e desviar de obstáculos) é necessário proceder à sua integração, uma vez que o veículo tem de ser capaz de alcançar o alvo numa determinada coordenada, desviando-se simultaneamente de obstáculos. Desta forma, a dinâmica que integra os dois comportamentos, provém da equação (2), que originou na equação (11).

$$\frac{d\phi_{veic}}{dt} = \mathrm{F}(\phi, \text{parâmetros}) = f_{alvo}(\phi_{veíc}) + f_{obs_{total}}(\phi_{veíc}) + f_{stock} \tag{11}$$

Sendo esta dinâmica definida por dois comportamentos, o desvio de obstáculos, $f_{obs,i}(\phi_{veíc})$, deve ter um peso superior face a $f_{alvo}(\phi_{veíc})$. Desta forma, a força de repulsão deve ser superior à taxa de relaxamento que diz respeito à convergência para o alvo, ou seja, $\lambda_{alvo} \ll \lambda_{obs}$.

4.2.2. Controlo da Velocidade Linear

A operação num ambiente dinâmico requer mais segurança, estabilidade e adaptabilidade no que toca à navegação. Consequentemente, acrescenta-se uma dinâmica de controlo da velocidade linear baseada na equação (3). Esta dinâmica permite ter comportamentos distintos em duas situações: convergir para o alvo e o desviar de obstáculos e, portanto, só um comportamento pode estar ativo de cada vez.

À medida que a plataforma se move, a informação sensorial varia, fazendo com que os atratores e repulsores se desloquem. O mesmo ocorre quando a plataforma está parada e os obstáculos se movem. Como mencionado por Bicho (1999), para que o sistema mantenha a sua estabilidade (estado assimptoticamente estável), ou seja, permaneça em ou próximo de um atrator, a velocidade de deslocamento dos pontos fixos deve ser controlada de forma a permitir que o sistema dinâmico siga de perto um dos atratores à medida que este se desloca. É natural e verdadeiro em todas as abordagens de robótica autónoma que obstáculos dinâmicos só podem ser evitados se a velocidade for inferior a um valor crítico, enunciou Bicho (1999). Desta forma, para tornar isto possível, é necessário controlar a velocidade de translação do veículo. A velocidade linear relativa da plataforma em relação ao ambiente

que a rodeia determina a velocidade de deslocamento dos pontos fixos. Desta forma, a velocidade linear é calculada pela equação (12).

$$\frac{dv_x}{dt} = c_{alvo} \times (v_x - v_{alvo}) + c_{obs} \times (v_x - v_{obs}) \tag{12}$$

Visto que só pode estar um comportamento ativo, neste sistema dinâmico, o campo vetorial contém dois termos que representam forças atrativas (por outras palavras dois atratores que "puxam" a velocidade da plataforma para os valores desejados v_{alvo} e v_{obs}), com intensidades de atração c_{alvo} e c_{obs}, respetivamente. Estas intensidades são ajustadas de modo que, na presença de fortes influências de obstáculos, a segunda força predomina (velocidade em função da distância a obstáculos), enquanto na ausência dessas influências, a primera força se torna dominante (velocidade em função da distância a alvos ou velocidade máxima estipulada).

De modo a complementar o cálculo da velocidade linear, no comportamento de dirigir para o alvo, o valor de v_{alvo} deve tender para a velocidade máxima estipulada. Porém, se estiver perto do último alvo, a velocidade deve reduzir de forma proporcional à distância para o alvo.

No que toca ao comportamento de desvio de obstáculos, v_{obs} varia relativamente ao obstáculo que se encontra mais perto da plataforma. Esta velocidade deve ser limitada, de modo a $v_{obs} < v_{alvo}$.

Uma abordagem para criar uma função que determine a presença de contribuições de obstáculos é integrar as forças de repulsão dos mesmos. Isto resulta em uma função potencial, equação (13), do sistema dinâmico, que produz o comportamento de "evitar colisões com obstáculos".

$$U(\phi_{veíc}) = \sum_{i=1}^{n} \left[\lambda_{obs,i}({\sigma_{obs,i}}^2) \times e^{-\frac{(\phi_{veíc} - \psi_{obs,i})^2}{2 \times ({\sigma_{obs,i}}^2)}} - \frac{\lambda_{obs,i} \times {\sigma_{obs,i}}^2}{\sqrt{e}} \right] \tag{13}$$

Valores positivos desta função indicam que a direção de navegação está dentro de uma zona repulsiva criada pelos obstáculos, necessitando que $c_{obs} > 0$ e $c_{alvo} = 0$. Por outro lado, valores negativos do potencial indicam que a direção de navegação está fora da zona repulsiva ou que as forças repulsivas dos obstáculos são fracas, fazendo com que $c_{obs} = 0$ e $c_{alvo} > 0$ sejam necessários nesse caso.

Para converter os níveis de potencial em valores de intensidade dessas forças de atração, é utilizada uma função sigmoidal, como mostra a equação 14, estando limitada entre os valores $-\frac{1}{2}$ e $\frac{1}{2}$.

$$\alpha(\phi) = \frac{arctan[c \times U(\phi)]}{\pi} \tag{14}$$

Onde:

- $\boldsymbol{c}$ é o parâmetro que controla o declive máximo.

Assim, as intensidades das forças que contribuem para a dinâmica da velocidade são calculadas seguindo as seguintes expressões:

$$c_{obs} = c_{v,obs}\left(\frac{1}{2} + \alpha(\phi)\right) \tag{15}$$

$$c_{alvo} = c_{v,alvo}\left(\frac{1}{2} - \alpha(\phi)\right) \tag{16}$$

Os parâmetros $c_{v,obs}$ e $c_{v,alvo}$, especificam a taxa de relaxação da dinâmica da velocidade nas duas situações, quando as contribuições dos obstáculos ou do alvo dominarem, respetivamente. Existe uma hierarquia para estas taxas de relaxação, segundo

$$\lambda_{alvo} \ll c_{v,alvo} \qquad \lambda_{obs} \ll c_{v,obs} \qquad \lambda_{alvo} \ll \lambda_{obs}$$

4.3. Navegação Holonómica

Os veículos omnidirecionais possuem uma flexibilidade e manobrabilidade superiores, o que lhes garante uma maior liberdade de deslocamento. Esta manobrabilidade é descrita tanto pelo movimento angular em torno do seu próprio centro de massa, quanto pelo vetor de velocidade linear, que é dividido nas suas componentes cartesianas.

Para garantir o controlo holonómico do veículo, permitiu-se a movimentação em três graus de liberdade, permitindo movimentos nas suas componentes cartesianas, assim como na velocidade angular. Desta forma, a estratégia de controlo é repartida em duas frentes complementares:

- Controlo pela orientação do vetor de velocidade linear;
- Controlo pela orientação do próprio veículo (movimento angular).

Esta estratégia seguiu o mesmo método de controlo da navegação não-holonómica, que se baseia em uma dinâmica de navegação baseada em sistemas dinâmicos não lineares. Isto é, permite navegar para um alvo desejado e evitar a colisão com obstáculos. Teve por base a implementação realizada por Correia (2023), sendo que foi acrescentado ao controlo pela orientação do próprio veículo, o movimento angular poder assumir um valor diferente da orientação em relação alvo, como por exemplo a orientação do corredor.

4.3.1. Controlo da Direção do Vetor Velocidade Linear na Navegação Holonómica

Inicialmente, no que diz respeito à orientação do vetor velocidade linear para controlar a movimentação da navegação holonómica, este resulta das componentes cartesianas, v_x e v_y, que podem ser obtidas através das equações (17) e (18).

$$v_x = v_{linear} \times \cos(\Phi_{resultante}) \tag{17}$$

$$v_y = v_{linear} \times \sin(\Phi_{resultante}) \tag{18}$$

A formulação do vetor velocidade linear é obtido pela conversão retangular das componentes cartesianas do vetor v_{linear}, como está presente na Figura 37. Ou seja, é definido pelo módulo da velocidade linear pretendida e pela amplitude angular face à orientação *yaw*, (frontal), do robô.

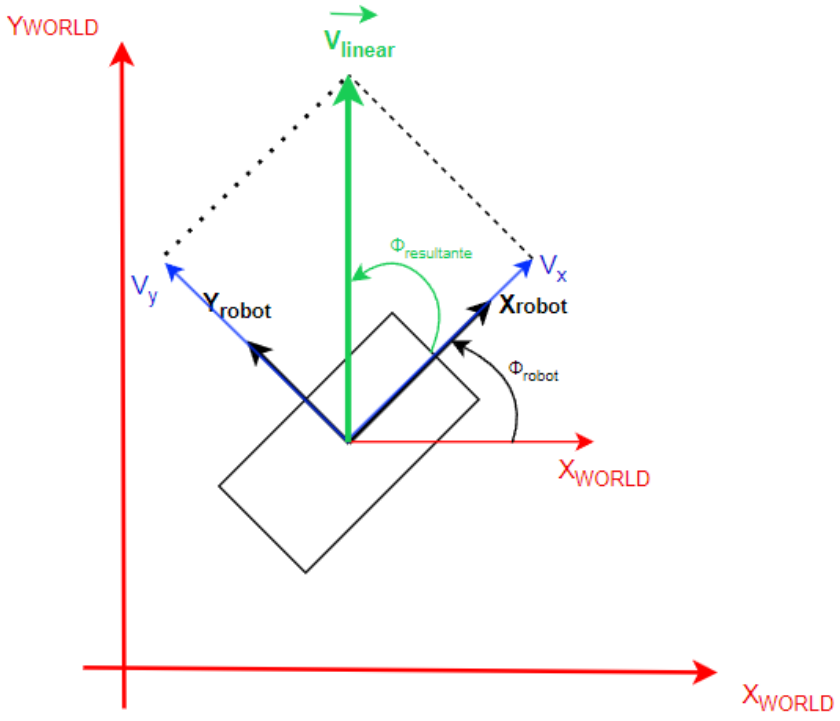


Figura 37: Dimensionamento do vetor velocidade linear bidimensional.

A orientação do movimento linear do veículo ($\Phi_{resultante}$) é definida pelo vetor velocidade. O comportamento desta dinâmica funciona da mesma forma que a navegação diferencial, ou seja, dirige--se na direção de um alvo e evita a colisão com obstáculos, com a diferença que a orientação real de navegação pode não ser a mesma que a orientação da plataforma, uma vez que a orientação de navegação será resultante da soma entre $\Phi_{resultante}$ e Φ_{robot}, como mostra a Figura 38.

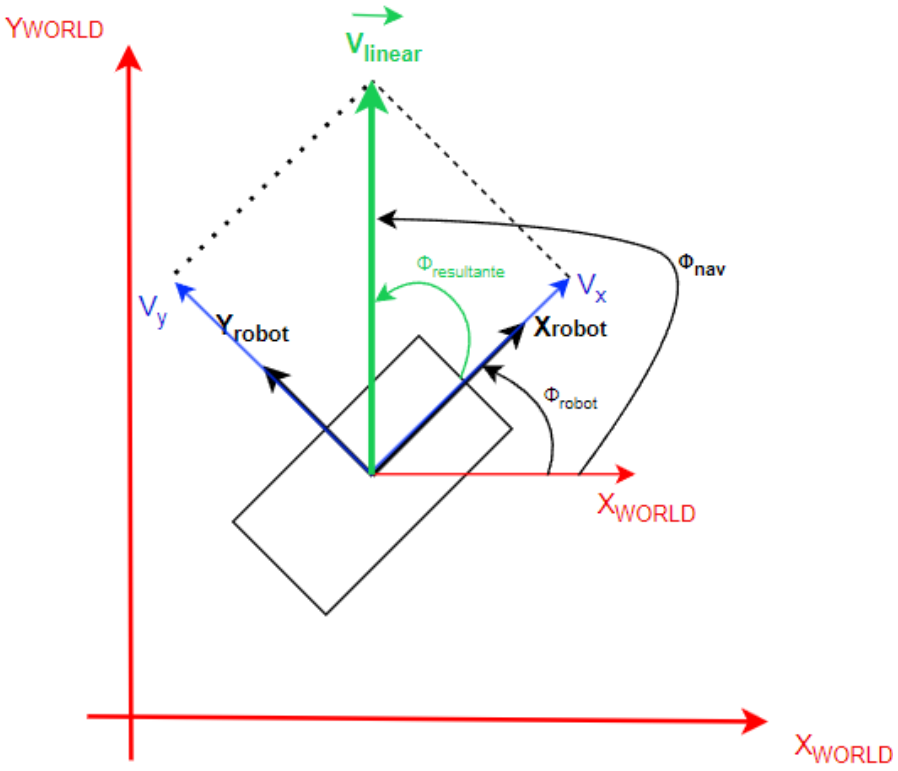


Figura 38: Representação da orientação de navegação, que é resultante da soma da orientação do robô com a orientação do vetor velocidade resultante.

Desta forma a orientação da representação pode ser apresentada como a equação abaixo:

$$\Phi_{nav} = \Phi_{resultante} + \Phi_{robot} \tag{19}$$

1. **Navegação na direção de uma posição alvo**, $f_{alvo}(\phi_{nav})$;

Para a implementação da dinâmica $f_{alvo}(\phi_{nav})$, considera-se a expressão matemática apresentada em (4), substituindo a orientação, resultando em:

$$\frac{d\phi_{nav}}{dt} = f_{alvo}(\phi_{nav}) = -\lambda_{alvo}\sin(\phi_{nav} - \psi_{alvo}) \tag{20}$$

Como ψ_{alvo} representa a orientação de navegação desejada, a dinâmica $f_{alvo}(\phi_{nav})$ deve atribuir o ponto atractor em $\phi_{nav} = \psi_{alvo}$.

2. **Evitar colisões com obstáculos**, $f_{obs}(\phi_{nav})$.

Para gerar o comportamento que faz com que a plataforma evite obstáculos, o comportamento é definido pela dinâmica $f_{obs}(\phi_{nav})$, sendo este semelhante ao apresentado na equação (6). Portanto, a dinâmica do comportamento para evitar obstáculos é governada por:

$$f_{obs_{total}}(\phi_{nav}) = \sum_{i=1}^{n} f_{obs,i}(\phi_{nav}) + f_{stock} \tag{21}$$

Onde:

$$f_{obs,i}(\phi_{nav}) = \lambda_{obs,i}(\phi_{nav} - \psi_{obs,i})e^{-\frac{(\phi_{nav}-\psi_{obs,i})^2}{2\sigma_i{}^2}} \tag{22}$$

Desta forma, e igualmente ao apresentado na equação (11), a dinâmica que integra os dois comportamentos originou a equação (22)

$$\frac{d\phi_{nav}}{dt} = \mathrm{F}(\phi, \text{parâmetros}) = f_{alvo}(\phi_{nav}) + f_{obs_{total}}(\phi_{nav}) + f_{stock} \tag{23}$$

Sendo esta dinâmica definida pelos dois comportamentos, o desvio de obstáculos, $f_{obs,i}(\phi_{nav})$, deve ter um peso superior face a $f_{alvo}(\phi_{nav})$. Desta forma, a força de repulsão deve ser superior à taxa de relaxamento que diz respeito à convergência para o alvo, ou seja, $\lambda_{alvo} \ll \lambda_{obs}$.

A estratégia apresentada baseia-se na decomposição do vetor da velocidade linear nas suas componentes cartesianas. O algoritmo calcula o ângulo de direção de navegação (ϕ_{nav}), com base na sua evolução temporal. O processo apoia-se numa simplificação da equação 22, transformando-a numa equação mais simples de calcular, assumindo o intervalo de tempo entre cálculos, denominando-se equação algébrica recursiva.

O método utilizado para resolver a equação é o de *Euler*, que permite calcular a derivada do ângulo de navegação. Este método considera que a derivada deve ser aproximada pela diferença entre o valor futuro e o valor atual do ângulo, dividida pelo intervalo de tempo, ou seja:

$$\frac{d(\phi_{nav}(t))}{dt} \approx \frac{\phi_{nav}(\Delta t + t) - \phi_{nav}(\Delta t)}{\Delta t} \tag{24}$$

Ao reorganizar a equação (24), pode-se prever o valor futuro da direção de navegação ($\phi_{nav}(\Delta t + t)$). Com base no valor atual $\phi_{nav}(\Delta t)$ e na taxa de variação $\frac{d(\phi_{nav}(t))}{dt}$ obtém-se o valor atualizado. Pode-se

então calcular o ângulo de orientação em relação ao movimento do sistema. A diferença entre o ângulo de navegação e o ângulo correspondente ao eixo de referência do sistema (normalmente, o eixo do veículo) resulta no ângulo de orientação relativo, ou seja:

$$\phi_{resultante} = \phi_{nav} - \phi_{robot} \tag{25}$$

Em resumo, o algoritmo usa o método de *Euler* para prever como o ângulo de navegação se vai alterar com o tempo e, assim, ajusta a direção do movimento conforme necessário, considerando sempre as mudanças instantâneas que ocorrem nos intervalos de tempo.

4.3.2. Controlo da Orientação da Plataforma na Navegação Holonómica

O controlo da orientação é um dos dois componentes para o controlo da navegação, permitindo ajustes de direção que complementam a movimentação. Este mecanismo baseia-se na alteração do ângulo do veículo conforme as necessidades de navegação. Além do que foi apresentado por Correia (2023), este módulo permite que a orientação da plataforma assuma valores diferentes da direção do alvo. Isso é útil devido ao cenário escolhido no projeto, uma vez que o robô, em determinadas circunstâncias, adota a orientação do corredor.

O veículo ajusta a orientação pretendida e segue para o alvo. Assumindo que tanto a orientação desejada quanto a do veículo sejam conhecidas, define-se um campo de forças dinâmico. Este campo é calculado com base na diferença angular entre a orientação atual do veículo e a orientação desejada, modulada por uma constante de relaxamento, que regula a velocidade de ajuste. O efeito dessa força atrativa é expresso como:

$$f_{alvo_w}(\phi_{robot}) = -\lambda_{alvo_w} \sin(\phi_{robot} - \psi_{desejado}) \tag{26}$$

Salienta-se que, $\psi_{desejado}$ no controlo da orientação da plataforma, não tende somente para a orientação em relação ao alvo, mas também pode tomar outros valores de orientação, tal como a orientação de um corredor, como é o caso deste projeto. Sendo esta, uma diferença, face ao trabalho realizado por Correia (2023).

Para obter o controlo da orientação final do veículo, as forças de atração para o alvo resultam na orientação do veículo no espaço, que é controlada pela variação da sua velocidade angular w_{robot},que é diretamente proporcional à soma das forças que influenciam a sua orientação.

$$w_{robot} = \frac{d\phi_{robot}}{dt} = f_{alvo_w}(\phi_{robot}) \tag{27}$$

Deve haver uma hierarquia que assegure que o ajuste do ângulo de movimento tenha maior peso que o controlo da orientação angular do veículo.

4.3.3. Controlo da Velocidade Linear Holonómica

O módulo de controlo da velocidade linear holonómica segue a mesma dinâmica apresentada para a navegação não-holonómica, ou seja, a dinâmica permite ter comportamentos distintos em duas situações: convergir para o alvo e o desviar de obstáculos e, portanto, só um comportamento pode estar ativo de cada vez.

Desta forma, a velocidade linear é calculada pela equação (28), que provém da equação 11:

$$\frac{dv_{linear}}{dt} = c_{alvo} \times (v_{linear} - v_{alvo}) + c_{obs} \times (v_{linear} - v_{obs}) \tag{28}$$

O método de controlo da velocidade linear holonómica, segue igualmente as equações 11 até à equação 15, de modo a funcionar apenas uma dinâmica de cada vez (convergir para o alvo ou desviar obstáculos).

Os parâmetros $c_{v,obs}$ e $c_{v,alvo}$, especificam a taxa de relaxação da dinâmica da velocidade nas duas situações, quando as contribuições dos obstáculos ou do alvo dominarem, respetivamente. Existe uma hierarquia para estas taxas de relaxação, segundo:

$$\lambda_{alvo} \ll c_{v,alvo} \qquad \lambda_{obs} \ll c_{v,obs} \qquad \lambda_{alvo} \ll \lambda_{obs}$$

Não obstante, essa velocidade linear é posteriormente devidamente repartida pelas duas componentes cartesianas, de modo a obter o vetor velocidade.

4.4. Navegar em Marcha-Atrás

Devido às dimensões e formato do *layout*, a plataforma tem de estar instrumentada de modo a navegar em marcha atrás. Para navegar em marcha-atrás, a principal diferença na dinâmica em relação à marcha para a frente reside nos ajustes necessários para lidar com o espelhamento das direções dos setores e a inversão das forças. O robô calcula as forças de atração e repulsão considerando a direção de movimento e ajusta a velocidade linear e orientação com base na presença de obstáculos e na distância ao alvo, garantindo segurança e eficiência para marcha-atrás.

De modo a determinar se o robô se deve mover para a frente ou de marcha atrás, efetuou-se a diferença angular entre a orientação do robô, $\phi_{veíc}$, e a orientação para o alvo, ψ_{alvo}. Caso a diferença seja maior que 90 graus, o robô deve navegar em marcha atrás. Caso contrário deve navegar em frente.

Antes de calcular o $f_{alvo}(\phi_{veíc})$, será ajustada a orientação da plataforma, adicionando $180º$ (π) para refletir a orientação, uma vez que está a navegar na direção aposta.

$$\phi_{veíc_adjusted} = \phi_{veíc} + \pi \tag{29}$$

Com a orientação corrigida, $\phi_{veíc_adjusted}$, é então a nova orientação a ser utilizada para a navegação para calcular a força de atração.

$$\frac{d\phi_{veíc_adjusted}}{dt} = f_{alvo}(\phi_{veíc_adjusted}) = -\lambda_{alvo}\sin(\phi_{veíc_adjusted} - \psi_{alvo}) \tag{30}$$

No que diz respeito à força de repulsão dos obstáculos, foi necessário converter as orientações em cada um dos setores. Desta forma, seguindo o mesmo raciocínio da força de atração, aqui deve-se somar 180° ao setor para refletir a direção oposta do movimento.

$$f_{obs,i}(\phi_{veíc_adjusted}) = \lambda_{obs,i}(\phi_{veíc_adjusted} - \psi_{obs,i})e^{-\frac{(\phi-\psi_{obs,i})^2}{2\sigma_i{}^2}} \tag{31}$$

4.5. Manobras de Acostagem

As manobras de acostagem foram formuladas da mesma forma que a navegação holonómica, com a diferença de que o controlo da velocidade linear é calculado exclusivamente com base no comportamento em presença de obstáculos, uma vez que o veículo se desloca para posições mais próximas de colisões.

O controlo da movimentação foi garantido em três graus de liberdade, permitindo movimentos nas suas componentes cartesianas, assim como na velocidade angular. Igualmente ao que foi apresentado na Navegação Holonómica, a estratégia de controlo é repartida em duas frentes:

- Controlo pela orientação do vetor velocidade linear;
- Controlo pela orientação do próprio veículo (movimentação angular).

4.5.1. Controlo da Orientação do Vetor Velocidade nas Manobras de Acostagem

O controlo da orientação do vetor de velocidade linear é definido pelas componentes cartesianas, v_x e v_y, que podem ser obtidas através das equações 17 e 18.

O vetor de velocidade linear é calculado a partir das componentes cartesianas, considerando o módulo da velocidade linear e a amplitude angular $\Phi_{resultante}$, que corresponde à orientação desejada do robô. A direção real de navegação, entretanto, é a soma da orientação do vetor de velocidade e da orientação atual do robô Φ_{robot}, conforme a equação 18.

1.Navegação na direção de uma posição alvo, $f_{alvo}(\phi_{nav})$

A dinâmica que orienta a navegação na direção de um alvo é representada pela equação 20, de igual forma à Navegação Holonómica, em que a orientação do veículo é diferente da direção do alvo, porque trata-se da orientação de docking.

2.Evitar colisões com obstáculos, $f_{obs}(\phi_{nav})$

A dinâmica do comportamento para evitar obstáculos é governada pela equação 22, resultando na equação 23. Sendo que, a integração dos comportamentos de direcionar o robô para o alvo e evitar de obstáculos é descrita pela dinâmica da equação 23. Nesta equação, a dinâmica $f_{obs,i}(\phi_{nav})$ possui um peso maior que $f_{alvo}(\phi_{nav})$ garantindo que a força de repulsão seja superior à taxa de convergência para o alvo.

4.5.2. Controlo da Orientação nas Manobras de Acostagem

O controlo da orientação desempenha um papel fundamental nas manobras de acostagem, permitindo que o veículo ajuste a direção de forma precisa para se alinhar corretamente com o ponto de acostagem desejado. Este mecanismo envolve a modificação do ângulo de orientação do veículo, garantindo que ele siga a trajetória correta durante o processo de aproximação e posicionamento final, ou seja, segue a mesma estratégia apresentada na Navegação Holonómica.

1. Alinhamento com o Ponto de Acostagem

O veículo ajusta a sua orientação para se direcionar de forma precisa relativamente à orientação desejada de acostagem. Considerando que as posições do ponto de acostagem e do veículo são conhecidas, é definido um campo de forças que orienta o veículo na direção correta. Esse campo é gerado com base na diferença entre a orientação atual do veículo e a direção do ponto de acostagem, ajustada por uma

constante de relaxamento que determina a rapidez desse realinhamento. A força de atração que guia o veículo até o ponto desejado pode ser descrita como:

$$f_{docking_w}(\phi_{robot}) = -\lambda_{docking_w} \sin(\phi_{robot} - \psi_{docking}) \tag{32}$$

O controlo da orientação durante a manobra de acostagem é o resultado da combinação das forças de atração para o ponto de acostagem. A orientação final do veículo é governada pela variação de sua velocidade angular w_{robot},que é diretamente proporcional à soma das forças que influenciam a sua orientação.

$$w_{robot} = \frac{d\phi_{robot}}{dt} = f_{docking_w}(\phi_{robot}) \tag{33}$$

Deve haver uma hierarquia que assegure que o ajuste do ângulo de movimento tenha maior peso que o controlo da orientação angular do veículo.

4.5.3. Controlo da Velocidade Linear das Manobras de Acostagem

A operação em ambientes dinâmicos requer maior estabilidade e adaptabilidade na navegação. O módulo de controlo de velocidade das manobras de acostagem segue a dinâmica:

$$\frac{dv_{linear}}{dt} = c_{obs} \times (v_{linear} - v_{obs}) \tag{33}$$

Não obstante, neste comportamento, v_{obs}, reduz em função da distância da distância ao obstáculo mais próximo, min_{value}.

$$v_{obs} = min_{value} \times vel_{max,obs} \tag{34}$$

Este cálculo, utiliza o parâmetro $vel_{max,obs}$, que delimita a velocidade máxima neste comportamento. Ainda, impôs-se condições de modo a que vá reduzindo a velocidade linear à medida que a distância é inferior e, se tiver demasiado próximo de um obstáculo, v_{obs} deve ser zero.

Por fim, a velocidade linear resultante é dividida nas componentes cartesianas, v_x e v_y, formando o vetor de velocidade que define o movimento do veículo.

4.6. Cinemática Inversa do Veículo

O estudo da cinemática é fundamental para entender o movimento dos robôs móveis, uma vez que permite modelar o comportamento do veículo sem considerar as forças envolvidas. Correia (2023),

analisou diferentes tipos de robôs móveis, incluindo triciclos, diferenciais e omnidirecionais, cada um com as suas características e variáveis cinemáticas.

No contexto deste projeto, o modelo do veículo considerado é uma plataforma de 4 rodas omnidirecionais, tal como no apresentado por Correia (2023). Neste caso, cada motor é apresentado por $q = [w_1, w_2, w_3, w_4]^T$, enquanto $p = \left[V_x, V_y,\ W\right]^T$ corresponde às variáveis no espaço.

A ideia central da cinemática inversa é que, dado um conjunto de variáveis desejadas no espaço cartesiano, como:

- V_x – Velocidade linear na direção x,
- V_y – Velocidade linear na direção y,
- W – Velocidade angular,

o sistema deve determinar as velocidades individuais de cada roda w_1, w_2, w_3, w_4que fazem o robô realizar o movimento no espaço.

No caso do robô omnidirecional, que pode mover-se em qualquer direção no plano horizontal sem precisar girar, cada roda desempenha um papel essencial no movimento combinado. Para calcular as velocidades das rodas a partir das velocidades no espaço cartesiano, utilizou-se a cinemática inversa:

- **Entrada**: pretende-se que o robô se mova com determinada velocidade V_x e V_y no plano xy, e também que ele rode com velocidade angular W.
- **Saída**: calculam-se as velocidades w_1, w_2, w_3, w_4 de cada roda do robô para garantir que atenda aos valores desejados de V_x , V_y e W.

A relação entre as velocidades das rodas e o movimento global do robô é descrita por equações que dependem da configuração mecânica do robô (disposição e o ângulo das rodas) e são modeladas com base no sistema dinâmico do robô. Desta forma, é permitido controlar não apenas a translação em qualquer direção, mas também a rotação simultânea.

Para descrever o controlo das quatro rodas omnidirecionais da plataforma, utilizou-se o sistema de equações descrito por Correia (2023), que relacionou as velocidades lineares e angulares da plataforma

com as velocidades angulares de cada uma das rodas. Este sistema de equações é expresso seguindo a equação 35.

$$\begin{bmatrix} w1 \\ w2 \\ w3 \\ w4 \end{bmatrix} = \frac{1}{raio_{roda}} \times \begin{bmatrix} 1 & -1 & -(l_x + l_y) \\ 1 & 1 & (l_x + l_y) \\ 1 & 1 & -(l_x + l_y) \\ 1 & -1 & (l_x + l_y) \end{bmatrix} \times \begin{bmatrix} Vx_{robot} \\ Vy_{robot} \\ W_{robot} \end{bmatrix} \tag{35}$$

- $\begin{bmatrix} w1 \\ w2 \\ w3 \\ w4 \end{bmatrix}$ são as velocidades angulares das quatro rodas.
- $raio_{roda}$ é o raio de cada roda.
- $l_x \; e \; l_y$ são as distâncias do centro da plataforma até as rodas, nos eixos XX e YY, respetivamente.
- $\begin{bmatrix} Vx_{robot} \\ Vy_{robot} \\ W_{robot} \end{bmatrix}$ são as componentes da velocidade linear da plataforma nos eixos X e Y e a velocidade angular da plataforma em torno do seu eixo vertical (z).

Este sistema de equações permite calcular as velocidades necessárias para cada roda com base no movimento desejado da plataforma omnidirecional. A demonstração deste sistema encontra-se em Correia (2023), onde são detalhados os passos para a obtenção da equação 35, bem como as características específicas do robô.

5. ARQUITETURA DO SISTEMA

Neste capítulo é abordada a arquitetura do sistema. Esta arquitetura possui três módulos principais, o *Service Manager*, o *Environment Perception* e o *Movement Controller*.

Primeiramente, no que diz respeito ao *Service Manager*, este recebe um serviço de um gestor de frotas, dividindo o serviço a ser realizado pelo sistema em um conjunto de tarefas, solicitando ao módulo do controlo de movimentos a sua execução.

Como são solicitadas tarefas a efetuar, é necessário ter um gestor de tarefas dentro do módulo do controlo de movimentos. Para isso foi criado o *Task Manager*, que é responsável por gerir os sub módulos associados à movimentação da plataforma, conforme as necessidades da tarefa e do sistema.

De modo a ser possível uma navegação segura da plataforma, é necessário o controlo de movimentos receber informação do espaço circundante, nomeadamente da informação relativa a obstáculos. Não obstante, é necessária a informação do posicionamento no cenário.

De modo a tornar mais simples a interligação dos módulos, na Figura 39, está presente um diagrama com a arquitetura do sistema.

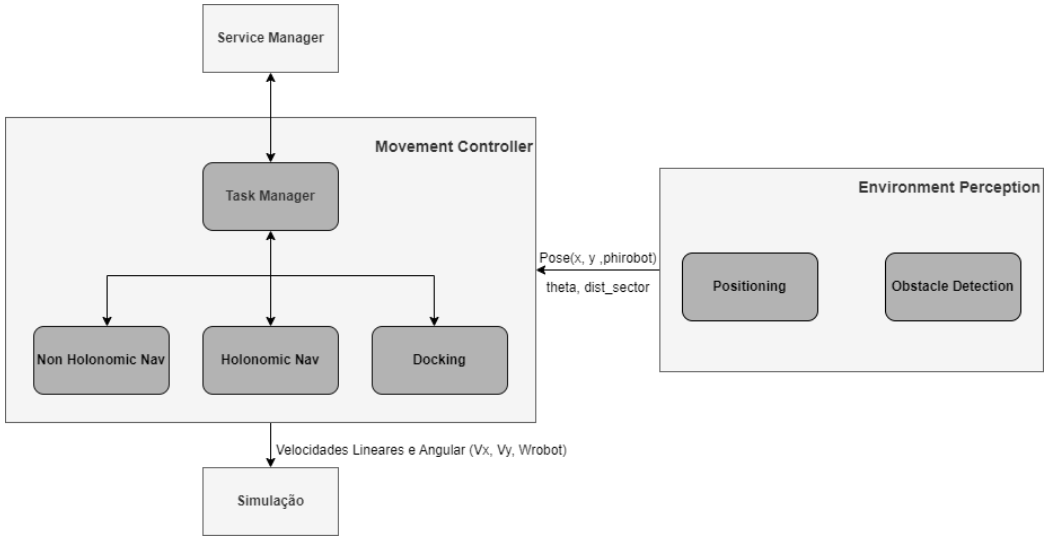


Figura 39: Arquitetura do sistema, composta pelos módulos: Service Manager, Environment Perception e Movement Controller.

5.1. *SERVICE MANAGER*

Este módulo, recebe comunicação de um gestor de frotas com informação de um serviço completo a comunicar ao sistema robótico. Cada tarefa é comunicada ao robô na forma de uma *action*, sendo cada tarefa/*task* individual.

A definição das tarefas encontra-se em um *action goal*, que caracteriza o tipo de tarefa a executar (*load*, *unload*, *dock*, *charge ou pick*). Para comunicar a informação pertinente, recorreu-se ao uso de *actions*, e criou-se *Task.action*, para o *Service Manager* comunicar com o *Movement Controller* (*Task Manager*). Esta comunicação permite obter feedback de cada tarefa à medida que a vai executando e, no fim da tarefa permite obter o estado final da mesma.

O conteúdo da mensagem da comunicação bidirecional entre o *Service Manager* e o *Task Manager* está presente na Figura 40 e na Figura 41.

<Task.action>

```
# Goal
string ACTION_TYPE_LOAD="load"
string ACTION_TYPE_UNLOAD="unload"
string ACTION_TYPE_CHARGE="charge"
string ACTION_TYPE_PICK="pick"
string ACTION_TYPE_PICK="dock"

uint32 robot_id                    # identificação do robô
string action_goal                 # load, unload, charge, dock e pick -> tipo de tarefa
GraphNode[] route                  # rota desejada para alcançar o action_goal
geometry_msgs/Point target         # coordenadas finais do final_target
float32 action_orientation         # em radianos orientacao final da rampa
Task[] tasks_possible              # Lista de tarefas possíveis
---
# Result
uint32 RESULT_SUCCESS=0
uint32 RESULT_FAILURE=1
uint32 RESULT_CANCELED=2

uint32 result_value            # valores de resultados possíveis: SUCCESS; FAILURE; or CANCELED.
string result_description      # descrição do resultado
---
# Feedback
GraphNode[] route_performed    # rota percorrida
```

Figura 40: Conteúdo do tipo de mensagem que é estabelecida entre o Service Manager e o Movement Controller (task Manager).

/miar_msgs::msg::GraphNode

```
std_msgs/Header header
geometry_msgs/Point point     # coordenadas graph node
float32 reaching_distance     # distancia para trocar de via point
float32 maxspeed              # velocidade maxima permitida
float32 vehicle_orientation   # orientação para o veículo
uint32 narrow_area            # narrow areas = 1, zonasapertadas
```

Figura 41: Estrutura e descrição da mensagem *miar_msgs::msg::GraphNode.*

5.2. *ENVIRONMENT PERCEPTION*

O módulo de *Environment Perception* é responsável por recolher, tratar e processar informações sensoriais referentes ao ambiente que rodeia o veículo, bem como determinar a sua localização no ambiente de navegação. Para isso, o módulo inclui componentes dedicados à deteção de obstáculos e ao posicionamento espacial. Este módulo apenas comunica via *topic*s em sentido único, isto é, partilha os dados com o controlo de movimentos.

O mesmo encontra-se dividido, pois publica tanto mensagens relacionadas ao posicionamento quanto informações sobre os setores e distâncias que são detetadas.

5.2.1. *POSITIONING*

Este *package* é responsável por calcular a posição e orientação da plataforma no ambiente circundante. Esta componente recebe os dados da localização provenientes do ambiente de simulação computacional, sendo fundamentais para a geração de movimentos. O ambiente envia a posição em quaternião, pelo *topic /pose_vehicle_coppelia*. Como esse *topic* envia em quaternião, a posição é guardada diretamente, mas para retirar o valor da orientação converte-se para ângulo de Euler, sendo que a orientação do veículo é dada pelo valor de *yaw*. Após conversão, a *pose* é publicada no *topic /vehicle_pose_real*, para todos os packages do *Movement Controller* terem acesso. Ambos os *topic*s utilizam a variável *geometry_msgs::msg::Pose2D*, que é a mensagem padrão do *ROS2* para representar a *pose*(posição e orientação) num plano 2D.

5.2.2. *OBSTACLE DETECTION*

Para a geração de movimentos, além de ser necessária a informação da localização do veículo, é relevante ter a informação das obstruções que rodeiam a plataforma. Para tal, elaborou-se o *package Obstacle Detection*, para recolha e processamento dos dados provenientes dos sensores *lidar,* que a plataforma possui. Como os sensores se encontram posicionados em extremos opostos do robô, foi necessário reajustar os dados para o centro do mesmo. Após conversão dos dados, organizaram-se os dados em setores de interesse para a navegação, e as distâncias mínimas de segurança perante os obstáculos. Este *package* também comunica com os módulos do *Movement Controller*, à exceção do *Task Manager.*

5.3. *MOVEMENT CONTROLLER*

Este módulo é responsável por definir os tipos de navegação do veículo. Estas navegações são repartidas em três tipos - *Navigation Non-Holonomic*, *Navigation Holonomic*, e *Docking* – sendo que todos estes *packages* comunicam bidireccionalmente com o *Task Manager*, de modo a haver coordenação e gestão do tipo de navegação a utilizar. O módulo deve estar capacitado de navegar de marcha-atrás devido ao formato do cenário do projeto. O *Task Manager* recebe tarefas do *Service Manager* e simplifica-as em operações básicas, que os componentes de geração de movimento consigam executar. Conforme a *task*, ele ativa as componentes necessárias do *Movement Controller*.

5.3.1. *TASK MANAGER*

Neste *package* foi implementado um algoritmo para gestão de tarefas pedidas e monitorização da execução das mesmas. À medida que as *tasks* vão sendo executadas, o *Service Manager* envia mais tarefas até não ter mais nenhuma a realizar. Ao receber as tarefas, o *Task Manager* gere qual comportamento deve ser utilizado e, vai monitorizar como está a decorrer a tarefa, sendo que no final devolve o estado da tarefa ao *Service Manager*.

A seleção do comportamento depende do tipo de tarefa recebida. Com base no *goal* recebido, o algoritmo escolhe entre os *packages* do *Movement Controller,* qual o mais indicado a ser utilizado ao longo da tarefa. Quando concluído com sucesso, o *Task Manager* seleciona o próximo comportamento com base na rota fornecida.

Caso o *narrow_area* se encontre ativo, o veículo deve deslocar-se de forma holonómica ou a movimentar-se para as manobras de acostagem. No cenário contrário, deve seguir um comportamento diferencial, de modo a ser mais percetível a sua intenção de movimento para os operadores.

5.3.2. *NON-HOLONOMIC NAVIGATION*

O *package Non-Holonomic Navigation* é responsável pela implementação do comportamento de navegação diferencial entre pontos, previamente definidos. Este comportamento foi implementado de modo a garantir segurança, tanto para o operador como para o veículo. Neste modo, a plataforma executa movimentos de fácil compreensão, facilitando a interpretação da intenção de movimento do veículo para os trabalhadores. Não obstante, este *package* comunica bidireccionalmente com o *Task Manager*, via *Actions*.

5.3.3. HOLONOMIC NAVIGATION

No que concerne a este *package*, o mesmo baseia-se no comportamento gerado no *Non-Holonomic Navigation,* mas distingue-se por efetuar movimentos holonómicos. Esta caraterística é relevante principalmente quando se navega em espaços mais exíguos, que os tornam mais exigentes. Contudo, não é um movimento tão legível para o trabalhador entender a intenção de navegação. À semelhança do *Non-Holonomic*, o *package*, comunica bidireccionalmente com o *Task Manager* via *Actions*.

5.3.4. DOCKING

A componente *Docking,* implementa um algoritmo idêntico ao *Holonomic Navigation*, diferindo apenas nas restrições de aproximação aos obstáculos, que são menores nesta componente. Esta navegação permite realizar manobras de acostagem a locais previamente estipulados para o efeito, e, comunica bidireccionalmente com o *Task Manager* via *Actions*.

6. IMPLEMENTAÇÃO

Este capítulo descreve a composição do cenário de simulação, e a utilização do simulador como recurso a verificação e validação da implementação. Também é descrita a implementação e os módulos que fazem parte da arquitetura do sistema, assim como a forma como estão interligados entre si.

6.1. SIMULAÇÃO

Para demonstrar o funcionamento do projeto, foi desenvolvida uma solução capaz de realizar a navegação e, posteriormente, executar as tarefas desejadas, nomeadamente o transporte de caixas de material ou caixas vazias. Para esta demonstração, foi necessário elaborar um cenário de simulação, a fim de similar o ambiente industrial pretendido. Este cenário é composto por rampas de abastecimento de caixas e um supermercado, que inclui guias: uma guia com *dollies* que transportam caixas com material e outras guias para canais de *dollies*, destinadas a caixas vazias..

O esboço do cenário da linha do ambiente industrial está apresentado na Figura 42.

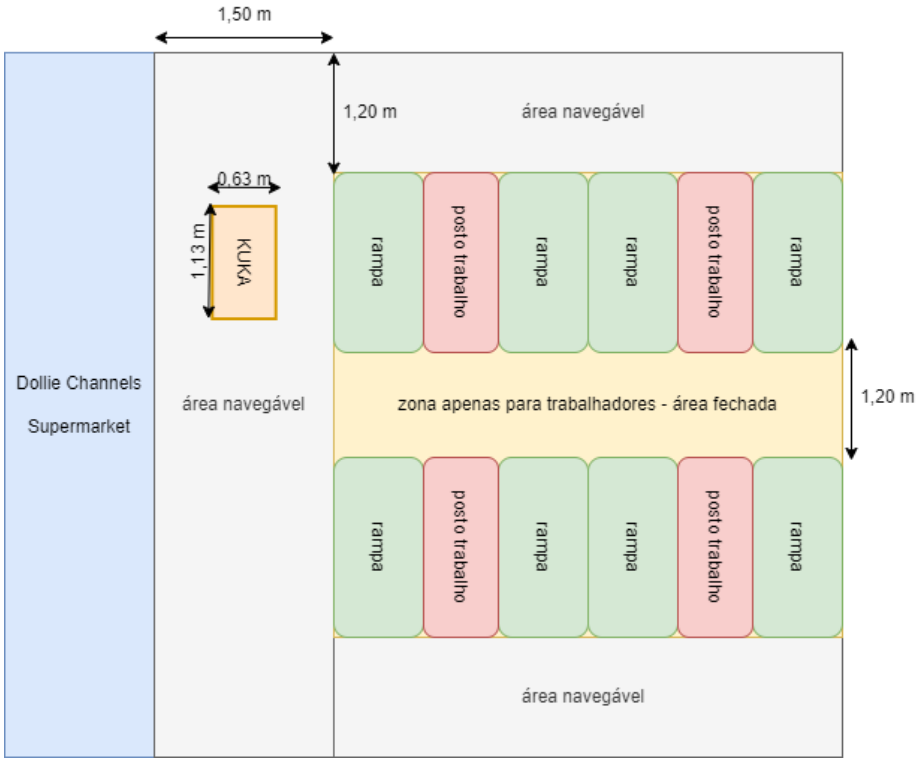


Figura 42: Cenário da linha do ambiente industrial.

Perante o cenário industrial apresentado para efetuar as tarefas de transporte de caixas, elaborou-se o cenário de simulação como ilustrado na Figura 43, de modo a demonstrar o conceito de navegação.

Figura 43: Cenário elaborado no *CoppeliaSim* para demonstrar o funcionamento do projeto.

O cenário é composto por um supermercado de caixas cheias e vazias (localizado no corredor principal), rampas de abastecimento e postos de trabalho nos corredores horizontais. Este cenário foi elaborado tendo por base as características do projeto, nomeadamente as dimensões pretendidas para o mesmo. As zonas de *docking* são nas rampas de abastecimentos e nos *dollies channel* (supermercado). Neste cenário, a direção de entrada do veículo num corredor deve ser determinada pela localização das rampas de abastecimento, uma vez que o braço manipulador irá estar colocado no lado direito da plataforma móvel. Portanto, para que o processo de carga e descarga seja eficiente, as rampas precisam de estar localizadas no lado direito (nos corredores horizontais) face à direção de deslocamento do veículo. Assim, o veículo entra sempre nesses corredores, de modo a que o braço fique alinhado com as rampas, facilitando o acesso às caixas nas rampas de acesso.

O objetivo da criação deste cenário é permitir que o veículo se aproxime da área do supermercado, com a finalidade de pegar caixas ou deixar lá as caixas vazias, além de transportar as mesmas para e a partir das rampas de acesso sinalizadas no cenário. O *docking* será efetuado perto da zona do *supermarket* e perto das rampas de acesso. No corredor principal o veículo navega de forma diferencial, nos corredores horizontais navega de forma holonómica e realiza as manobras de acostagem, exemplificando todos os módulos de navegação. Salienta-se ainda que, o robô está habilitado a navegar em marcha-atrás, devido à estrutura do cenário e das suas dimensões.

Para possibilitar a comunicação entre o cenário e os módulos desenvolvidos, foi elaborada uma interface *ROS*. Atendendo aos requisitos, o simulador precisa de publicar a informação da *pose* do veículo, subscrever a velocidade linear e angular e publicar a informação dos sensores *lidar* da frente e de trás

da plataforma utilizando a biblioteca *tf2* que padroniza os referenciais de coordenadas, possibilitando, a transformação de dados entre diferentes referenciais. Para complementar a informação que transita entre o simulador e os módulos, apenas foi necessário criar um publicador para as velocidades linear e angular. Todas estas comunicações são efetuadas via *ROS topic*.

6.2. POSICIONAMENTO

A localização do veículo foi obtida através da comunicação do simulador com o módulo de *Positioning*. Esta comunicação realizou-se através da subscrição do *topic /pose_vehicle_coppelia* que contém a informação da *pose* em formato de quaternião (*x, y, z* e *w*). Perante isso, foi necessário transformar a orientação do veículo de quaternião para ângulos de Euler, uma vez que o *yaw* (Ângulo de Euler) corresponde à orientação do robô (Φ) em duas dimensões. É possível visualizar o fluxo da aquisição da *pose* através da Figura 44.

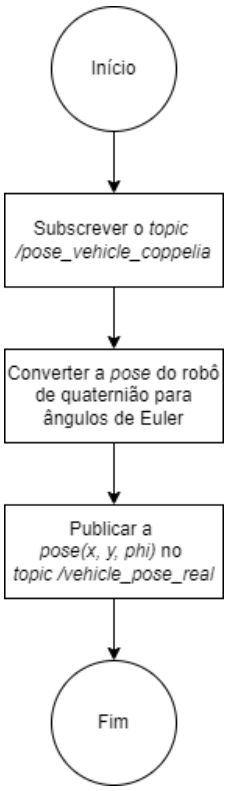


Figura 44: Fluxo da aquisição da *pose* do veículo.

A publicação da *pose* é realizada através do *topic /geometry_msgs::msg::Pose2D* que é estruturado conforme apresentado na Figura 45.

/geometry_msgs::msg::Pose2D
float64 x float64 y float64 theta

Figura 45: Definição da mensagem */geometry_msgs::msg::Pose2D*.

6.3. Deteção de Obstáculos

Para a recolha de obstáculos são utilizados dois sensores *lidar SICK S300 Expert*, que fazem parte da constituição da plataforma móvel. Estes sensores, tem como objetivo capacitar a plataforma em detetar a presença de obstáculo em volta da mesma.

Os dados sensoriais dos sensores são recebidos por meio de *topic*s ROS, sendo um *topic* para os dados do sensor da frente (*/lidar_front_data*) e outro para os dados do sensor traseiro (*/lidar_rear_data*). Ambos utilizam mensagens do tipo */sensor_msgs::msg::LaserScan*, que é uma estrutura de mensagem que contém informações detalhadas do espaço circundante capturado pelos sensores. A definição da estrutura está presente na Figura 46.

/sensor_msgs::msg::LaserScan
std_msgs/Header header float32 angle_min float32 angle_max float32 angle_increment float32 time_increment float32 scan_time float32 range_min float32 range_max float32[] ranges float32[] intensities

Figura 46: Estrutura da mensagem */sensor_msgs::msg::LaserScan.*

Utilizando ROS, o algoritmo pode estar a processar a informação independentemente da receção de dados dos sensores. Assim que uma nova mensagem é recebida, o sistema começa a funcionar e a processar os dados:

1. **Determina os setores de interesse:** foca na divisão do campo de visão dos sensores em setores que são relevantes para a navegação do robô.

2. **Calcula as distâncias lineares de segurança:** consiste na determinação da distância mínima entre o veículo e obstáculos detetados, a fim de evitar colisões.

Todos os cálculos são realizados considerando o centro de massa do veículo como referência. Esta conversão é fundamental para que a posição dos obstáculos e as distâncias sejam interpretadas corretamente na navegação da plataforma móvel. E, por fim, após o processamento, as informações são enviadas aos módulos responsáveis por gerar comportamentos de navegação do veículo, também via ROS.

O fluxo de execução do algoritmo é apresentado na máquina de estados, referenciada na Figura 47.

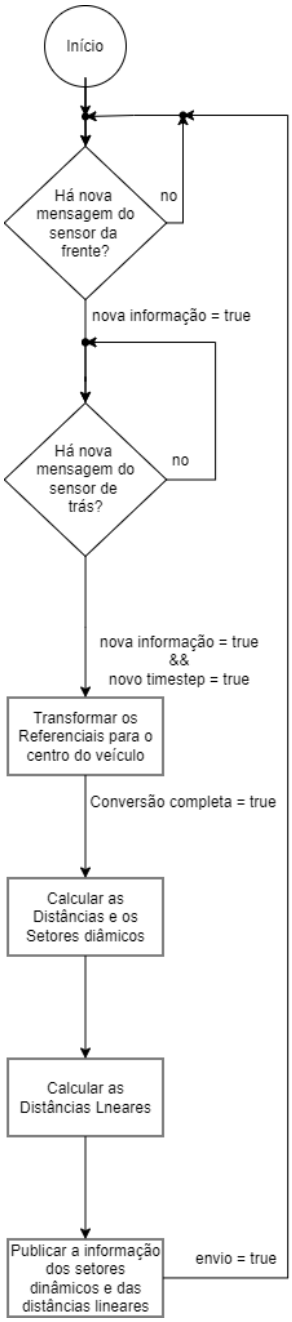


Figura 47: Máquina de estados que sustenta o módulo *Obstacle Detection*.

A cada novo *timestep*, no *topic /sector_dist_obs*, são publicadas as distâncias respetivas a cada setor de interesse à navegação. As mensagens do *topic* são do tipo */miar_msgs::msg::DistanceSector*, sendo as principais variáveis a orientação de cada setor (*theta_obs*), e as distâncias absolutas correspondentes a cada orientação (*distances_obs*). As mensagens publicadas nesse *topic* seguem a estrutura descrita na Figura 48.

/miar_msgs::msg::DistanceSector	
std_msgs/Header header	# Identificação da msg
int num_sectores	# Número de setores considerados
float64 range_field	# Ângulo de Varrimento
float64 theta_obs	# Orientação de cada setor
float64 distances_sector	# Distância absoluta na respetiva orientação

Figura 48: Estrutura e descrição da mensagem */miar_msgs::msg::DistanceSector*, que é a que comunica entre o *Obstacle Detection* e as componentes de navegação.

No que diz respeito às distâncias lineares do espaço envolvente, estas também são publicadas a cada *timestep*. Estas são publicadas no *topic /protect_dist_obs* utilizando mensagens do tipo */miar_msgs::msg::ProtectDistance*. Estas mensagens contêm informações detalhadas sobre as distâncias lineares e são utilizadas por todos os módulos do controlador de movimento (*Movement Controller*) para definir e ajustar o comportamento de movimento do veículo, isto é, mais precisamente, se está numa zona de paragem de emergência ou não. A Figura 49 detalha a estrutura dessas mensagens.

/miar_msgs::msg::ProtectDistance	
std_msgs/Header header	# Identificação da msg
float64 front_dist_obs	# Distancias detetadas parte frontal
float64 rear_dist_obs	# Distancias detetadas parte traseira
float64 right_dist_obs	# Distancias dos setores laterais direitos
float64 left_dist_obs	# Distancias dos setores laterais esquerdos
float64 min_lateral_pos	# Primeiro feixe lateral a ser detetado
float64 min_fr_rear_pos	# Primeiro feixe frentre/tras a ser detetado
float64 incr_points_lat	# Incremento entre feixes laterais
float64 incr_points_fr_rear	# Incremento entre feixes frontais/traseiros

Figura 49: Estrutura e descrição da mensagem */miar_msgs::msg::ProtectDistance*, que comunica com os módulos do controlo de movimentos e, contém as distâncias lineares entre o veículo e os obstáculos.

Todos os módulos relacionados com a movimentação do veículo subscrevem o *topic /protect_dist_obs* e */sector_dist_obs* para receberem as distâncias lineares e ajustar o movimento em função dessa informação. Em situações de navegação em ambientes dinâmicos ou congestionados, os módulos utilizam as distâncias lineares para ativar mecanismos de paragem de segurança.

Posto isto, a informação que se obtém diretamente dos sensores, precisa de ser processada de modo que se obtenha num formato mais percetível e adequado. Para converter esses dados utilizou-se o método presente na Figura 50.

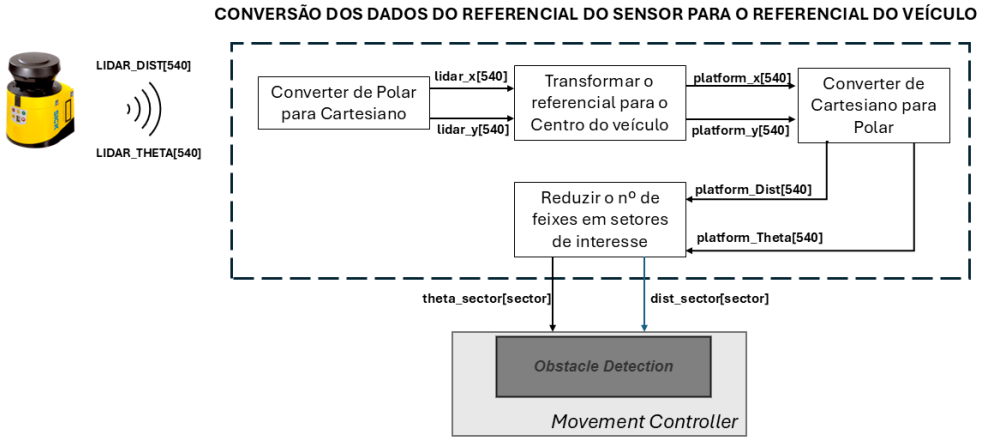


Figura 50: Método de conversão dos dados do referencial do sensor para o referencial do veículo e de interesse à navegação.

Primeiramente, o referencial precisa de ser transformado do sensor para o centro de massa do veículo. Para tal, como a informação proveniente do *lidar* são coordenadas polares, converteu-se essa informação polar para cartesiana.

Cada *lidar* inclui 540 feixes, com o intuito de otimizar a navegação e garantir estabilidade do algoritmo, foi necessário reduzir o número de setores analisados, simplificando o processamento.

Primeiramente, definiu-se os setores de interesse, que neste caso, seriam os 360°, dividindo essa janela de varredura em 36 setores, resultando num espaçamento de 10 graus entre setores. Salienta-se que não vão estar os 36 setores ativos em simultâneo. O objetivo é cumprir uma varredura de 210°, criando uma variação dos setores entre -105 a +105 graus, sendo 0° a referência na direção frontal do veículo. Os setores de interesse são então selecionados de forma dinâmica, de modo a estarem 21 ativos a navegar para a frente e outros 21 setores ao navegar em marcha atrás. Os respetivos setores estão apresentados na Figura 51.

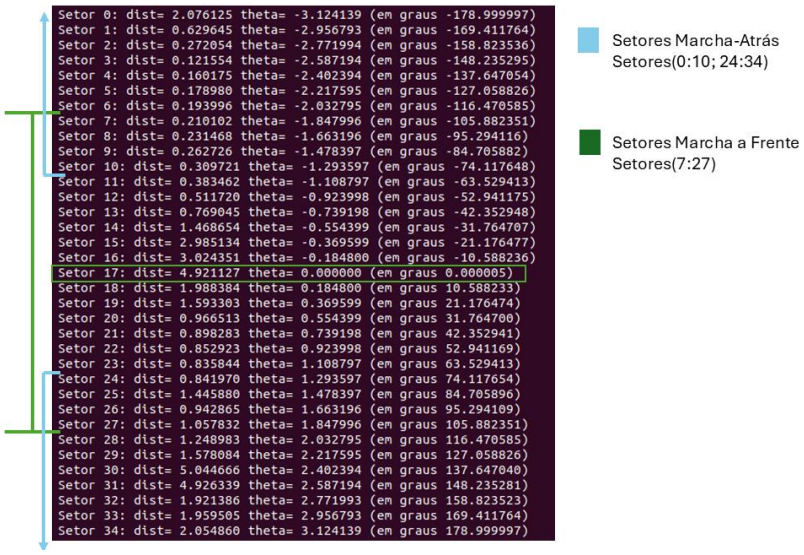


Figura 51: Setores de interesse consoante a marcha.

Posto isto, do setor sete até ao vinte e sete, são utilizados ao navegar para a frente. No que diz respeito à marcha atrás, os setores utilizados são entre o zero e o dez e, o vinte e quatro e o trinta e quatro.

O processo de conversão dos dados do sensor em setores dinâmicos é dividido em três fases:

1. Cálculo dos pontos cartesianos de cada setor

Considera-se a distância máxima detetável pelo sensor e os ângulos de cada setor. Os pontos cartesianos para um setor i são calculados por:

$$x_{max,setor,i} = d_{max} \times \cos(\theta_i) \tag{49}$$

$$y_{max,setor,i} = d_{max} \times sin(\theta_i) \tag{50}$$

No que diz respeito ao ângulo de cada setor, este é calculado pela seguinte expressão:

$$\theta_i = -\frac{\gamma_{varredura}}{2} + \frac{\gamma_{varredura}}{n-1} \tag{51}$$

Onde, $\gamma_{varredura}$ corresponde à janela de varredura e n é o número total de setores.

2. Cálculo do Segmento de Reta entre Dois Feixes Laser

Para cada par de feixes consecutivos, são calculados o declive e o coeficiente linear do segmento de reta que os conecta. O declive m e o coeficiente b são obtidos com:

$$m_i = \frac{y_{feixe,i} - y_{feixe,i-1}}{x_{feixe,i} - x_{feixe,i-1}} \tag{52}$$

$$b_i = y_{feixe,i} - m_i \times x_{feixe,i} \tag{53}$$

3. Interseção entre o segmento de reta dos feixes e do setor dinâmico:

O ponto de interseção é calculado entre o segmento de reta do setor e o segmento de reta entre dois feixes consecutivos, utilizando o seguinte sistema de equações:

$$x_{intersecao,i} = \frac{b_i}{m - m_i}, \qquad quando\ m - m_i \neq 0 \tag{54}$$

$$y_{intersecao,i} = m_i \times x_{intersecao,i} \tag{55}$$

Para evitar comportamentos irregulares durante a navegação, é necessário ter conta as dimensões internas do veículo. A plataforma móvel tem a forma geométrica de um retângulo com largura de 0,63 metros e comprimentos 1,13 metros. Utilizando a trigonometria, define-se um ângulo de separação, $\alpha = 29{,}14$ º. Pode-se ver a representação desse ângulo na Figura 52.

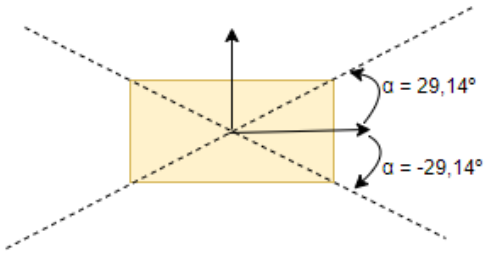


Figura 52: Representação dos limites entre secções: Regiões Frontal, Traseira, Lateral Direita e Lateral Esquerda.

Este ângulo de separação é obtido através da seguinte equação:

$$\alpha = tan^{-1}\left(\frac{\frac{largura}{2}}{\frac{comprimento}{2}}\right) \tag{56}$$

No caso de um setor estar na região frontal ou traseira do veículo a distância interna é calculada por:

$$dist_{intersecao,i} = \left|\frac{comprimento}{2} \times \cos(\theta_i)\right| \tag{57}$$

Caso o setor seja na região lateral, a distância interna é dada por:

$$dist_{intersecao,i} = \left|\frac{largura}{2} \times \sin(\theta_i)\right| \tag{58}$$

4. Cálculo das Distâncias Lineares em relação ao corpo do veículo

Durante a navegação é necessário manter uma distância mínima de segurança em relação aos obstáculos próximos. Para isso, foram divididos setores lineares nas quatro secções mencionadas anteriormente (frontal, traseira, lateral direita e lateral esquerda), com 31 setores alocados para as laterais e 15 para a zona frontal e traseira do veículo. As distâncias lineares são calculadas da seguinte forma:

- **Região Frontal/Traseira**

$$dist_{linear,i} = \left|x_{feixe,i}\right| - \frac{comprimento}{2} \tag{59}$$

- **Região Lateral**

$$dist_{linear,i} = \left|y_{feixe,i}\right| \tag{60}$$

Este estudo propõe uma abordagem para processar e simplificar as informações dos sensores, dividindo o ambiente em setores e realizando cálculos que permitem ao veículo navegar de forma autónoma e segura. O processamento é otimizado de forma a minimizar o tempo de cálculo sem sacrificar a precisão na deteção de obstáculos e na manutenção de distâncias seguras em ambientes dinâmicos.

6.4. TASK MANAGER

A componente *Task Manager* é responsável por gerir as tarefas e monitorizar a execução das mesmas. Este processo inicia quando o *Service Manager* envia uma ou mais tarefas para serem executadas. O *Task Manager* tem a função de receber as tarefas e geri-las de acordo com o seu tipo (*load, unload, dock, charge*), além de monitorizar a execução da tarefa.

Assim que seja recebida uma tarefa, é identificado o tipo para decidir qual comportamento é mais adequado para executar. Se o veículo estiver acoplado a uma rampa ou *dollie*, o *Task Manager* inicia a tarefa com um *undocking* (que seria uma função do módulo de docking). Em seguida, o sistema estabelece o primeiro ponto de navegação e inicia o movimento em direção a ele. Como comunica diretamente aos módulos de movimentação do veículo, o *Task Manager*, deve escolher qual das navegações é mais adequada para a situação, com base nas condições e no objetivo da tarefa. Uma condição considerada pelo *Task Manager* é se o veículo está a operar numa *narrow_area* (uma área apertada ou de espaço limitado). Quando o veículo se encontra numa dessas áreas, o sistema deve optar por um comportamento holonómico, que permita ao veículo se mover em todas as direções sem precisar rodar. Este comportamento é útil para realizar manobras precisas, como a aproximação a uma rampa para realizar operações de acostagem. Se o ambiente não for uma *narrow_area*, o veículo deve seguir um comportamento diferencial, que é mais previsível para os trabalhadores ao redor, pois o movimento ocorre de maneira linear, facilitando a perceção da sua direção.

Durante a execução da tarefa, o *Task Manager* acompanha continuamente o progresso, monitorizando se tudo está a decorrer conforme planeado. Esta monitorização é utilizada para garantir que a tarefa seja concluída com sucesso. Quando a execução da tarefa é concluída, o sistema verifica o resultado. Se a tarefa foi bem-sucedida, o *Task Manager* devolve o estado final da tarefa ao *Service Manager*, informando-o sobre a conclusão. Além disso, o *Task Manager*, se houver outra tarefa, seleciona o comportamento adequado para a próxima tarefa, de acordo com a rota previamente definida, repetindo o ciclo explicado. No entanto, se a tarefa não for concluída com sucesso, o sistema reporta a falha diretamente ao *Service Manager* para que ações sejam tomadas. Uma vez que o estado/resultado da tarefa é enviado de volta ao *Service Manager*, este verifica se ainda há mais tarefas a serem executadas. Se houver, o ciclo recomeça com o envio de novas tarefas para o *Task Manager*. Caso contrário, o processo é encerrado. Este fluxo de trabalho assegura que as tarefas sejam realizadas de forma contínua e eficiente, com o *Task Manager* a gerir não apenas a execução de cada tarefa, mas também adaptando o comportamento

do veículo às condições do ambiente e ao tipo de tarefa em questão, encontrando-se visível na Figura 53.

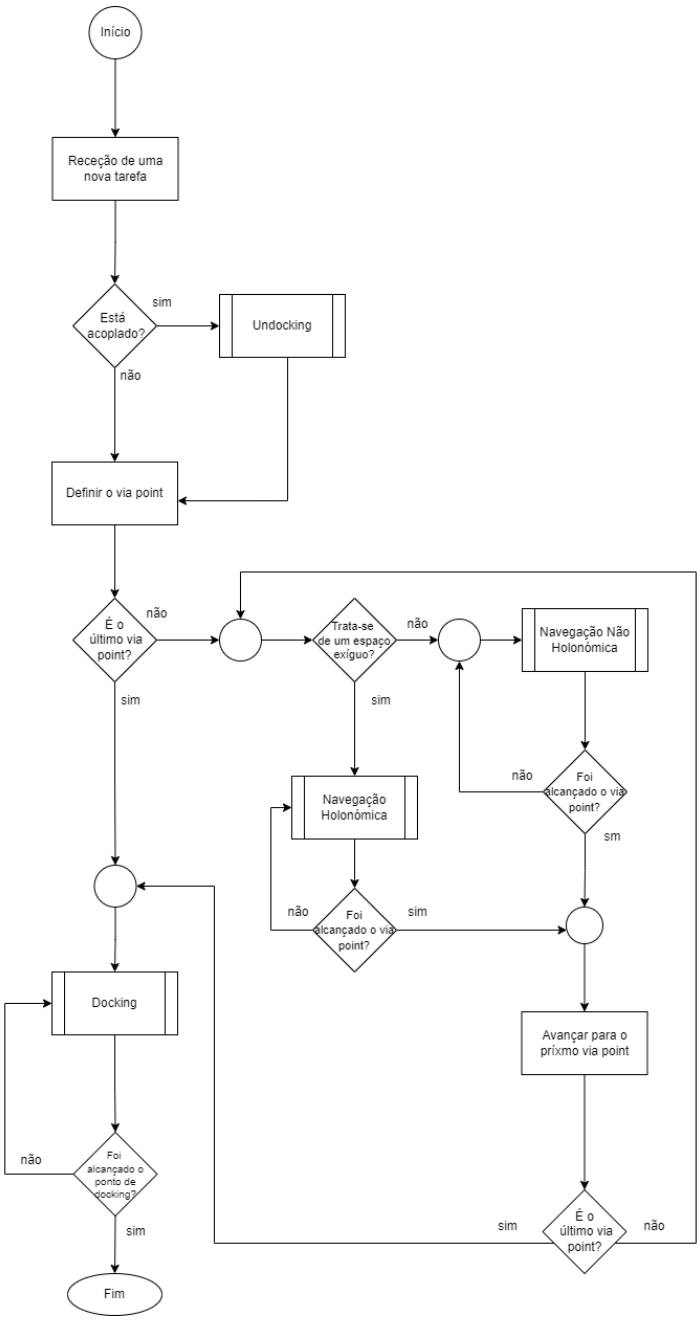


Figura 53: Fluxograma do algoritmo executado pelo *Task Manager.*

A comunicação entre o *Task Manager* e o *Service Manager* é estabelecida através da estrutura do tipo */miar_action::action::Task*, que especificam a rota e orientações finais desejadas, e está detalhada na Figura 54.

```
/miar_actions::action::Task
# Goal
string ACTION_TYPE_LOAD="load"
string ACTION_TYPE_UNLOAD="unload"
string ACTION_TYPE_PARK="dock"
string ACTION_TYPE_CHARGE="charge"

string action_goal                  # load, unload, parking, etc... tipo de tarefa
miar_msgs/GraphNode[] route         # rota desejada (sequencia de GraphNodes) para chegar ao destino final para realizar o action_goal
float32 action_orientation          # em radianos - orientacao final da rampa
---
# Result
uint32 RESULT_SUCCESS=0
uint32 RESULT_FAILURE=1
uint32 RESULT_CANCELED=2

uint32 result_value                 # possiveis valores de resultados: SUCCESS; FAILURE; or CANCELED.
string result_description           # descrição do resultado
---
# Feedback
miar_msgs/GraphNode[] route_performed        # rota percorrida (sequencia de GraphNodes)
```

Figura 54: Estrutura e descrição do tipo de mensagem que estabelece a comunicação entre o *Task Manager* e o *Service Manager*.

6.5. Navegação Não-Holonómica

Esta componente é responsável por controlar o movimento do veículo entre pontos definidos no espaço, considerando obstáculos. A navegação segue uma rota definida por uma série de pontos, que são recebidos no *Task Manager*, via ROS *Action Server*. O *Task* Manager após receber essa informação e de modo a comunicar bidireccionalmente com o *Non Holonomic Nav* criou-se a *action Movement*. Esta segue a estrutura apresentada na Figura 55.

```
/miar_actions::action::Movement
# Goal
miar_msgs/GraphNode[] route         # rota desejada (sequencia de GraphNodes)
float32 action_orientation          # em radianos a orientação final para a plataforma
---
# Result
uint32 SUCCESS=0
uint32 FAILURE=1
uint32 CANCELED=2

uint32 result_value                 # resultados possiveis: SUCCESS; FAILURE; or CANCELED.
string result_description           # descrição dos resultados
---
# Feedback
miar_msgs/GraphNode[] route_performed    # rota percorrida
float32 vrobot                           # feedback da velocidade longitudinal
float32 wrobot                           # feedback da velocidade angular
```

Figura 55: Estrutura e descrição da *action Movement* que comunica entre o *Task Manager* e o *Non Holonomic Nav*.

A estrutura da rota */miar_msgs::msg::GraphNode* é detalhada na Figura 56.

/miar_msgs::msg::GraphNode	
std_msgs/Header header	
geometry_msgs/Point point	# coordenadas graph node
float32 reaching_distance	# distancia para trocar de via point
float32 maxspeed	# velocidade maxima permitida
float32 vehicle_orientation	# orientação para o veículo
uint32 narrow_area	# narrow areas = 1, zonasapertadas

Figura 56: Estrutura e descrição da mensagem que contém a rota desejada, bem como as caraterísticas a seguir para a navegação.

O movimento do robô é controlado com base em *via points* recebidos e nas distâncias detetadas relativamente aos obstáculos. Embora a plataforma seja omnidirecional, neste módulo o sistema apresenta um comportamento similar aos veículos de navegação diferencial, isto é, utilizando dois graus de liberdade (velocidade longitudinal e a velocidade angular).

A navegação não-holonómica segue uma máquina de estados que permanece à espera até receção de um novo *goal*. Ao iniciar a navegação, a velocidade é calculada com base na posição do *via point*, no posicionamento do veículo e na informação sensorial. Caso as distâncias não sejam respeitadas, o veículo para e comunica ao *Task Manager*. Se as condições forem atendidas e o ponto for alcançado, verifica se é o último *via point*, caso seja o último termina a navegação, caso contrário repete o processo até ao próximo ponto. Não obstante, a execução da navegação pode ser cancelada a qualquer momento pelo *Task Manager*. É possível visualizar a máquina de estados na Figura 57.

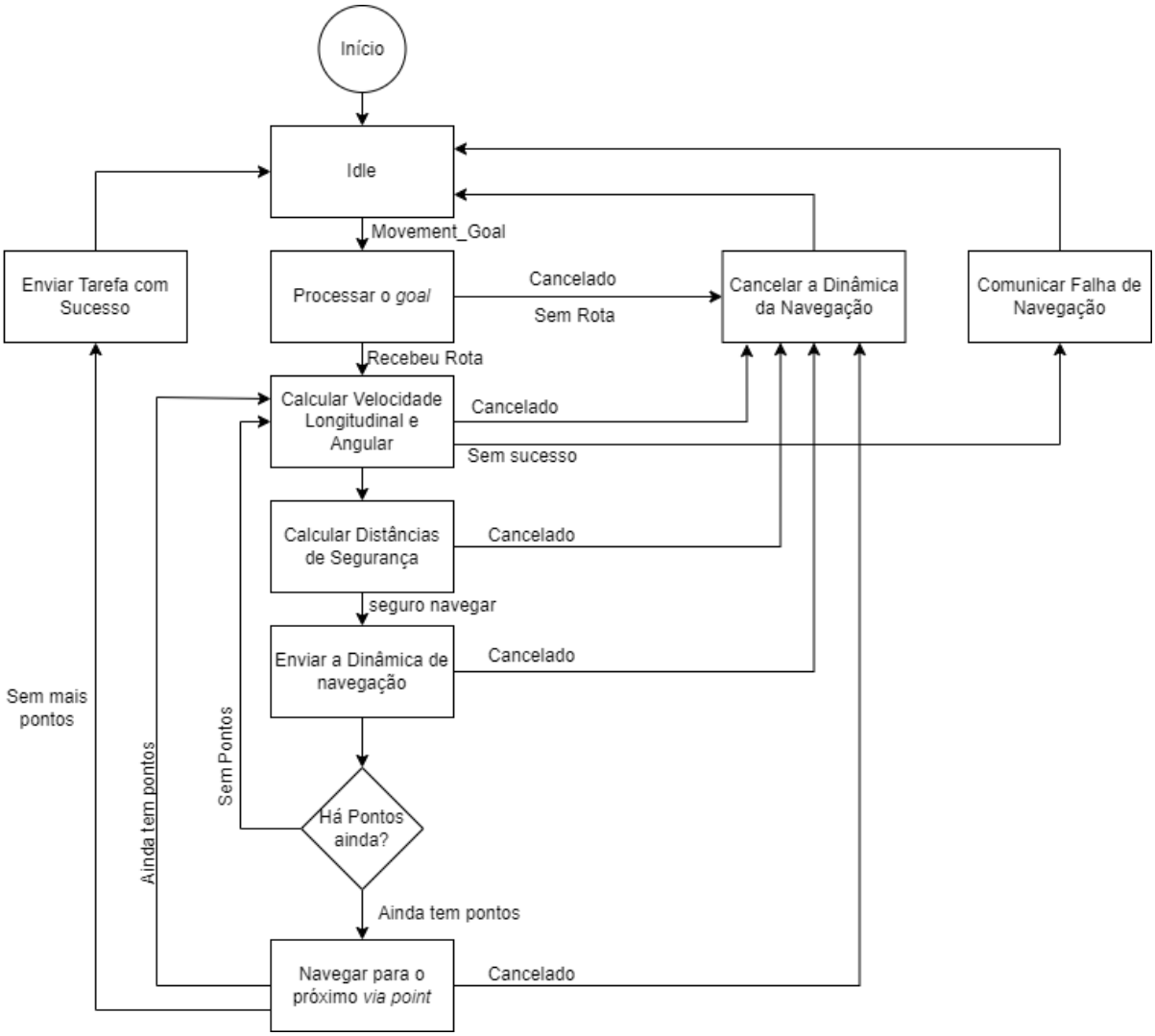


Figura 57: Máquina de Estados do comportamento de navegação não-holonómica.

6.6. NAVEGAÇÃO HOLONÓMICA

A navegação holonómica é um dos *packages* pertencentes ao *Movement Controller*, sendo o seu foco a navegação de forma omnidirecional, isto é, uma movimentação mais flexível, considerando três graus de liberdade (Velocidade Longitudinal, Velocidade Lateral e Velocidade Angular). Assim, o veículo está capacitado para se mover em qualquer direção e rodar em simultâneo, o que é fundamental em ambientes exíguos como é o caso deste projeto.

O objetivo desta componente é mover a plataforma de modo holonómico entre pontos, utilizando um vetor de *via points* recebidos via ROS *Action Server* do *Task Manager*. A comunicação é feita através da *action MovementOmni*. As mensagens que são comunicadas bidireccionalmente seguem a estrutura apresentada na Figura 58.

```
/miar_actions::action::MovementOmni
# Goal
miar_msgs/GraphNode[] route        # rota desejada (sequencia de GraphNodes) para cumprimir a action_goal
float32 action_orientation         # em radianos a orientação final da plataforma
---
# Result
uint32 SUCCESS=0
uint32 FAILURE=1
uint32 CANCELED=2

uint32 result_value                # valores dos resultados possíveis SUCCESS; FAILURE; or CANCELED.
string result_description          # result description
---
# Feedback
miar_msgs/GraphNode[] route_performed        # feedback da  rota percorrida
float32 vx_robot                   # feedback da velocidade longitudinal
float32 vy_robot                   # feedback da velocidade lateral
float32 wrobot                     # feedback da velocidade angular
```

Figura 58: Estrutura e descrição da *action* que comunica entre o *Task Manager* e o *Holonomic Nav.*

Destaca-se o conteúdo do */miar_msgs::msg::GraphNode[] route*, que nesta navegação o *narrow_area=1*, e o veículo irá navegar de forma holonómica com a orientação (que corresponde à velocidade angular da plataforma) que for recebida na variável *float32 vehicle_orientation* até esse *via point.* À medida que vai trocando de *via point* deve trocar também para a respetiva *vehicle_orientation*.

A navegação holonómica segue uma máquina de estados que organiza o comportamento do sistema, desde a receção de um novo *goal* até à conclusão da navegação. A máquina de estados permanece num ciclo de espera até receber um novo *goal* do *Task Manager*. Enquanto isso, o *package* está inativo e a aguardar instruções. Assim que o *goal* é recebido, a mensagem é processada e armazenada.

Em seguida, é iniciado o algoritmo de navegação holonómica, que ajusta as velocidades longitudinais, laterais e angulares conforme os parâmetros dinâmicos e a rota estabelecida. Para controlar a navegação holonómica, foram utilizados sistemas dinâmicos não lineares ajustados para capacidades holonómicas, ou seja, a cada *timestep*, são calculados os vetores de velocidade (v_x, v_y e w), com base na posição atual do veículo, posição e orientação do próximo *via point* e informação sensorial dos obstáculos em redor da plataforma.

Antes de enviar os comandos da movimentação do veículo, o sistema verifica se as distâncias de segurança relativas aos obstáculos são mantidas. Se algum obstáculo estiver muito próximo, o veículo permanece parado até que a condição de segurança seja restaurada. Caso contrário, os comandos são enviados para o veículo.

Quando o veículo atinge a proximidade de um *via point*, o algoritmo verifica se esse é o último ponto da rota. Se for, o processo termina com bem-sucedido e retorna ao estado em que fica à espera de um novo *goal*. Caso contrário, se não for o último ponto, o próximo *via point* é definido como alvo a seguir e o ciclo de navegação continua.

Não obstante, em qualquer estado, o processo de navegação pode ser interrompido pelo *Task Manager*. Quando isso ocorre, a máquina de estados retorna ao estado que aguarda por um novo *goal*.

O funcionamento da máquina de estados deste módulo está detalhado na Figura 59.

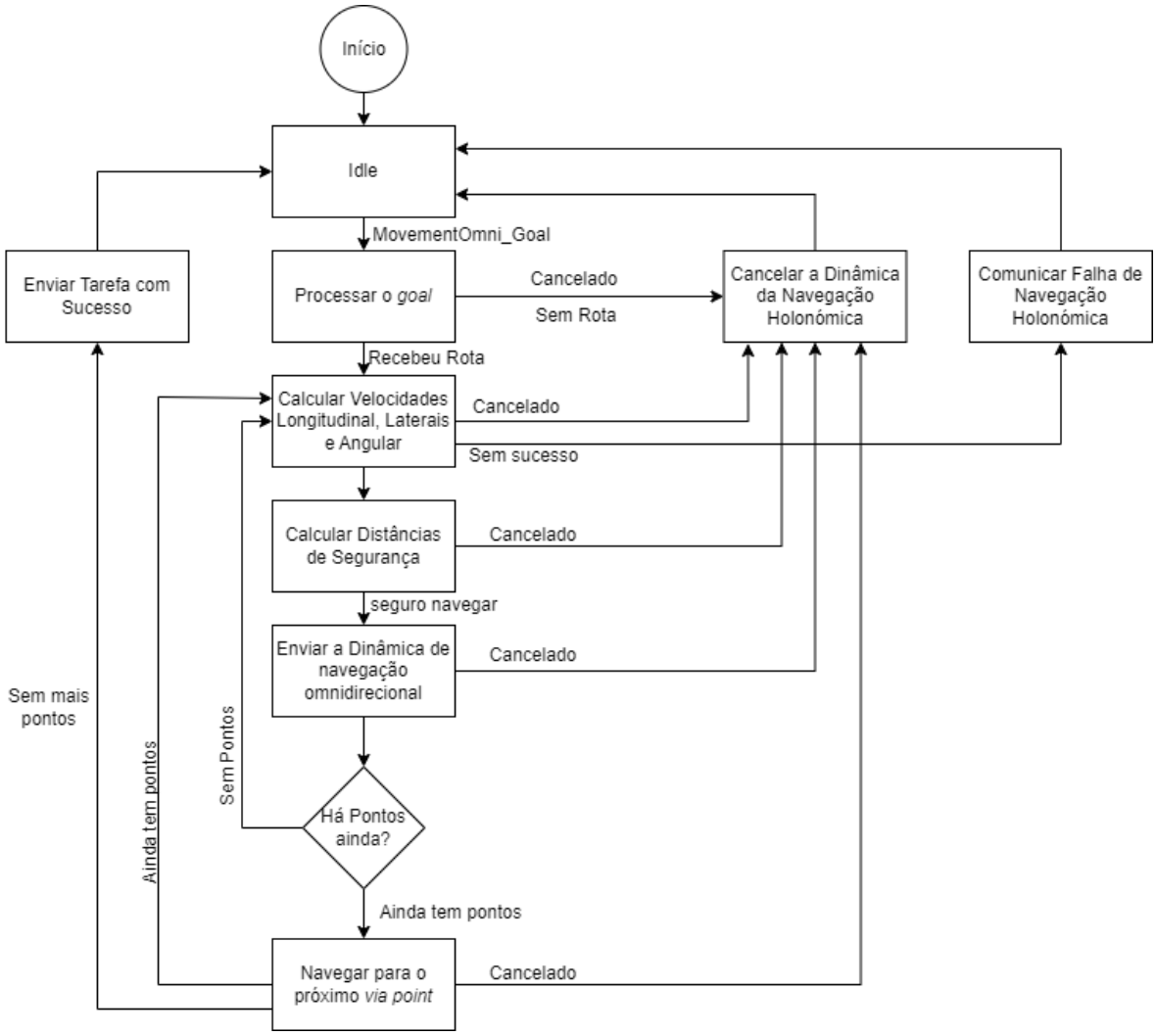


Figura 59: Máquina de estados que define o comportamento da *Holonomic Nav.*

6.7. MANOBRAS DE ACOSTAGEM

A componente de *docking* define o movimento da plataforma de aproximação às rampas de abastecimento ou aos *supermarkets*. O algoritmo funciona de forma idêntica à navegação holonómica,

requerendo um único *input* de ângulo de interesse, o *vehicle_orientation* (variável que provém da *miar_msgs::msg::GraphNode route*), de modo a obrigar a movimentar-se para acostar com a orientação pretendida.

A movimentação de *Docking* é um dos *packages* pertencentes ao *Movement Controller*, sendo uma navegação de forma omnidirecional, isto é, considerando três graus de liberdade (Velocidade Longitudinal, Velocidade Lateral e Velocidade Angular). Assim, o veículo está capacitado para se mover em qualquer direção e rodar em simultâneo, o que é fundamental em ambientes exíguos como é o caso deste projeto, o que é fundamental para as acostagens.

Esta componente recebe um *via point* via ROS *Action Server* do *Task Manager*. A comunicação é feita através da *action Docking*. As mensagens que são comunicadas bidireccionalmente e seguem a estrutura apresentada na Figura 60.

/miar_actions::action::Docking

```
# Goal
miar_msgs/GraphNode route          # ponto desejado para o action_goal
float32 action_orientation         # em radianos a orientação pretendida
---
# Result
uint32 SUCCESS=0
uint32 FAILURE=1
uint32 CANCELED=2

uint32 result_value                # valores possível para resultado: SUCCESS; FAILURE; or CANCELED.
string result_description          # result description
---
# Feedback
miar_msgs/GraphNode route_performed      # feedback da  rota percorrida
float32 vx_robot                   # feedback da velocidade longitudinal
float32 vy_robot                   # feedback da velocidade lateral
float32 wrobot                     # feedback da velocidade angular
```

Figura 60: Estrutura e descrição da *action Docking* que comunica bidireccionalmente entre o *Docking* e o *Task Manager.*

A manobra de acostagem/desacostagem, segue uma máquina de estados que organiza o comportamento do sistema desde a receção de um novo *goal* até à sua conclusão. O processo inicia-se num estado de *idle*, aguardando a receção do objetivo enviado pelo *Task Manager*. Enquanto não recebe essas instruções, a componente permanece inativa. Assim que o *goal* é recebido, a mensagem é processada e armazenada. Por conseguinte, o sistema inicia o algoritmo de acostagem, que procede ao ajuste das velocidades longitudinais, laterais e angulares conforme os parâmetros estabelecidos. A acostagem é efetuada como se fosse a navegação holonómica, isto é, a cada ciclo de computação, o algoritmo calcula os vetores (v_x, v_y e w) considerando a posição atual do veículo, a posição e orientação da via *point* e a informação sensorial dos obstáculos em torno do veículo. Porém, antes de enviar os

comandos para a movimentação, o algoritmo efetua uma validação das distâncias de segurança. Por último, o processo de acostagem, pode ser interrompido a qualquer momento pelo *Task Manager* e retorna ao estado *idle.*

O fluxo de execução das manobras de acostagem está apresentado na Figura 61.

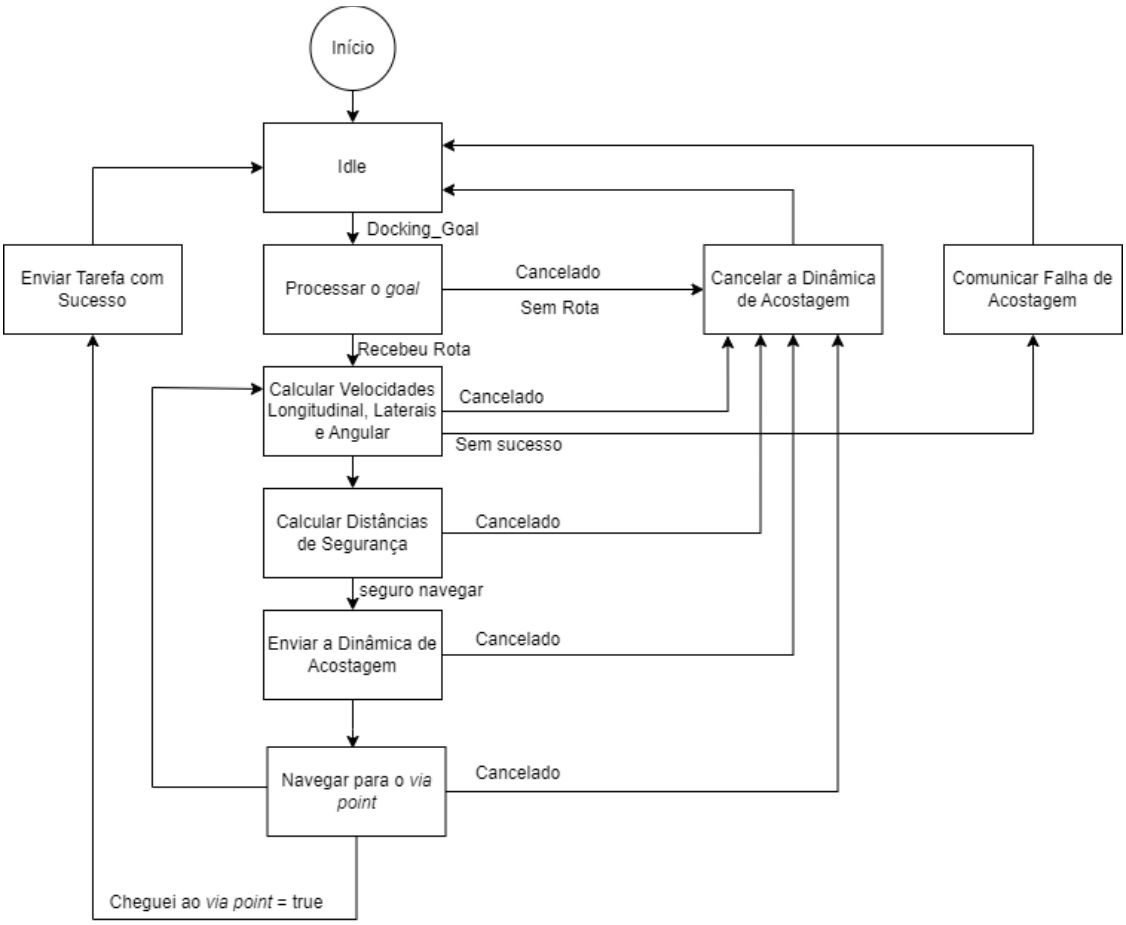


Figura 61: Máquina de estados que define o comportamento das manobras de acostagem.

7. Ensaios e Resultados Experimentais

Este capítulo descreve os ensaios efetuados e, os respetivos resultados, referentes à validação dos comportamentos de navegação da plataforma móvel. Os ensaios foram conduzidos num ambiente de simulação, respeitando os requisitos e as especificações definidas para o projeto. Para isso, foi utilizado um cenário simulado, detalhado no subcapítulo Simulação, que reproduz uma linha industrial no *software CoppeliaSim*. O sistema é controlado através da *framework* ROS, permitindo verificar a coordenação das operações e validar os comportamentos necessários para executar as tarefas, de acordo com as condições circundantes.

De forma a exemplificar o funcionamento do projeto, foram estabelecidos vários ensaios.

7.1. Ensaio Navegação com Obstáculos Espalhados pelo Caminho

A primeira missão a ensaiar é complexa uma vez que o robô navega no cenário previsto desviando-se de obstáculos: navegar até ao supermercado, e encostar no *dollie channel* pretendido (imaginando que pega numa caixa – "Docking 1"), fazer o *undocking* e navegar de forma holonómica por um dos corredores horizontais até um *via point* perto das rampas de abastecimento. Ao chegar aí, o robô deve efetuar o *docking* (para entregar a caixa) – "Docking 2" -, sair da acostagem e, dá concluída uma tarefa de *load*. A partir do ponto de *undock* até à outra rampa localizada no outro corredor horizontal, irá repetir o processo realizado, porém no sentido inverso. Nesta missão, o veículo tem ainda movimentações que são efetuadas em marcha-atrás, devido ao *layout* do cenário.

Para representar de forma ilustrativa o cenário, que inclui os pontos de interesse, tanto os pontos de *docking* nos supermercados como nas rampas de abastecimento, assim como a localização dos obstáculos, da zona de supermercado e das rampas de abastecimento, foi elaborada a Figura 62.

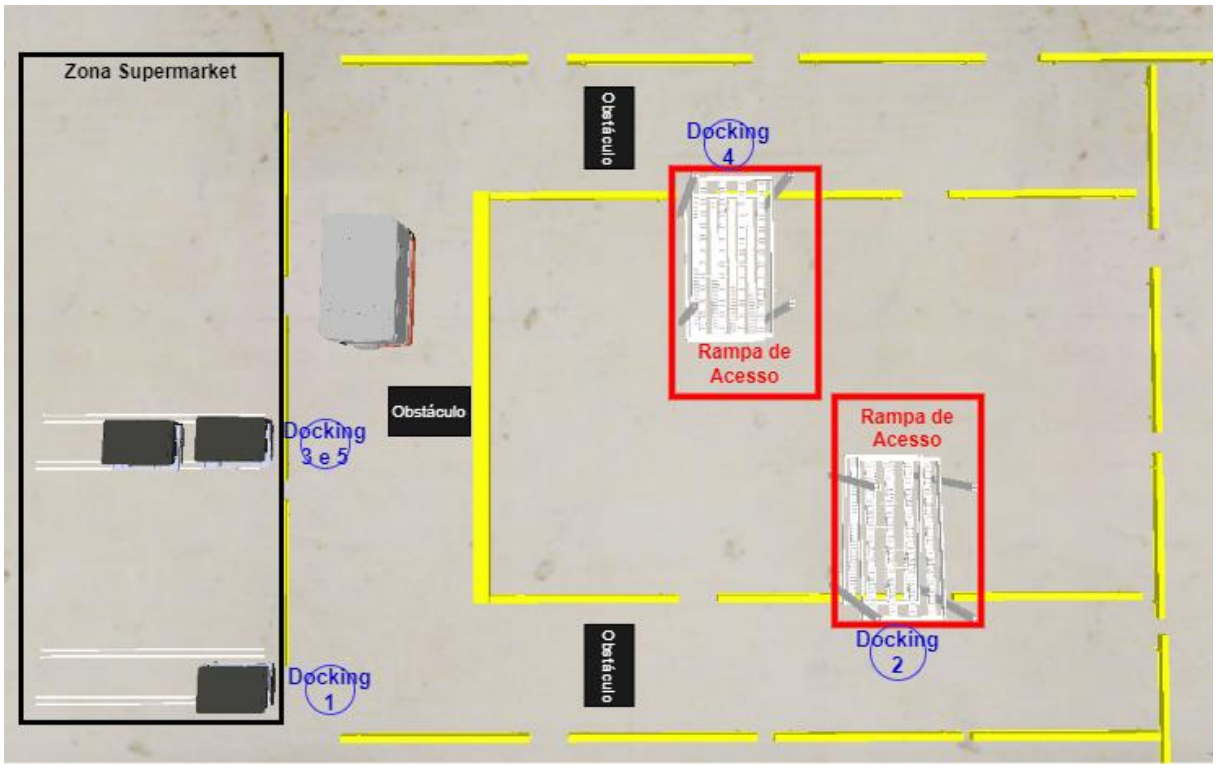
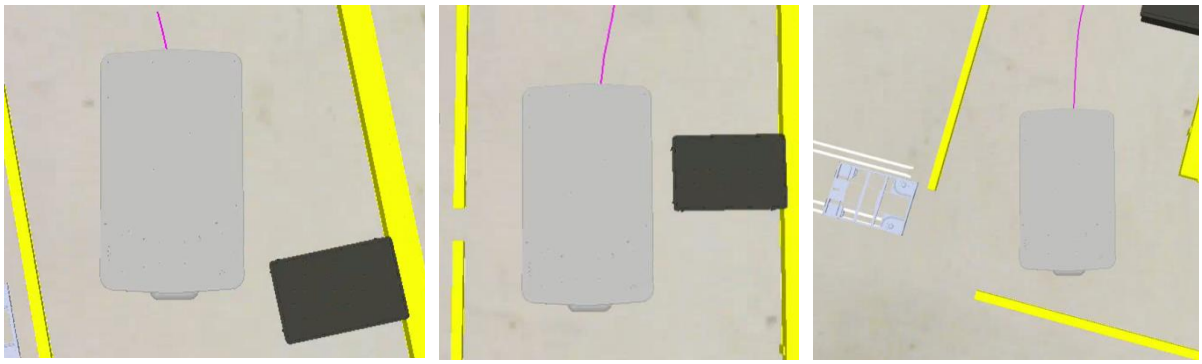


Figura 62: Cenário exemplo do ensaio com obstáculos. Apresenta a localização inicial do robô, das rampas de abastecimento, dos pontos de *docking*, dos obstáculos e do supermercado.

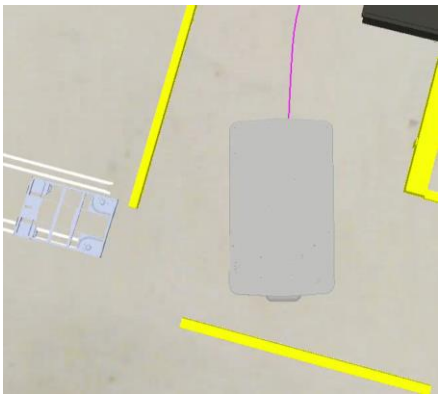
Para fins de demonstração, o veículo começou no meio do cenário, iniciando a sua marcha de forma diferencial, em marcha-atrás, para se dirigir ao *via point* desejado. Para alcançar esse ponto, o veículo teve de realizar uma navegação segura, evitando colisões com um obstáculo que foi colocado no seu caminho. Essa sequência de *frames* está presente na Figura 63.



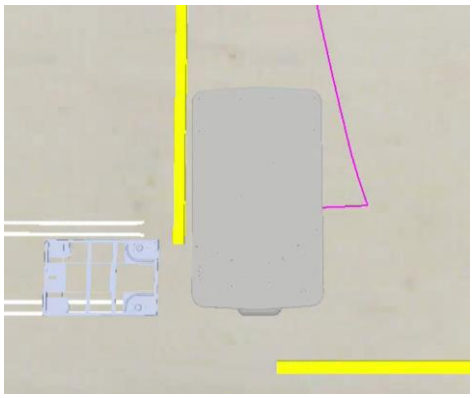
(a) (b) (c)

Figura 63: Navegação Não-Holonómica de marcha atrás a contornar um obstáculo. Analisar o Vídeo no Youtube.

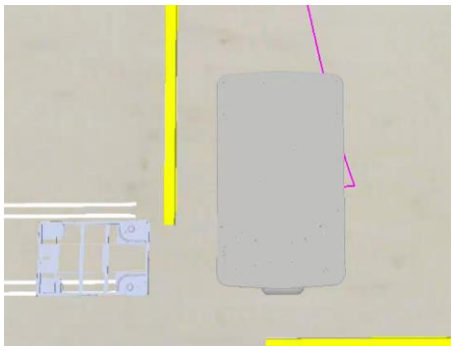
Após isso, o veículo alcança um *via point* onde inicia as manobras de acostagem ao supermercado. Quando o veículo concluiu a manobra, o veículo parou, e uma caixa foi colocada em cima da plataforma através de um braço que faz parte do veículo. Em seguida, a plataforma afastou-se para continuar o trajeto com a caixa. De modo a ilustrar esses movimentos, apresenta-se na Figura 64 uma sequência dos *frames*.



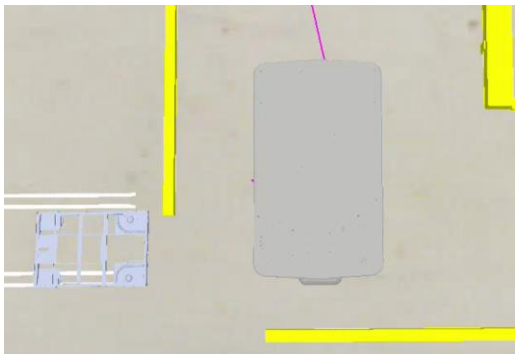
(a): Ponto que inicia acostagem



(b): *Pose* final da manobra de acostagem



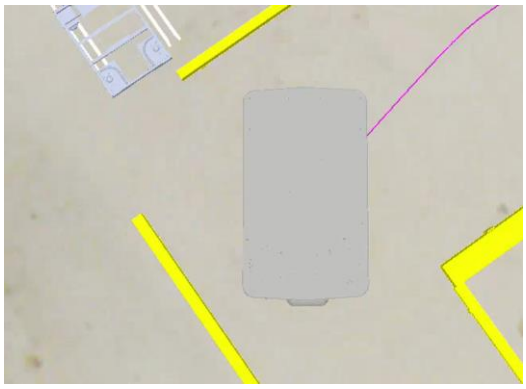
(c): Começa a sair da acostagem



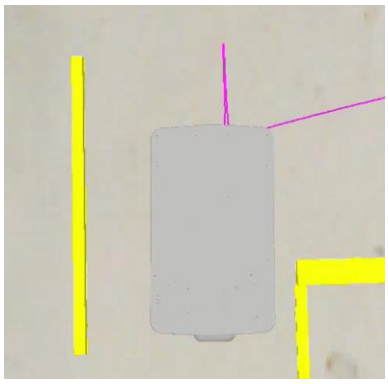
(d): *Pose* final

Figura 64: *Docking* para replicar o ir buscar caixas de material.

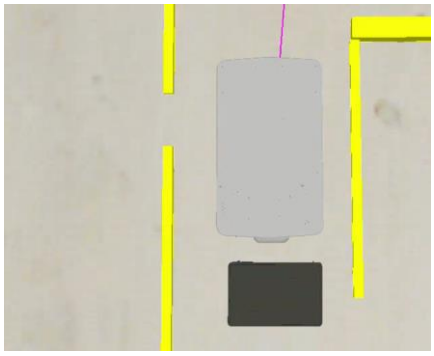
Após visitar o supermercado, a plataforma realiza o transporte do material até às rampas de abastecimento, navegando agora de forma holonómica. Podem-se visualizar esses movimentos na Figura 65. Nesses movimentos, demonstra-se ainda o veículo a parar a marcha atempadamanete perante um obstáculo que se encontra a impedir o movimento.



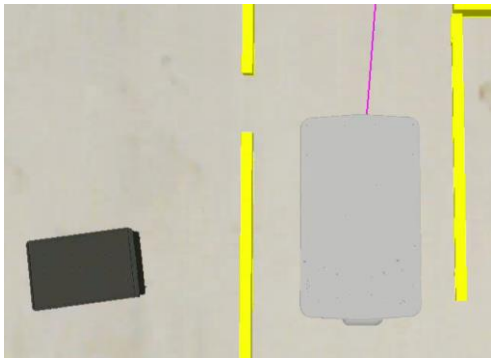
(a): Inicia a sua movimentação holonómica



(b): Move-se com a orientação fixa devido ao cenário



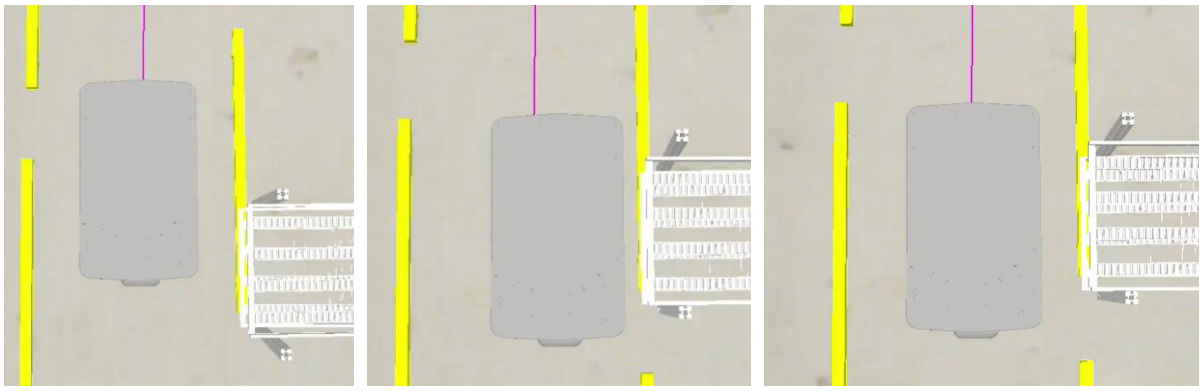
(c): Coloca em *stand-by* a navegação ao detetar obstáculo, parando o veículo.



(d): Quando deixa de detetar o obstáculo impeditivo, prossegue caminho

Figura 65: Navegação Holonómica capacitada de evitar colisões com obstáculos.

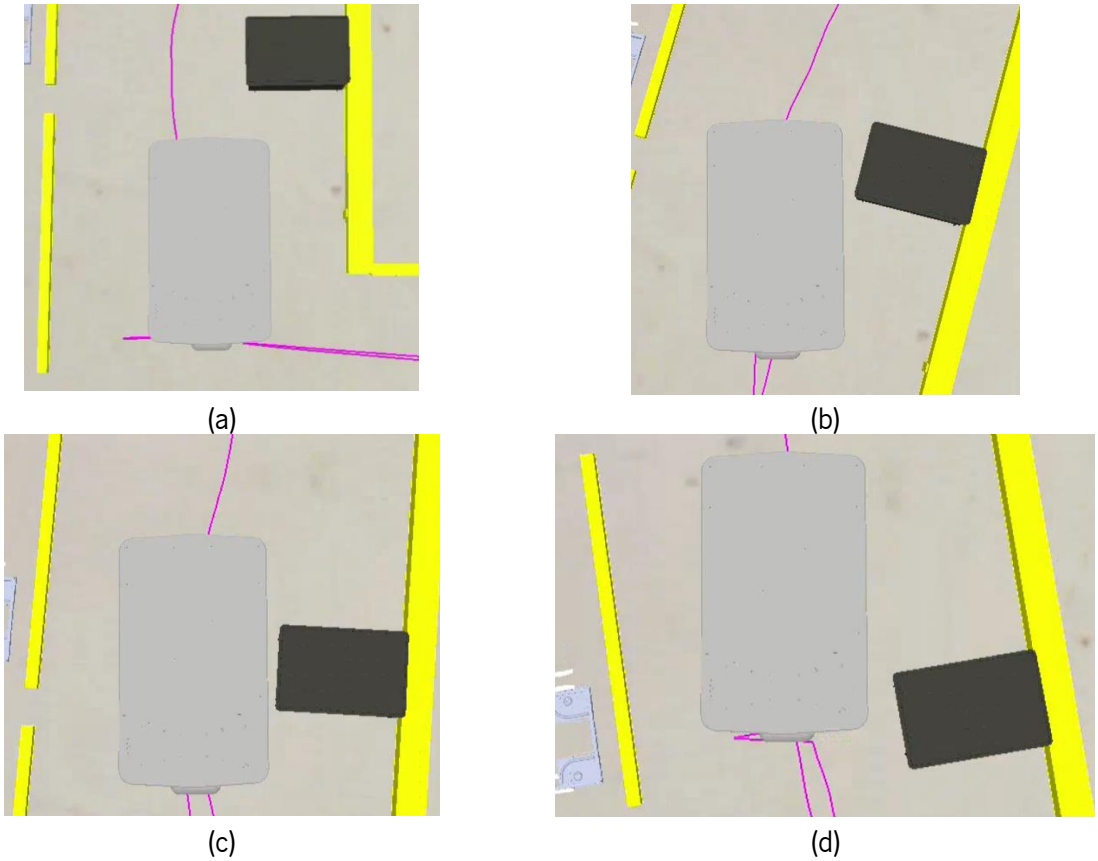
Nesse corredor, existe uma rampa de abastecimento na qual o robô deve efetuar a manobra de acostagem (verificar as manobras na Figura 66). Aqui, quando o robô se encostar, deve parar a marcha para que o braço possa entregar o material na rampa de acesso e, se necessário, recolher caixas vazias para devolver ao supermercado.



(a) (b) (c)

Figura 66: Manobras de Acostagem na rampa de abastecimento.

Posteriormente, como é possível visualizar na Figura 67, o veículo retrocedeu pelo corredor de forma holonómica até chegar ao corredor principal. Uma vez no corredor principal, retomou a navegação diferencial e precisou ultrapassar o obstáculo que inicialmente estava colocado no cenário, continuando o seu percurso sem parar e evitando colisões, uma vez que tem espaço suficiente para prosseguir a viagem.



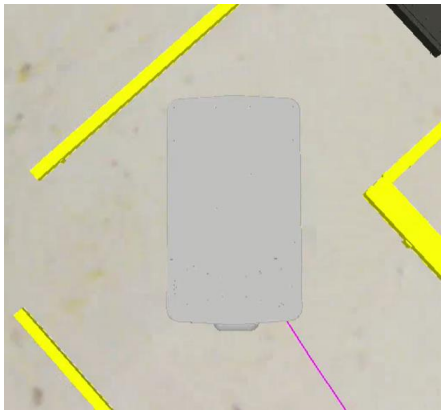
(a) (b)

(c) (d)

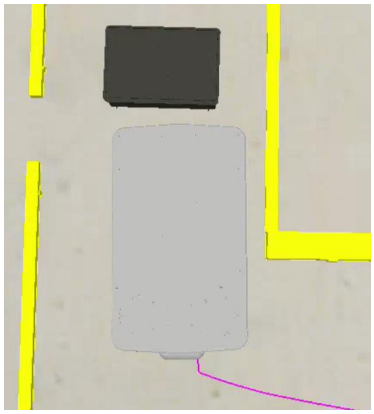
Figura 67: Navegação Não-Holonómica no corredor principal evitando colisões com obstáculos.

Seguidamente no processo, o veículo realiza a mesma navegação no corredor superior horizontal, conforme ilustrado na Figura 68. Desta vez, encontra um obstáculo no chão, que é detetado pelos

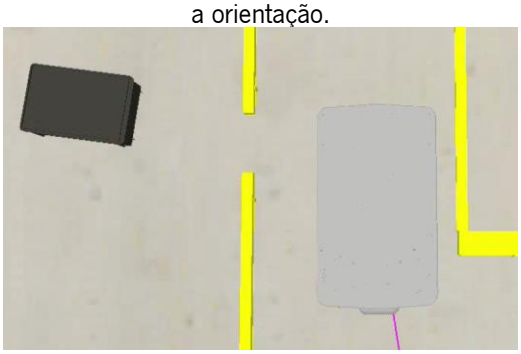
sensores frontais do robô, fazendo com que o robô pare até que o obstáculo seja removido do seu caminho, permitindo assim a continuação do percurso.



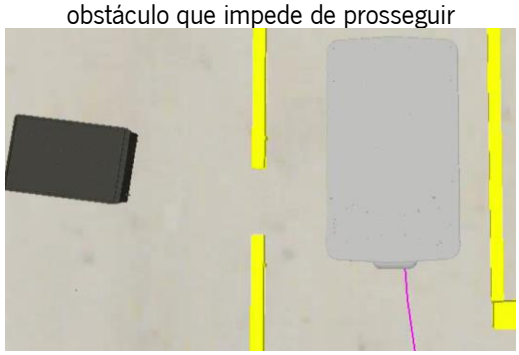
(a): Inicio da navegação holonómica, ainda a corrigir a orientação.



(b): Entra em *standby* uma vez que tem um obstáculo que impede de prosseguir



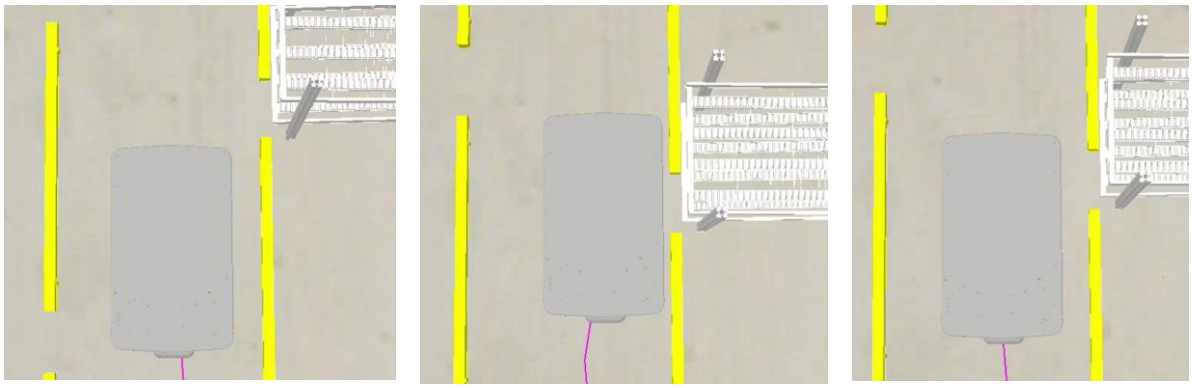
(c): Prossegue caminho



(d): Prossegue caminho

Figura 68: Navegação Holonómica no corredor superior evitando colidir com obstáculos.

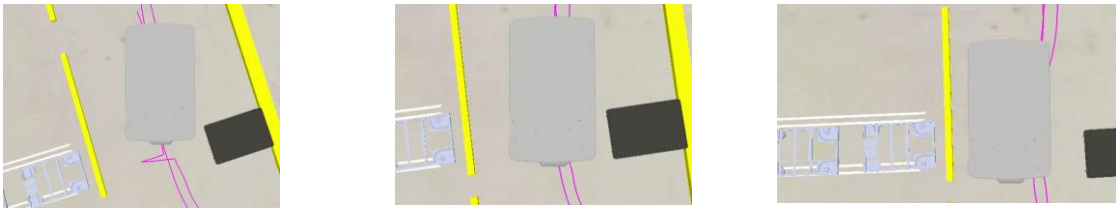
O robô continua a sua navegação holonómica até chegar ao *via point* indicado, onde inicia a manobra de acostagem. Quando se encostar, deve parar para que o braço possa recolher as caixas vazias da rampa de acesso e devolvê-las ao supermercado. Em seguida, deverá sair do sistema de acostagem para demonstrar o seu bom funcionamento, conforme ilustrado na sequência apresentada na Figura 69.



(a) (b) (c)

Figura 69: Manobras de Acostagem no corredor superior.

Por fim, a plataforma deve simular um último docking, como se estivesse a entregar caixas vazias ao supermercado, conforme demonstrado na Figura 70.



(a) (b) (c)

Figura 70: *Docking* na zona do *supermarket*.

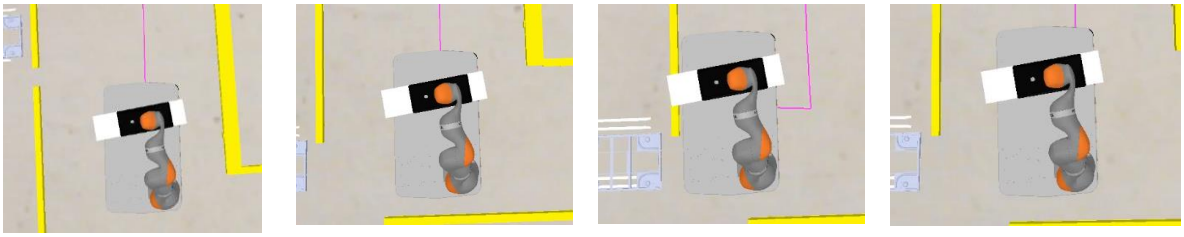
Refira-se que este ensaio teve uma duração de 4 min e 20 segundos.

7.2. Ensaio Navegação Sem Obstáculos pelo Caminho

De modo a comparar a rapidez de movimentos, ensaiou-se o mesmo cenário da Figura 62, porém em condições ideias, isto é, sem nenhum obstáculo. A plataforma realizou a mesma trajetória, assumindo os mesmos *viapoints* alvos e, os mesmos pontos de acostagem.

Nesta missão, o robô deve navegar de forma precisa e eficiente para realizar tarefas de transporte de caixas entre o supermercado e as rampas de abastecimento. Ao contrário da versão anterior, nesta variação do cenário não há necessidade de desvio de obstáculos, permitindo uma navegação mais direta e fluida. O cenário é o mesmo ao da Figura 71, retirando os obstáculos.

Refira-se ainda, que neste cenário, sem a presença de obstáculos, a navegação do robô teve uma melhor fluidez e mais eficiência, uma vez que realizou o percurso em menos tempo (o tempo percorrido foi 3 minutos)



(a) (b) (c) (d)

Figura 71: Trajetória realizada no ensaio sem obstáculos. A ordem dos acontecimentos é de (a) até (s). Analisar Vídeo Ensaio sem Obstáculos.

Perante os resultados obtidos, elaborou-se a Tabela 6, de forma a comparar os tempos com a presença de obstáculos e sem, para alcançar os respetivos pontos de interesse

Tabela 6: Comparação dos tempos dos Ensaios com e Sem Obstáculos.

Zonas	Tempos Sem Obstáculos	Tempos Com Obstáculos
Docking 1: load caixa cheia dos *dollies*	00:20	01:01
Docking 2: levar caixas cheias dos *dollies* até à rampa de abastecimento	00:55 – 00:20 = 00:35	(Retirando de imediato o obstáculo, portanto demora mais tempo) 01:38 – 01:01 = 00:37
Docking 3: levar caixas vazias desde a rampa de abastecimento até aos respetivos *dollies*	01:35 – 00:55 = 00:40	02:21 – 01:38 = 00:43
Docking 4: levar caixas cheias desde os *dollies* até ao posto e pegar nas caixas vazias do posto de abastecimento	02:24 – 01:35 = 00:49	(Retirando de imediato o obstáculo, portanto demora mais tempo) 03:24 – 02:21 = 01:03
Docking 5: colocar as caixas vazias nos respetivos *dollies* do supermercado	02:56 – 02:24 = 00:32	04:10 – 03:24 = 00:46

Os tempos apresentados reforçam o impacto dos obstáculos, visto que aumentam significativamente o tempo necessário para completar o percurso.

A diferença entre os tempos com e sem obstáculos demonstra que mesmo no início da navegação (como no caso do "1° *Docking* Supermercado" e do "1° Posto"), a presença de obstáculos causa um aumento relevante no tempo de navegação. Nas zonas seguintes, como o "2° Posto" e o "3° *Docking* Supermercado", esse impacto torna-se ainda mais expressivo, devido ao acumular dos atrasos. A menção de que os obstáculos são removidos "de imediato", refere-se que os tempos totais podem ser

ainda maiores se a remoção do obstáculo for mais demorada, o que pode tornar o tempo ainda mais lento e imprevisível.

A navegação sem obstáculos é visivelmente mais rápida e eficiente, permitindo atingir os pontos de interesse em tempos menores. Comparando diretamente os tempos, refira-se que os obstáculos podem quase dobrar o tempo total da navegação. A presença de obstáculos ao longo da navegação aumenta consideravelmente o tempo total necessário para alcançar os pontos de interesse. Mesmo com a remoção imediata dos obstáculos, o tempo total é significativamente afetado, o que pode comprometer a eficiência e previsibilidade da navegação.

7.3. Ensaio Navegação com Operadores Humanos

Uma vez que no ambiente industrial os corredores também podem ser frequentados por operadores, realizaram-se ensaios para avaliar a eficácia da navegação com operadores estáticos e em movimento.

Posto isto, colocou-se um operador a caminhar no sentido contrário ao veículo no corredor principal e, um operador parado no corredor inferior horizontal como se estivesse distraído e não soubesse que o robô se estava a aproximar. Exemplo deste cenário é apresentado pela Figura 72.

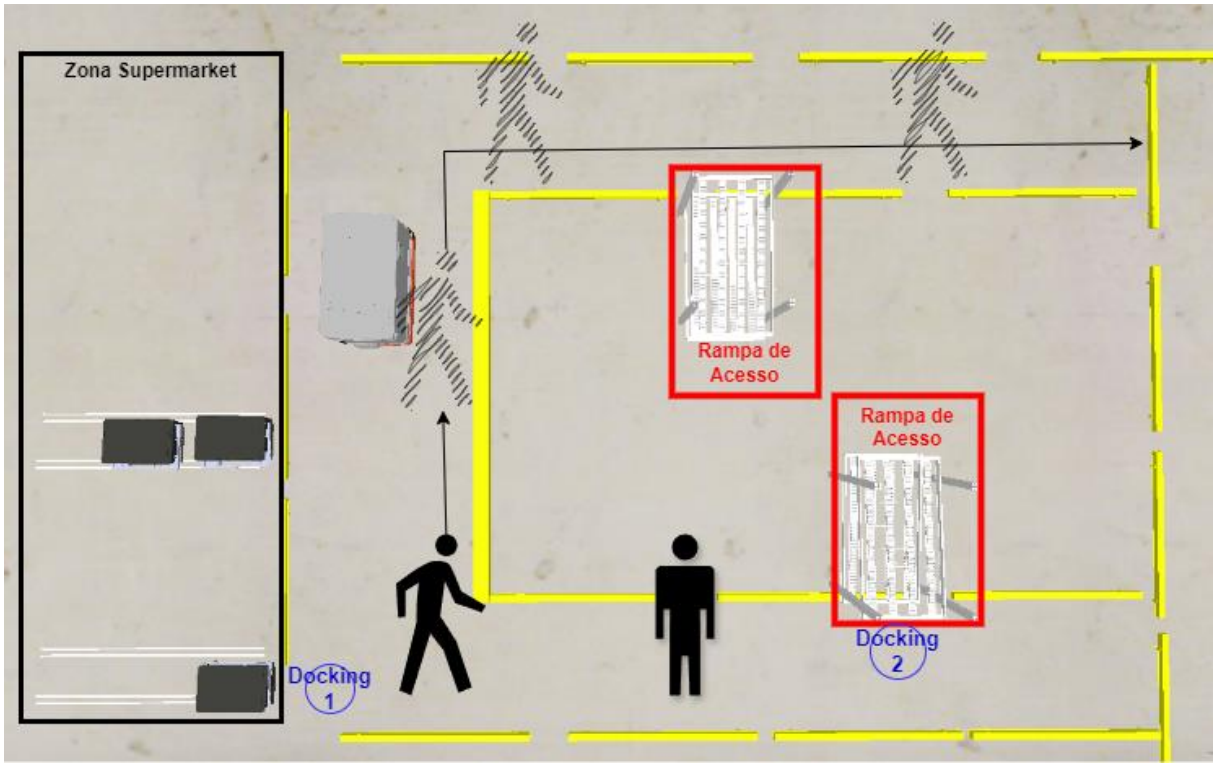


Figura 72: Cenário exemplo do ensaio com operadores humanos em movimento. Apresenta a localização inicial do robô, das rampas de abastecimento, dos pontos de docking, dos operadores (em movimento e parado) e do supermercado.Analisar Vídeo Navegação Segura coexistindo com Operadores.

Deste modo, a plataforma inicia o seu percurso e começa a detetar o operador à medida que se aproxima. Com a aproximação, o veículo ajusta a sua orientação para continuar a avançar e contornar o operador em movimento, evitando assim uma colisão. Após superar com sucesso o primeiro desafio de contornar o operador, o veículo prossegue até ao corredor horizontal inferior. Neste corredor, encontra um operador de costas e, como não há espaço suficiente para o contornar, suspende a sua navegação até que o caminho esteja desimpedido.

O trajeto realizado e os resultados deste desafio encontram-se pela respetiva ordem de execução na Figura 73.

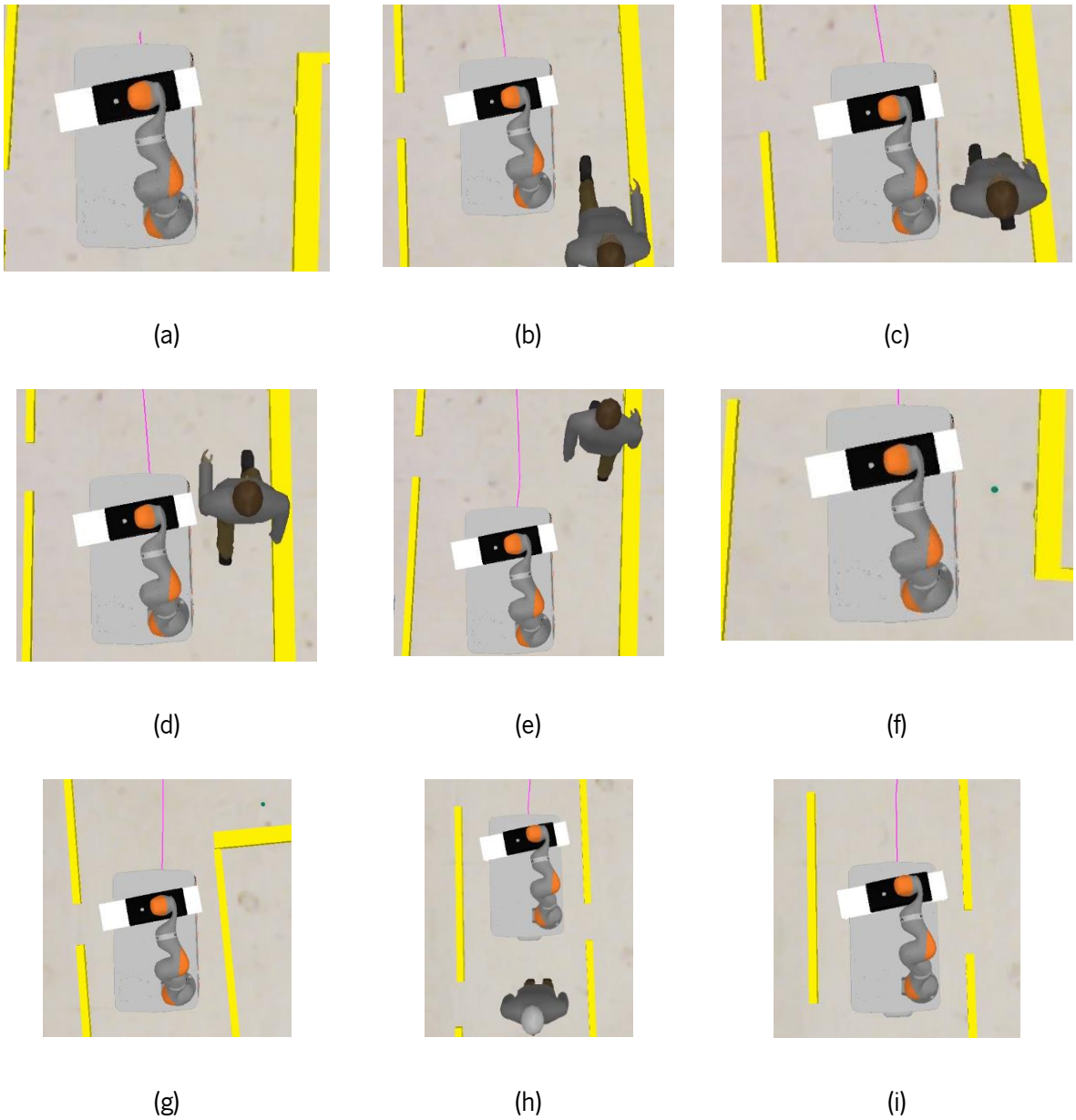


Figura 73: Trajetória realizada no ensaio com a presença de operadores humanos em movimento e estático.

7.4. Ensaio Contorno de Obstáculos através da Navegação Holonómica.

O ensaio de contorno de obstáculos através da navegação holonómica tem como objetivo testar a capacidade do robô de desviar-se de obstáculos de forma eficiente, utilizando as características deste tipo de navegação, que permite movimentos em qualquer direção, independentemente da orientação do robô.

7.4.1. Ensaio Contorno de Obstáculos através da Navegação Holonómica em Espaços Amplos

Neste ensaio, o robô tem a sua orientação sempre fixa e é testado num espaço amplo, como está apresentado o cenário da Figura 74.

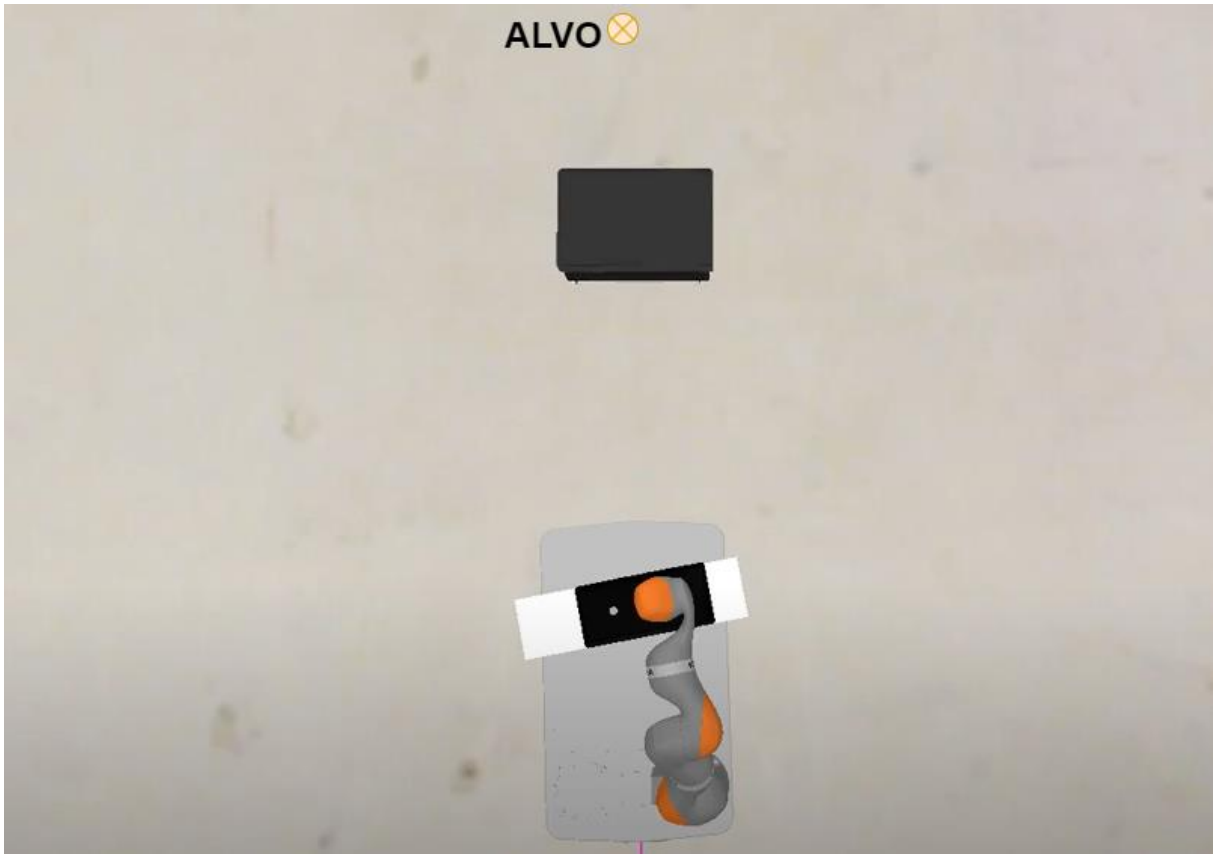
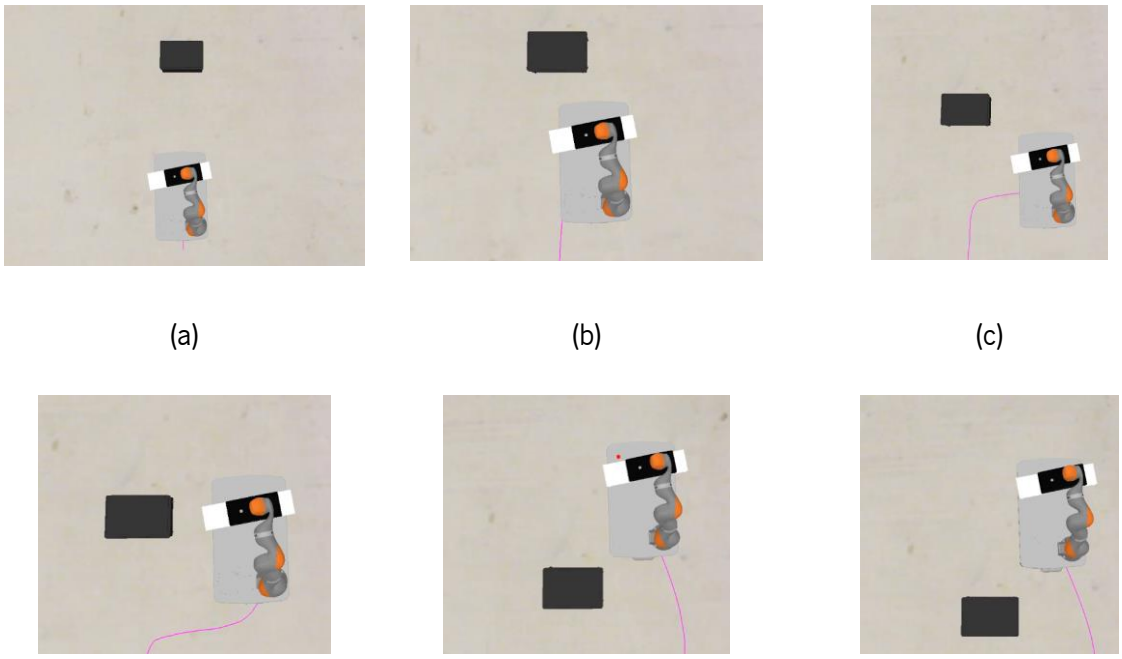


Figura 74: Cenário exemplo da simulação de contorno de obstáculos através da navegação holonómica, com orientação fixa, em espaço amplo. Analisar Vídeo Desvio Obstáculo em Navegação Holonómica.

Inicialmente, o veículo é posicionado num ponto de partida dentro de um cenário distinto dos ensaios anteriores, tendo como objetivo alcançar um ponto intermédio predefinido, localizado após um obstáculo. Neste módulo, a navegação holonómica foi concebida de forma a permitir que o robô, ao detetar o obstáculo no seu percurso, ajuste a sua trajetória sem necessidade de realizar manobras de rotação, como seria exigido nos sistemas de navegação diferencial.

O movimento inicia-se com o robô avançando em direção ao alvo, localizado atrás do obstáculo. Quando o sensor lidar deteta a presença de um objeto à frente, o robô desloca-se lateral e diagonalmente para evitar a colisão, mantendo a orientação fixa durante todo o desvio. Após contornar o obstáculo, o robô retoma a trajetória original em direção ao ponto intermédio.

Durante este ensaio, a navegação holonómica mostrou-se eficiente em cenários de espaços amplos, permitindo o contorno de obstáculos sem a necessidade de mudanças na orientação do robô. Este ensaio comprova a capacidade do sistema de evitar colisões de forma fluida e, demonstra a flexibilidade e a segurança da navegação holonómica. O resultado deste ensaio está demonstrado na Figura 75.



(a) (b) (c)

(d) (e) (f)

Figura 75: Ensaio de contorno de obstáculos através da navegação holonómica, em espaços amplos.

7.4.2. Ensaio Contorno de Obstáculos através da Navegação Holonómica em Espaços Exíguos

Neste ensaio, o robô tem a sua orientação sempre fixa e é testado num cenário exíguo, com as dimensões do corredor idealizado para o projeto. Exemplo do cenário para este ensaio está evidenciado na Figura 76.

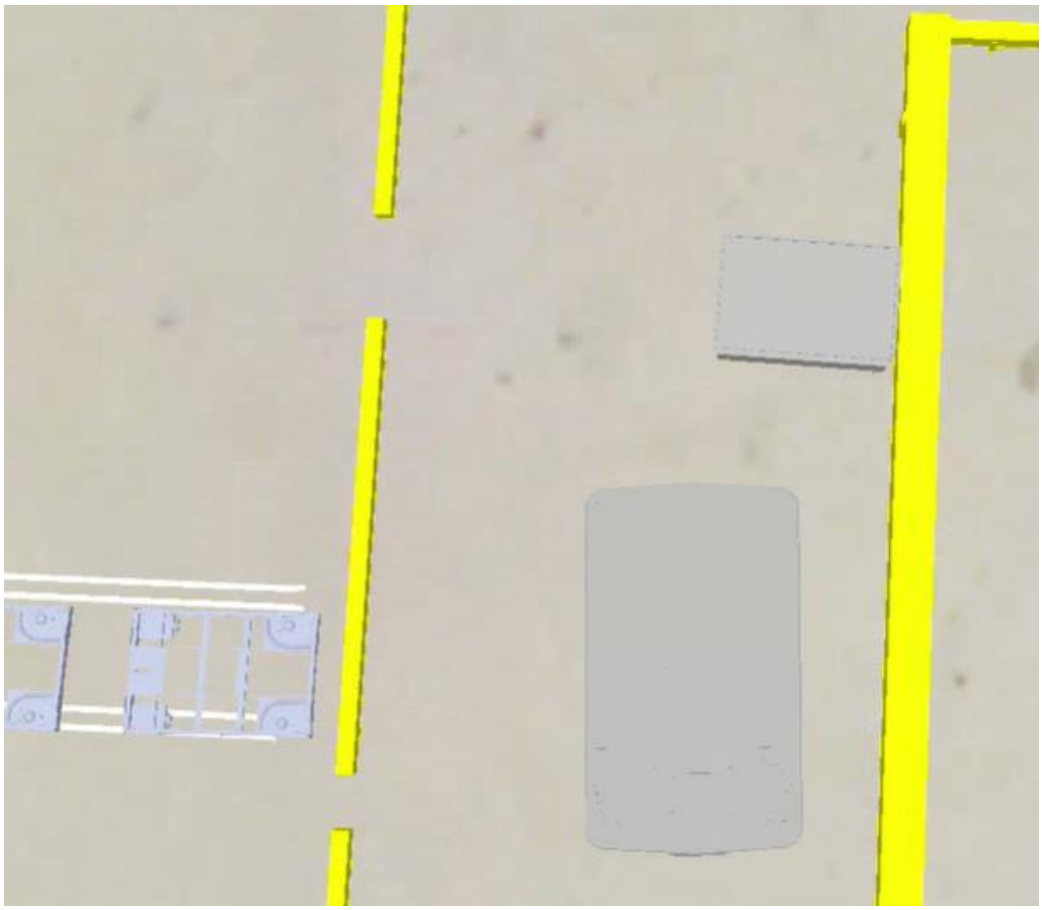
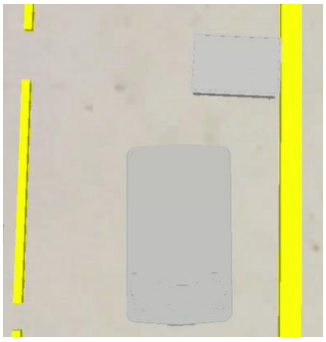
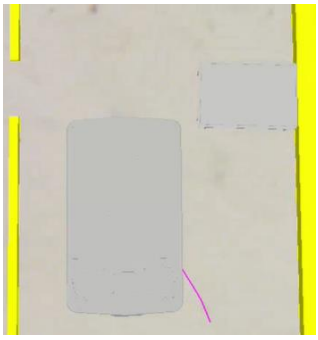


Figura 76: Cenário exemplo para o ensaio de contorno de obstáculos a navegar de forma holonómica, com a orientação fixa, em espaços exíguos. Analisar Vídeo da Colisão em Navegação Holonómica a desviar de obstáculos em espaços exíguos.

Inicialmente, a plataforma é posicionada no ponto de partida e inicia o seu movimento, aproximando-se do obstáculo a ser contornado. Ao começar a detetá-lo, o veículo inicia a manobra de desvio. Esse movimento de contorno foi parcialmente bem-sucedido, pois, ao detetar o obstáculo (caixa) apenas com os sensores diagonais traseiros e continuar a identificar obstáculos à sua esquerda (proteção amarela), o veículo retorna à trajetória original cedo demais, o que resulta em uma colisão com a parte traseira na caixa posicionada no meio do percurso. A evolução sequencial da movimentação encontra-se evidenciada na Figura 77.





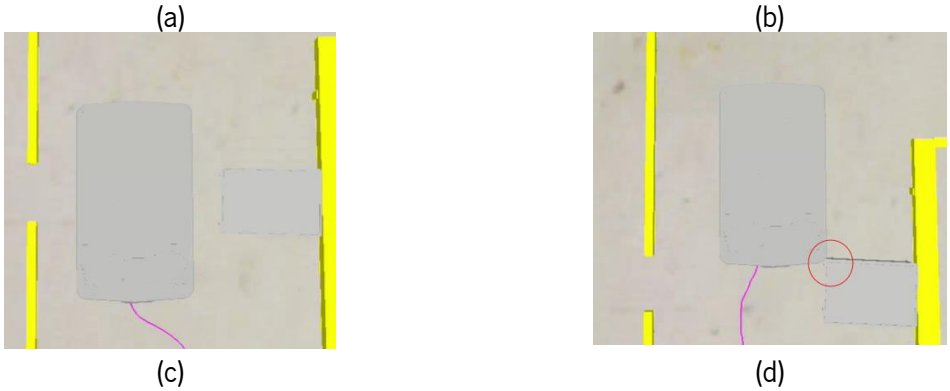


Figura 77: Ensaio de contorno de obstáculos através da navegação holonómica, em espaços exíguos.

Diante dos resultados negativos obtidos neste ensaio, conclui-se que não é seguro utilizar este contorno de obstáculos por meio de navegação holonómica neste cenário restrito.

8. CONCLUSÕES E TRABALHO FUTURO

Neste capítulo são descritas as conclusões do projeto efetuado, discutidos os resultados alcançados e as oportunidades de trabalho futuro.

8.1. CONCLUSÃO

O aumento da competitividade no mercado global, obriga a uma maior eficiência operacional nas indústrias, a fim de serem capazes de manterem a sua competitividade no mercado em que atuam. Tendo em consideração isto, este projeto apresenta uma solução flexível para automação de processos. O foco incidiu no desenvolvimento de módulos que permitem a navegação autónoma da plataforma omnidirecional Kuka KMP 200 OmniMove, em ambiente industrial, atendendo às exigências do processo industrial.

A partir de uma análise detalhada das soluções tecnológicas existentes, identificaram-se estratégias eficazes para o controlo de movimentos, tendo em consideração as características holonómicas da plataforma e o desafio do espaço exíguo para ambiente de operação. A solução apresentada, recorreu aos métodos de navegação baseados em sistemas dinâmicos não-lineares, o que possibilita uma flexibilidade e adaptação a mudanças no ambiente. Estes sistemas dinâmicos não lineares, foram também utilizados para a navegação holonómica e para as manobras de acostagem. As manobras de acostagem são facilitadoras para a interação com as rampas de abastecimento e com as zonas do *supermarket*. Não obstante, devido ao cenário ser exíguo e em forma de "U", o veículo teve de ser capacitado para executar navegação em marcha-atrás, de forma segura e eficiente. Por conseguinte, o robô tem de entrar nos corredores mais exíguos, com uma certa orientação, devido à necessidade do braço estar do lado direito relativamente à rampa de abastecimento.

A arquitetura do sistema foi concebida de forma modular, permitindo o desenvolvimento independente de componentes da solução. Entre esses destacam-se o *Positioning*,, responsável por determinar a localização e orientação do veículo, o *Obstacle Detection*, que assegura a deteção de obstáculos no trajeto, para garantir segurança nas operações, e o *Movement Controller*, que contém os *packages* de navegação. Estes módulos, são interligados entre si, através de um sistema central que gere os comandos e as tarefas, denominado *Task Manager*. Este tem como função orquestrar a execução das

tarefas, distribuindo-as entre os módulos, por forma a garantir que cada componente funcione de forma coordenada e eficiente.

O trabalho desenvolvido também visa superar algumas limitações observadas em projetos anteriores, como o trabalho realizado por Correia (2023). No seu projeto, identificaram-se dificuldades operacionais em ambientes mais exíguos, onde a navegação holonómica e a execução de manobras em espaços reduzidos eram limitadas. A solução apresentada neste trabalho aborda essas lacunas, permitindo a navegação em corredores estreitos por meio do controlo preciso da orientação do veículo, mesmo durante a navegação não holonómica. Além disso, foi implementada a capacidade de *docking* utilizando Sistemas Dinâmicos Não Lineares (SDNL), garantindo uma maior precisão nas manobras de acostagem.

Outro avanço foi a inclusão de uma gama de deteção de obstáculos com cobertura de 360 graus, que permite ao veículo identificar obstáculos e reportar a informação conforme a direção de marcha, assegurando uma navegação segura nos dois sentidos de marcha.

Além disso, foi realizada a migração de todos os módulos para o *ROS2*, um *middleware* que oferece maior flexibilidade e desempenho, proporcionando uma arquitetura mais robusta e escalável.

No que toca ao contorno de obstáculos de forma holonómica, nos ensaios realizados em espaços exíguos, o robô apresentou dificuldades em contornar obstáculos de forma eficaz. Durante essas tentativas, foi observado que a parte diagonal traseira do robô colide repetidamente com os objetos que tenta contornar, indicando que o sistema de navegação não foi capaz de ajustar a trajetória adequadamente em ambientes confinados. Essa limitação comprometeu a segurança e a precisão da movimentação, resultando em falhas na execução das tarefas sem colisões. Definiu-se assim que em espaço exíguos, em que o movimento seja holonómico o veículo deve parar se detetar um obstáculo, avisando o *Service Manager* da falha do serviço.

Por outro lado, nos ensaios realizados em espaços amplos, os resultados foram positivos. A navegação holonómica demonstrou potencial para o robô realizar os contornos de obstáculos com sucesso, sem colisões ou interrupções. A capacidade de mover-se lateralmente e de forma precisa garantiu que o robô completasse os contornos dos obstáculos de forma fluida, mostrando-se possível para ambientes com mais liberdade de movimento. Esses resultados mostram que, em cenários amplos, o robô é capaz de operar de modo totalmente holonómico com eficiência e segurança, garantindo o sucesso das operações.

No que diz respeito à navegação em ambientes com operadores em movimento e estáticos, os ensaios também apresentaram resultados positivos. O robô conseguiu adaptar-se à presença de pessoas, mantendo uma navegação segura. Ao identificar operadores a caminhar, o sistema ajustou sua trajetória de forma a evitar colisões com os mesmos, desviando-se ou parando (nos espaços que não consegue contornar).

A validação do sistema foi realizada num ambiente de simulação, onde cada *package* e comportamento foi testado individualmente, sendo posteriormente realizados os testes de integração. Os ensaios demonstraram a viabilidade da solução proposta, atingindo os objetivos definidos inicialmente e tendo uma base para implementações em ambiente real.

Do acima exposto, conclui-se que a solução desenvolvida cumpre o seu propósito de automatizar o processo de transportar caixas e, demonstra a sua adaptabilidade ao meio que o rodeia, sendo uma potencial alternativa para a automação em ambientes industriais exíguos e exigentes.

8.2. TRABALHO FUTURO

Os desenvolvimentos e soluções demonstrados neste projeto, pressupõe que os mesmos continuem a ser acompanhados e explorados, a fim de garantir a melhor solução, sendo passível a alterações e melhorias face às propostas apresentadas. Apesar do balanço final deste projeto ser positivo, existem aspetos que carecem melhorias.

Em primeiro lugar, é de realçar a importância de implementar este estudo em contexto real, uma vez que apenas foi simulado. Esta implementação poderá identificar algumas fragilidades do sistema, não previstas em simulação, sendo necessárias algumas correções do mesmo.

De salientar ainda que, em paralelo com esta dissertação, há outras dissertações a serem desenvolvidas, englobando outros módulos da temática em estudo, como a visão, a junção com o braço robótico e um gestor de frotas, sendo de futuro necessária a incorporação dos diferentes módulos, num único sistema, a fim de validar o funcionamento dos mesmos, em conjunto.

Outro ponto a ser explorado seria a melhoria no desvio de obstáculos na navegação holonómica. Esta navegação apenas permite o contorno de obstáculos em espaços mais amplos, isto é, corredores mais largos, sendo, por isso de interesse ao projeto, melhorar este aspeto para os espaços exíguos. Desta forma, deve ser então melhorado o desenvolvimento de algoritmos de contorno de obstáculos de modo holonómico, focado em áreas exíguas.

Outra área de investigação futura pode ser a incorporação de módulos de interação com operadores humanos ou interfaces de voz para comandar o veículo ou solicitar ações. Assim, os trabalhadores poderiam interagir diretamente com o veículo, enviando comandos simples por voz, como "levar a caixa para a rampa 3" ou "parar imediatamente" e, o veículo efetuar essas ações solicitadas.

Para concluir, e como exposto acima, dar continuidade ao desenvolvimento dos diversos requisitos do projeto em causa, visando a melhoria contínua do mesmo, a fim de obter as operações cada vez mais eficientes.

REFERÊNCIAS

Almayyahi, A., Wang, W., Hussein, A. A., & Birch, P. (2017). Motion control design for unmanned ground vehicle in dynamic environment using intelligent controller. *International Journal of Intelligent Computing and Cybernetics*, *10*(4), 530–548. https://doi.org/10.1108/IJICC-11-2016-0044

Alves, J., Lima, T. M., & Gaspar, P. D. (2023). Is Industry 5.0 a Human-Centred Approach? A Systematic Review. In *Processes* (Vol. 11, Issue 1). MDPI. https://doi.org/10.3390/pr11010193

Amaral, M. (2022). *Indústria 4.0: Otimização de Linhas de Produção com Automated Mobile Robots*. Universidade do Minho.

António da Silva Santos. (2013). *Logística baseada em AGVs*. FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO.

B. Graf, U. Reiser, M. Hägele, K. Mauz, & P. Klein. (2009). Robotic Home Assistant Care-O-bot® 3 - Product Vision and Innovation Platform. *IEEE Workshop on Advanced Robotics and Its Social Impacts, Tokyo, Japan*.

Bicho, E. (1999). *Dynamic Approach to Behavior-Based Robotics: design, specification, analysis, simulation and implementation*.

Bogue, R. (2016). Growth in e-commerce boosts innovation in the warehouse robot market. *Industrial Robot*, *43*(6), 583–587. https://doi.org/10.1108/IR-07-2016-0194

Bostelman, R. V., & Hong, T. H. (2016). *Review of Research for Docking Automatic Guided Vehicles and Mobile Robots*. https://doi.org/10.6028/NIST.IR.8140

Chen, C. S., Lin, C. J., Lai, C. C., & Lin, S. Y. (2022). Velocity Estimation and Cost Map Generation for Dynamic Obstacle Avoidance of ROS Based AMR. *Machines*, *10*(7). https://doi.org/10.3390/machines10070501

Cherni, F., Boutereaa, Y., Rekik, C., & Derbel, N. (2016). *Path Planning for mobile robots using fuzzy logic controller in the presence of static and moving obstacles*.

Correia, J. (2023). *Controlo da navegação e manobras de acostagem de manipuladores móveis autónomos em ambientes de logística interna*.

Costa Silva, E., Bicho, E., & Erlhagen, W. (2006). *THE POTENTIAL FIELD METHOD AND THE NONLINEAR ATTRACTOR DYNAMICS APPROACH: WHAT ARE THE DIFFERENCES?*

Crowther, S., & Ford, H. (1924). *My life and work*. William Heinemann.

DAVIES, R. (2015). *Industry 4.0: Digitalisation for productivity and growth*. European Parliamentary Research Service. https://policycommons.net/artifacts/1335939/industry-40/

Dong, H., Wu, Z., Wang, J., Chen, D., Tan, M., & Yu, J. (2023). Implementation of Autonomous Docking and Charging for a Supporting Robotic Fish. *IEEE Transactions on Industrial Electronics*, *70*(7), 7023–7031. https://doi.org/10.1109/TIE.2022.3201329

Eugene, K. (2015). *Amazon is now using a whole lot more of the robots from the company it bought for $775 million*.

Fraunhofer Institute for Factory Operationand Automation IFF. (n.d.). *Research Platform ANNIE - Fraunhofer IFF. (s.d.).* Https://Www.Iff.Fraunhofer.de/En/Business-Units/Robotic-Systems/Research-Platform-Annie.Html.

Frederick Winslow, T. (2004). *Scientific management*. Routledge.

Ganesharajah, T., Hall, N. G., & Sriskandarajah, C. (1998). Design and operational issues in AGV-served manufacturing systems. *Annals of Operations Research, Springer*.

Graf, B. (2005). Dependability of Mobile Robots in Direct Interaction with Humans. *Advances in Human-Robot Interaction, 223-239.*

Heggem, C., & Wahl, N. M. (2020). *Mobile Navigation and Manipulation Configuration and Control of the KMR iiwa with ROS2*.

Iqbal, B. A., & Yadav, A. (2021). *FOURTH INDUSTRIAL REVOLUTION: Its Role and Contribution in Employment Generation and Skills Development*. https://doi.org/10.47509/JGETIB.1.1.2021.85-96

Ivanov, D. (2023). The Industry 5.0 framework: viability-based integration of the resilience, sustainability, and human-centricity perspectives. *International Journal of Production Research*, *61*(5), 1683–1695. https://doi.org/10.1080/00207543.2022.2118892

KUKA AG. (n.d.). Retrieved November 9, 2023, from https://www.kuka.com/pt-pt/produtos-servi%C3%A7os/mobilidade/rob%C3%B4s-m%C3%B3veis/kmr-iiwa

Laudon, K. C., & Traver, C. G. (2021). An Overview of Electronic Commerce (e-Commerce). *Journal of Contemporary Issues in Business and Government*, *27*(3). https://doi.org/10.47750/cibg.2021.27.03.090

Louro, L., Teixeira, D., Malheiro, T., Mesquita, L., Machado, T., Monteiro, S., Erlhagen, W., & Bicho, E. (2020). A safe autonomous stacker in human shared workspaces. *IECON 2020 The 46th Annual Conference of the IEEE Industrial Electronics Society (Pp. 4389-4396). IEEE*.

Louro, L., Teixeira, D., Malheiro, T., Monteiro, S., Erlhagen, W., & Bicho, E. (2019). Motion Control for Autonomous Tugger Vehicles in Dynamic Factory Floors Shared with Human Operators. *IEEE Open Journal of the Industrial Electronics Society*.

Lucas, J. M., Martínez-Barberá, H., & Jiménez, F. (2005). *Multi-objective Evolutionary Fuzzy Modeling for the Docking Maneuver of an Automated Guided Vehicle*.

Mokyr, J., & Strotz, R. H. (1998a). The Second Industrial Revolution, 1870-1914. *Storia Dell'economia Mondiale*, *21945*(1).

Mokyr, J., & Strotz, R. H. (1998b). The Second Industrial Revolution, 1870-1914. *Storia Dell'economia Mondiale*, *21945*(1).

Mpofu, R., & Nicolaides, A. (2019). Frankenstein and the Fourth Industrial Revolution (4IR): Ethics and Human Rights Considerations. In *African Journal of Hospitality, Tourism and Leisure* (Vol. 8, Issue 5). http//:www.ajhtl.com

Mukherjee, D., Gupta, K., Chang, L. H., & Najjaran, H. (2022). A Survey of Robot Learning Strategies for Human-Robot Collaboration in Industrial Settings. In *Robotics and Computer-Integrated Manufacturing* (Vol. 73). Elsevier Ltd. https://doi.org/10.1016/j.rcim.2021.102231

Ohno, T. (1998). *Toyota Production System: Beyond Large-Scale Production* (1st ed.). New York: Productivity Press.

Porunov, M., & Trifonov, A. (2020). Development of a Special Module for an Industrial Collaborative Robot for Assembling Standard Threaded Products. *2020 International Multi-Conference on Industrial Engineering and Modern Technologies (FarEastCon)*.

Quinlan, S., & Khatib, O. (1993). Elastic Bands: Connecting Path Planning and Control. *Proceedings IEEE International Conference on Robotics and Automation*.

Robotics, L. (2020). *Robots empowering people.*

Roehrig, C., Röhrig, C., & Heß, D. (2019). *OmniMan: An Omnidirectional Mobile Manipulator for Human-Robot Collaboration*. www.imsl.fh-dortmund.de,

Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F., & Bertram, T. (2023). *Trajectory modification considering dynamic constraints of autonomous robots*.

S. G. Tzafestas. (2014). Introduction to Mobile Robot Control. *Elsevier*.

Salge, C., & Polani, D. (2017). Empowerment as replacement for the three laws of robotics. *Frontiers Robotics AI*, *4*(JUN). https://doi.org/10.3389/frobt.2017.00025

Seder, M., & Petrovic, I. (2007). *Dynamic window based approach to mobile robot motion control in the presence of moving obstacles*.

Shneier, M., & Bostelman, R. (2015). *Literature Review of Mobile Robots for Manufacturing*. https://doi.org/10.6028/NIST.IR.8022

Sick | Sensor Intelligent. (n.d.). Https://Www.Sick.Com/Us/En/Catalog/Products/Safety/Safety-Laser-Scanners/S300-Expert/S30b-2011gb/p/P295845.

Smith, R. N., Schwager, M., Smith, S. L., Jones, B. H., Rus, D., & Sukhatme, G. S. (2011). Persistent ocean monitoring with underwater gliders: Adapting sampling resolution. *Journal of Field Robotics*, *28*(5), 714–741. https://doi.org/10.1002/rob.20405

Takahashi, M., Suzuki, T., Shitamoto, H., Moriguchi, T., & Yoshida, K. (2010). Developing a mobile robot for transport applications in the hospital domain. *Robotics and Autonomous Systems*, *58*(7), 889–899. https://doi.org/10.1016/j.robot.2010.03.010

Tomé de Andrade Silva, M. (2015). *Simulação e estudo da viabilidade económica de um sistema AGV para movimentação de resíduos numa empresa fornecedora da indústria automóvel*. Universidade do Minho - Escola de Engenharia.

Vadakkepat, P., Tan, K. C., & Ming-Liang, W. (2000). *Evolutionary Artificial Potential Fields and Their Application in Real Time Robot Path Planning*.

Viswanadham, N. (2002). The past, present, and future of supply-chain automation. *IEEE Robotics & Automation Magazine*, *9*(2), 48–56.

W. Geng, & F. Guangrui. (2017). Vision-Based Autonomous Docking and Re-charging System for Mobile Robot in Warehouse Environment. *International Conference on Robotics and Automation Engineering*.

Wallén, J. (2008). *The history of the industrial robot*. Linköping University Electronic Press.

Warren, C. W. (1989). Global path planning using artificial potential fields. *International Conference on Robotics and Automation*.

Wise, M., Ferguson, M., King, D., Diehr, E., & Dymesich, D. (n.d.). *Fetch & Freight: Standard Platforms for Service Robot Applications*.

Wurman, P. R., DAndrea, R., & Mountz, M. (2008). *Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses*.

X. Li, X. Liang, J. Fan, & S. Jia. (2018). Position-Based Visual Servo Control of Intelligent Wheelchair/bed Docking. *IEEE International Conference on Information and Automation (ICIA), Wuyishan, China*.

Zhang, M., Xu, R., Wu, H., Pan, J., & Luo, X. (2023). Human–robot collaboration for on-site construction. In *Automation in Construction* (Vol. 150). Elsevier B.V. https://doi.org/10.1016/j.autcon.2023.104812